كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING

الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

Public Transport Launch Center

Management System

Senior project 1

Prepared by:

Ahmad Al kzhaly

Ahmad Al Hafni

Under the supervision of:

Eng. Ranya Rajab

Eng. Mohamad Al Balky

06/01/2026

Dedication

To those who cleared the thorns from my path to pave the way for my journey of knowledge.. To my dear and generous parents..
To my esteemed teachers who never withheld knowledge or guidance from us..
To my colleagues and friends, and to everyone who contributed with a footstep or an idea in accomplishing this programming work..
We dedicate this humble effort to you.

مستخلص المشروع:

يعاني قطاع النقل العام البري بين المحافظات من عشوائية إدارية واعتماد كلي على المعاملات الورقية التقليدية، مما يؤدي إلى مشاكل في تنظيم المواعيد، تلاعب بأسعار التذاكر، وضياع شكاوى المواطنين. يقدم هذا المشروع حلاً برمجياً متكاملاً لأتمتة وحوكمة "مراكز انطلاق الحافلات" (الكراجات الموحدة).

تم بناء النظام باستخدام هندسة برمجية متطورة تعتمد على **معمارية الخدمات المصغرة** (Microservices Architecture) عبر بيئة عمل .NET 8 للخدمات الخلفية، وإطار عمل React لبناء واجهات المستخدم، مع عزل قواعد البيانات لكل خدمة باستخدام SQL Server وحزمها داخل حاويات Docker. يوفر النظام واجهات متزامنة تخدم ثلاثة أطراف: إدارة الكراج لضبط طوابير الانطلاق والخطوط، السائقين لمتابعة الرحلات، والمواطنين لحجز المقاعد وتقديم الشكاوى الفورية المربوطة بالرحلة. تم إخضاع النظام لاختبارات أداء وضغط مكثفة لضمان استقرار البوابة البرمجية وسرعة الاستجابة تحت ظروف الضغط العالي.

Abstract

Inter–provincial public transportation centers face significant operational challenges due to manual paper–based management, resulting in scheduling inefficiencies, ticket price manipulation, and unhandled passenger complaints. This project proposes a comprehensive software solution to automate and govern "Public Transport Launch Centers" (Unified Terminals).

The system is engineered using a modern **Microservices Architecture** via **.NET 8** for the backend services and **React** for building responsive user interfaces. Each service manages an independent **SQL Server** database, fully containerized using **Docker**. The platform integrates three synchronized interfaces to serve key stakeholders: Garage Administrators (to control dynamic line queuing), Bus Drivers (to track assigned shifts), and Citizens (for digital ticket booking and real–time complaint submission tied directly to trip data). To ensure system reliability and high availability, the architecture was subjected to rigorous performance and load testing, verifying optimal response times under heavy concurrent user traffic

Content

Diagram

Tables

Chapter One Introduction and Project Definition

Introduction to the Project Topic

1-Overview:

The starting point will be public transportation in the Launch Center system: the project will begin with the city of Damascus and Launch Centers (public garages), then expand to all cities and governorates of Syria. The Launch Centers.

The project aims to design and develop an integrated digital system for managing public transportation, including scheduling, handling public transportation operations, organizing trips, tracking buses, addressing passenger complaints, and providing analytical reports.

2-Project Justifications

To address organizational problems due to reliance on administrative procedures following a hierarchical sequence, using manual mail.

To reduce manipulation of fare prices and prevent exploitation.

Increasing reliance on public transportation requires smart systems that help improve the user experience, reduce congestion, and enhance operational oversight. The project addresses issues resulting from manual management, such as delayed trips and poor resource distribution.

3-Project Objective

To create an integrated information system for the Launch Center that facilitates daily management operations, and provides senior management with data-supported tools for analysis and decision-making.

Developing an electronic platform that facilitates travel between cities

Improving the reality of public transportation services

4-Project Scope Assumptions and Restrictions

4.1. Project Scope

This section precisely defines the technical and functional components included in the system, and what falls outside its scope:
What is Included:
Web Management Platform: A web-based control panel platform designed for the garage manager and staff to manage buses, drivers, trips, schedules, and handle complaints.

Citizen Responsive Web/Mobile Application: A responsive user interface designed for citizens to view available trips, book tickets, and file complaints.

Microservices Backend Core: Building a distributed and stable backend system that links all these operations together through a unified application gateway (API

Gateway).

What is Excluded:

The system does not include real financial payment gateways linked to local banks at present, but rather focuses on booking logic and documentation.

It does not include real-time bus tracking via GPS on maps; instead, trip status is updated programmatically by the driver or based on the trip start/delay/finish time.

4.2. Restrictions / Constraints

These are the limits and challenges imposed on the team that the reader should consider when evaluating the final product:

Time Constraint: Adhering to a strict timeline to complete all stages of analysis, design, programming, and performance testing before the graduation project discussion.

Lack of Real-world Data: Difficulty in obtaining immediate and accurate data for trips and drivers from public transportation companies due to the absence of digital systems, which prompted the team to rely on virtual data and software simulation (Mock Data) approximating reality.

Resource and Budget Limitations: The project was developed and performance testing (k6) was conducted using the personal hardware available to the team and free, open-source operating environments without external funding for large cloud servers.

4.3. Assumptions

These are the premises and hypotheses on which the team based their system design and considered available:

Basic Computer Literacy: It is assumed that the garage manager, staff, and citizens have at least basic knowledge of using browsers and basic web applications to operate the system.

Continuous Internet Connectivity: The system assumes that the starting center (central garage) and users have stable internet connectivity to ensure real-time operation of microservices and synchronized updating of ticket bookings and complaints.

Hardware Availability: It is assumed that personal computers or screens are available in the garage to run the manager's control panel, and that citizens possess smartphones or devices capable of accessing the system interfaces.

5-The project's contribution to Knowledge and the field F(What is New)

A locally unstudied case: complete and comprehensive digital automation of 'Interprovincial Bus Departure Garages' operations locally, and transforming them from the traditional paper system to a digital system that prevents price manipulation and congestion.

Integration of modern technologies and architectures: building the entire system on the modern microservices architecture via .NET 8, Docker, and a unified application gateway, which is a very advanced technical standard compared to similar local systems.

5- Research Methodology

. The project follows an analytical approach in studying requirements and designing the system .

6- Organizational Document Structure –

Report Structure and Objectives
The report is organized into five main chapters covering all aspects of the project:
Chapter One: Introduction and Project Definition

- **Introduction to the Project Topic**
- **1- Overview**
- **2- Project Justifications**
- **3- Project Objective**
- **4- Project Scope, Assumptions, and Restrictions**
- **5- Research Methodology**
- **6- Organizational Document Structure**
- **7- Basic Definitions and Concepts**

Chapter Two: Project Development and Management Methodology

- **Project Definition**
- **Project Objective**
- **Scope**
- **1- Development Method Used (Iterative Incremental)**
- **2- Work Plan**
- **Project Charter**
- **7- Roles and Responsibilities**
- **8- Risk Management**

- 9- Project Quality Management

Chapter Three: Reference Study

- 1- Similar Applications
- 2- Comparison Table Between Systems

Chapter Four: Analytical Study

- 1- Feasibility Study
- 2- Requirements Elicitation
- 3- Stakeholders
- 4- Functional Requirements
- 5- Nonfunctional Requirement
- 7- System Description of Features (High Level Use Case Diagram)
- 7-1- Main Use Case Diagrams
- 7-2- Use Case Descriptions
- 7-3- Activity Diagrams
- 7-4- Sequence Diagrams
- 7-5- Test Case Descriptions

Part Three: Design and Architecture - Chapter Five

- 5.1 High-Level System Design
    - 5.1.1 System Block Diagram
    - 5.1.2 Architectural Pattern
    - 5.1.3 System Architecture Description
- 5.2 Database Design
    - 5.2.1 Entity Relationship Diagram (ERD)
    - 5.2.2 Relational Schema

7-Basic Definitions and Concepts

This section provides brief, foundational definitions of the core architectural patterns, engineering methodologies, and technical concepts utilized throughout the development of the Public Transport Launch Center Management System.

Microservices Architecture (معمارية الخدمات المصغرة): An architectural pattern that structures an application as a collection of small, autonomous, and loosely coupled services. Each service (e.g., `AuthService`, `BookingService`) is self-contained, runs its own process, and manages its own unique database.

API Gateway (بوابة واجهة برمجة التطبيقات – **Ocelot**): A server that acts as a single entry point for all client requests. It is responsible for routing requests to the appropriate backend microservices, handling authentication, and enforcing security policies.

Containerization & Docker (الحاويات والدوكر): A technology used to package software code along with all its dependencies (libraries, frameworks, configurations) into a single container. This ensures that the microservices run consistently across different environments (Windows, Linux, etc.)..

Clean Architecture (العمارة النظيفة): A software design pattern that promotes the separation of concerns by dividing the codebase into independent layers (Domain, Application, Infrastructure, and WebAPI). It ensures that the core business logic does not depend on databases or external frameworks.

Object-Relational Mapping – ORM / EF Core (مخطط الكائنات العلاقية): A programming technique and tool (Entity Framework Core) that enables developers to interact with relational databases (`SQL Server`) using object-oriented code (`C#`) instead of writing raw SQL queries.

Single Page Application – SPA / React (تطبيقات الصفحة الواحدة): A web application or website that interacts with the user by dynamically rewriting the current web

page with new data from the web server, instead of the default method of a browser loading entire new pages.*).*

Load and Performance Testing (اختبارات الأداء والتحمل**):** A technical validation practice (using tools like `k6`) that simulates high numbers of concurrent users accessing the system simultaneously to measure response times, detect bottlenecks, and ensure application stability under stress.

Chapter Two: Project Development and Management Methodology

1-Project Definition

•In city centers and service locations, especially for state officials, due to its great role in completing their tasks, as it depends on papers and old tools, it has been congested, and it is not a luxury but an urgent need.

2-Project objective

• It enhances the quality of life for citizens in the Damascus Governorate. The project includes designing and implementing a website to manage public transportation (buses) to and from the governorates. The main goal is to provide safe, reliable, and easily accessible transportation services to citizens, which contributes to reducing traffic congestion, improving the quality of transportation service, and enhancing

3-Scope

• Public Transportation

This system offers many important functions that help facilitate the management of public transportation and provide information. It allows the manager to automate transportation operations accurately, in terms of their status, quality, and requirements. It enables adding, deleting, or modifying trips, buses, or lines, managing tickets for each transportation line, and inquiring about schedule or route changes. These operations make public transportation centers useful.

This system includes many important functions, allowing citizens (passengers) to know the number and times of trips, use it easily, and don't forget the drivers, who can communicate with the garage manager to report an issue or file a complaint online, making the process more flexible. This system reduces traditional paper-based methods and processes.

4-Development Method Used (Iterative Incremental)

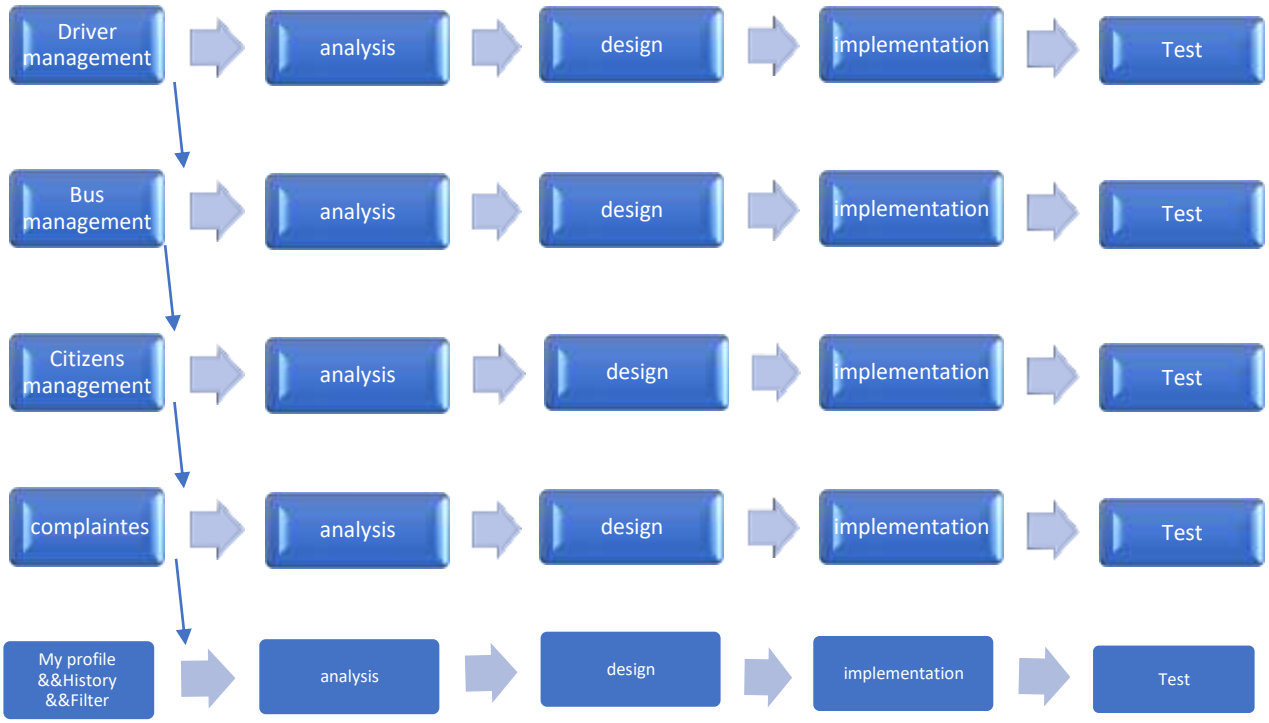
The incremental model is a development approach in which the system is divided into small functional parts (increments), with each part being developed and delivered sequentially until the system is complete.

1. Early and continuous delivery

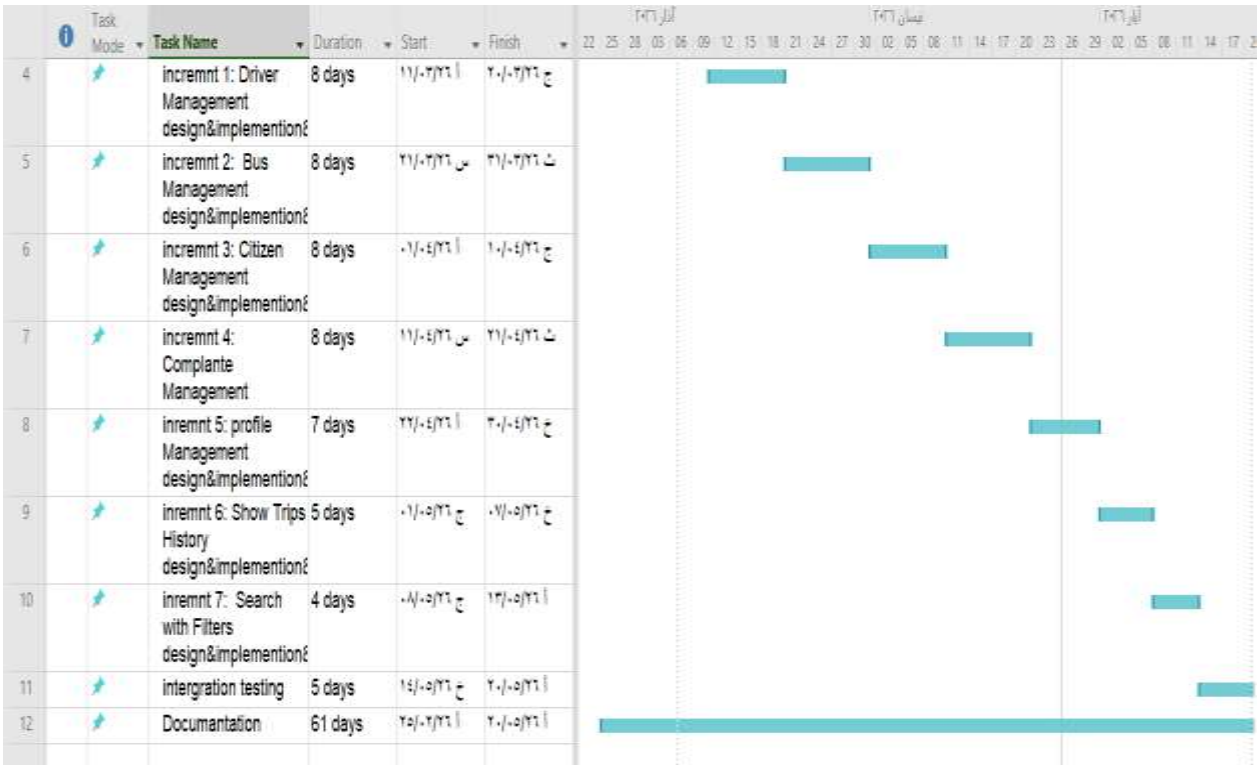
   Flexibility and change management

   Risk management

2. Parallel work



5-Work plan



	Task Mode	Task Name	Duration	Start	Finish
4		incremnt 1: Driver Management design&implemention&	8 days	١١/٠٣/٢١ أ	٢٠/٠٣/٢١ ج
5		incremnt 2: Bus Management design&implemention&	8 days	٢١/٠٣/٢١ س	٣١/٠٣/٢١ ث
6		incremnt 3: Citizen Management design&implemention&	8 days	٠١/٠٤/٢١ أ	١٠/٠٤/٢١ ج
7		incremnt 4: Complante Management	8 days	١١/٠٤/٢١ س	٢١/٠٤/٢١ ث
8		inremnt 5: profile Management design&implemention&	7 days	٢١/٠٤/٢١ أ	٣٠/٠٤/٢١ ج
9		inremnt 6: Show Trips History design&implemention&	5 days	٠١/٠٥/٢١ ج	٠٧/٠٥/٢١ ج
10		inremnt 7: Search with Filters design&implemention&	4 days	٠٨/٠٥/٢١ ج	١٣/٠٥/٢١ أ
11		intergration testing	5 days	١٤/٠٥/٢١ ج	٢٠/٠٥/٢١ أ
12		Documantation	61 days	٢٥/٠٣/٢١ أ	٢٠/٠٥/٢١ أ

6-Project charter

Project title	Public transport launch center management system
Project start date	11/3/2026
Project finish date	20/5/2026
Project manger Project objectives	Automating and developing transportation and public transit processes in order to manage transport lines and work on organizing the operations conducted at the transport center (garage)
Approach	1- Define the project scope and objectives. 2- Accurately analyze the system specifications and requirements. 3- Design and analyze the system architecture. 4- Develop the first phase with a focus on access permissions. 5- Test the functions of the first phase and verify its permissions. 6- Second phase: Integrate the system with the state's systems and institutions. 7- Conduct tests on the second phase. 8- Document all phases and their results.

Table 1 project chater

7-Roles and Responsibilities

Name	Rule	Responsibilities
Eng.Rania Rajab Eng .Mohamad Al Balky	supervisor	Highly project management
Mohamad alnasser	Developer	Frontend Development
Ahmad Al Hafny	Developer	Back end
Ahmad Al kzhaly	Developer	Analyst and tester

Table 2 responsibilities

8- Risk management

Risk Title	Risk Description	Raised Date	Tracking Frequency	State	Impact	Mitigation Plan
Delay in the schedule	Failure to adhere to the schedule due to development complexities or lack .of resources	2026/3/05	weekly	Active	High	Dividing the project ,into increments using tools like a Gantt Chart, with prioritization of .tasks
Conflict in task distribution	Tasks are accumulating on .one team member	2026/3/05	weekly	Active	Medium	.periodic meetings
Difficulty Learning new techniques	The team is slow in mastering .NET Docker, which .delays delivery	2026/3/10	weekly	Active	High	Allocating weekly educational .sessions
Problems in interface integration	Conflict between Frontend (React) and Backend (.NET) during .development	2026/3/10	weekly	Active	High	Frequent ,integration testing using detailed API documentation, and continuous coordination among .team members
Shortage Experts Testing	The appearance of undetected errors due to the absence of a specialized laboratory in the .team	2025/04/01	weekly	Active	High	Applying manual tests (Manual Testing) for each feature and assigning a member to understand the .basics of testing

Table 3 risk management

9- Project Quality Management

To ensure the delivery of a highly reliable software product, the following procedures were adopted:

* ***Code Review***: Periodically reviewing the code to ensure it follows best practices (Clean Code).

* ***Preliminary Testing:*** Conducting unit tests for the software logic related to route calculations and bus definitions.

* ***Acceptance Criteria:*** Defining precise criteria for each feature before considering it "completed" to ensure it is free of logical errors.

Project Management Tools Used

A set of digital tools was relied upon to organize workflow and document tasks:

* ***GitHub Projects:*** to manage programming tasks and distribute them via a Kanban board.

* Google Sheets: to document spreadsheets, technical comparisons, and Gantt charts.

Chapter3 Reference study

1- Similar applications

"Five leading transportation platforms were selected: three global systems focusing on public transit and navigation, and two regional/global applications focusing on on-demand private ride-hailing."

1- Lyft

2- Carem

3- Moovit

4- Transit App

5- Citymapper

1.1-Lyft

• A transportation service app that allows users to request private cars to take them from one place to another. It was founded in 2012 in the United States, offering the following features:

1. Individual transportation service that provides private cars to transport individuals from their location to their destination quickly and easily.
2. Shared ride service that allows users to share the ride with other passengers, reducing the cost.
3. Provides larger cars that accommodate more passengers, ideal for families or groups.
4. Luxury car service that offers a premium transportation experience with professional drivers.

1.2–Careem

• Careem was founded in Dubai, United Arab Emirates, by Mudassir Sheikha, Abdullah Elyas, and Tariq Hashmi.

1. Diverse Services: It offers a variety of services, including transportationservices.
2. Transportation Service: It provides private cars for transporting individuals, in addition to options such as Careem GO and Careem XL for groups.
3. Fast Delivery: It offers the Careem NOW service, which allows food to be delivered quickly from restaurants to homes.

1.3–Moovit

1. A global app that provides trip planning.
2. Supports real–time bus tracking.
3. Provides text and audio data for the stations.
4. Relies on artificial intelligence algorithms to reduce waiting time.

1.4–Transit App

1. An app that provides maps and transportation routes
2. Supports real–time notifications for line changes
3. Offers estimated arrival times
4. Used by millions of users in Europe and America

1.5-CityMapper

1. Supports trip planning by integrating more than one mode of transport (bus – microbus – metro)
2. Interactive user interface
3. Comparison of time and cost between transportation options
4. Provides usage reports

2-Comparison table between systems

SYSTEM OVERVIEW

Feature Comparison — Our System vs. Market Leaders

Feature	Moovit	Transit	Citymapper	Our System★
General Trip Info	✓	✓	✓	✓
Real-time Tracking	✓	✓	✓	X
Multi-modal Planning	✓	✓	✓	—
Reservations & Tickets	✓	✓	✓	—
Garage Operations Mgmt	—	—	—	✓★
Driver & Bus Management	—	—	—	✓★
Delay Notifications	✓	✓	✓	✓
Admin Interface	—	—	—	✓★
Data Analytics	✓	✓	✓	X
Map Integration	✓	✓	✓	X

6

3.Modern technologies and frameworks in the field

The field of distributed software system development and intelligent transportation management witnesses significant reliance on frameworks and technical architectures that ensure high efficiency, scalability, and service independence. In this project, the most prominent of these modern technologies were employed:

.NET 8 Environment: The latest framework from Microsoft for building advanced backend services, characterized by high performance, strong security, and full support for building Web APIs.

Microservices Architecture: The modern approach in building large systems by dividing the application into small, independent services that are easy to maintain and develop, instead of traditional monolithic systems.

React Framework: One of the most popular frontend frameworks for developing interactive, responsive user interfaces, relying on reusable components.

Docker Containers Technology: The modern standard environment for packaging software services and ensuring their consistent and stable operation across different operating systems (Windows, Linux), facilitating deployment and instant updates.

Ocelot API Gateway: A modern tool for securing a unified entry point for all user requests, intelligently routing them to the required microservice while handling security and digital verification.

6 Testing and Load Tool: One of the leading modern frameworks in quality engineering, used to simulate intense concurrent requests and test the stability of the software infrastructure under high stress.

4-Theoretical Framework and Reference Studies (Theoretical Framework)

The project relies in its computational and engineering structure on a set of well-established theories and scientific principles in software engineering:

Microservices & API Gateway Architecture: A computational theory that relies on dividing complex systems into completely independent and loosely coupled units, with requests directed to them through a single entry point (Ocelot) to manage routing and security centrally.

Clean Architecture Principles: An engineering theory that relies on the separation of concerns through circular layers, where the core business logic is at the center, completely isolated and independent from databases or external frameworks.

Digital Security Principles (JWT & Cryptography Authentication): Relying on the stateless token protocol based on cryptography to verify user identities and their roles without overloading the server.

Database-per-Service Pattern: A database design theory that ensures data non-interference and independence, where each service has a completely isolated SQL Server database.

5. Modern and Future Directions in the Field (Future Trends in the Field)

This project aligns with modern global trends and can be developed in the future to integrate with the following emerging technologies:

Edge AI & IoT: To process data from bus sensors and track their arrival and departure times instantly on the "edge" without consuming internet packages, which improves the accuracy of trip schedules.

Generative AI & LLMs: Integrating large language models into the "complaints system" to automatically classify citizens' complaints, analyze them, generate smart instant responses, and automatically detect attempts at price manipulation or violations.

MLOps: To build and deploy predictive models that anticipate peak times and passenger congestion at the terminal based on historical trip and seasonal data, and continuously and automatically update these models.

Chapter 4

Analytical study

1-Feasibility study

2-Requirements Elicitation

We carried out the process of deriving the project requirements through the theoretical study of the project, as well as through repeated meetings with the stakeholders (Daraa Garage Manager) and the assistance of our supervisor. The requirements were also derived and collected through the help of studying similar projects.

3-stakeholders

1- Bus Drivers :responsible for updating trip ,status , completing routes, and repor ting issues during transit .

2- Garage manager :manages drivers , buses , routes ,trips ,and handles

Citizen complaints and and operations .

3- Passengers : citizens who book tickets , track trips , view schedules, and submit complaints .

4-functctional requirements

Table 4 function requrment

ID	Titel
FR1	Garage manger can be add new driver
FR2	:Garage manger can be update driver
FR3	:Garage manger can be delete driver
FR4	: Garage manger can be show driver
FR5	: Garage manger can be add new vehicle
FR6	; Garage manger can be update information vehicle
FR7	: Garage manger can be delete vehicle
FR8	: Garage manger can be show vehicle
FR9	Garage manger can be add new citizen
FR10	: Garage manger can be update information citizen
FR11	: Garage manger can be delete citizen
FR12	: Garage manger can be show citizen
FR13	: driver and citizen can be send(submit) complaints
FR14	: driver and citizen can be show complaints
FR15	Garage manger can be view user complaints
FR16	: Garage manger can be respond complaints
FR17	; Drive can be view trip history
FR18	: citizen can be view booking history
FR19	:user can be view profile
FR20	: user can be edit profile
FR21	: all actors can be search with filter

;

5-Nonfunctional requirement

ID	Category	Requirement Description	Technical Specification & Implementation	Verification / Testing Method
NFR–1	Security	**Authentication** Ensure only verified users can log into the system dashboards and portals.	Implement an isolated AuthService handling **JWT tokens** with password encryption using a cryptographically secure hashing algorithm.	Functional manual checks via Postman Login endpoints; Token validation code review.
NFR–2	Security	**Authorization & Access Control** Enforce strict separation of system operations based on actor roles (Garage Manager, Bus Driver, and user	**Role–Based Access Control (RBAC)** enforced globally at the Ocelot API Gateway level and locally at each microservice endpoint.	Automated checking of response codes (e.g., 401 Unauthorized / 403 Forbidden) during testing.
NFR–3	Security	**Data Sanitization & API Security** Protect system APIs against malicious inputs, credential intercepting	Enforce request validation, input sanitization, HTTPS–based token exchanges, and token expiration windows.	API vulnerability code reviews and automated request parameter fuzzing.
NFR–4	Performance	**Response Latency** Core operations must provide immediate system	Average target response time must **not exceed 2.0 seconds** for basic data–driven transactions under normal conditions.	**k6 performance testing scripts** measuring http_req_duration metrics (averages

		feedback to minimize user friction.		and 95th percentiles).
NFR–5	**Availability**	**Service Resiliency** System services must maintain operational readiness to prevent garage logistical downtime.	Adopt a **Database–per–Service pattern** with containerized independent SQL Server instances inside Docker. Failure in one domain (e.g., ComplaintService) will not drop another (e.g., TripService).	System isolation checks; runtime container failover simulations.
NFR–6	**Usability**	**Simplicity & Intuitiveness** The user experience (UX) should allow seamless interaction without formal instruction.	Streamlined booking workflow letting a citizen complete a ticket reservation easily without training.	Manual accessibility walk–throughs and stakeholder interface sign–offs.
NFR–7	**Maintainability**	**Separation of Concerns** The architectural codebase structure must easily accommodate future components.	Enforce **Clean Architecture** patterns separating structural components into 4 distinct layers: Domain, Application, Infrastructure, and API.	Structural code reviews validating dependency direction boundaries.

Table 5 nonfunctional requirement

6–Prioritization of requirements and the mechanism for their management

ID	Requirement Name	Description	Actor(s)	MoSCoW	Justification
1	Driver Management	Manage drivers (add, edit, delete, search)	Garage Manager	Must	Core system functionality
2	Bus Management	Manage buses (add, edit, delete, search)	Garage Manager	Must	Essential for operations
3	Citizens Management	Manage citizens data	Garage Manager	Must	Required for service delivery
4	Complaints Management	Handle complaints (send, respond, view)	All Actors	Must	Critical for user interaction
5	Profile Management	Edit and view user profile	All Actors	Should	Improves user experience
6	History	View system activity history	All Actors	Should	Useful but not critical
7	Search & Filter	Search and filter data	All Actors	Could	Enhancement feature
8	Mobile Application	Mobile version of the system	–	Won't	Outside the scope of the current project
9	GPS Integration	Real–time bus tracking	–	Won't	Needs additional resources

Table 6 MOSCOW

7-system description of features(High level use case diagram)

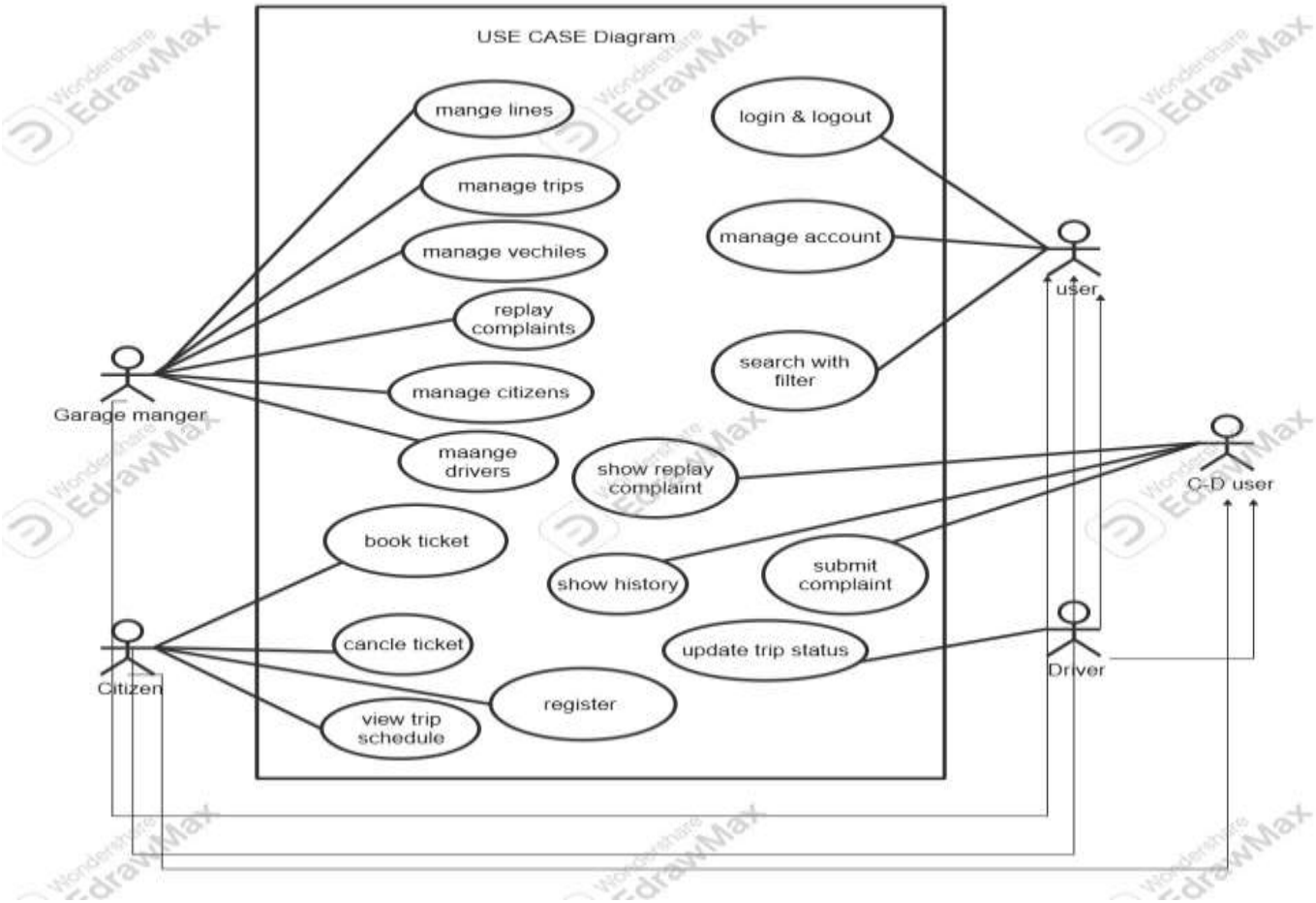


Figure 1 use case high level

7-1-Main Use Case Diagrams

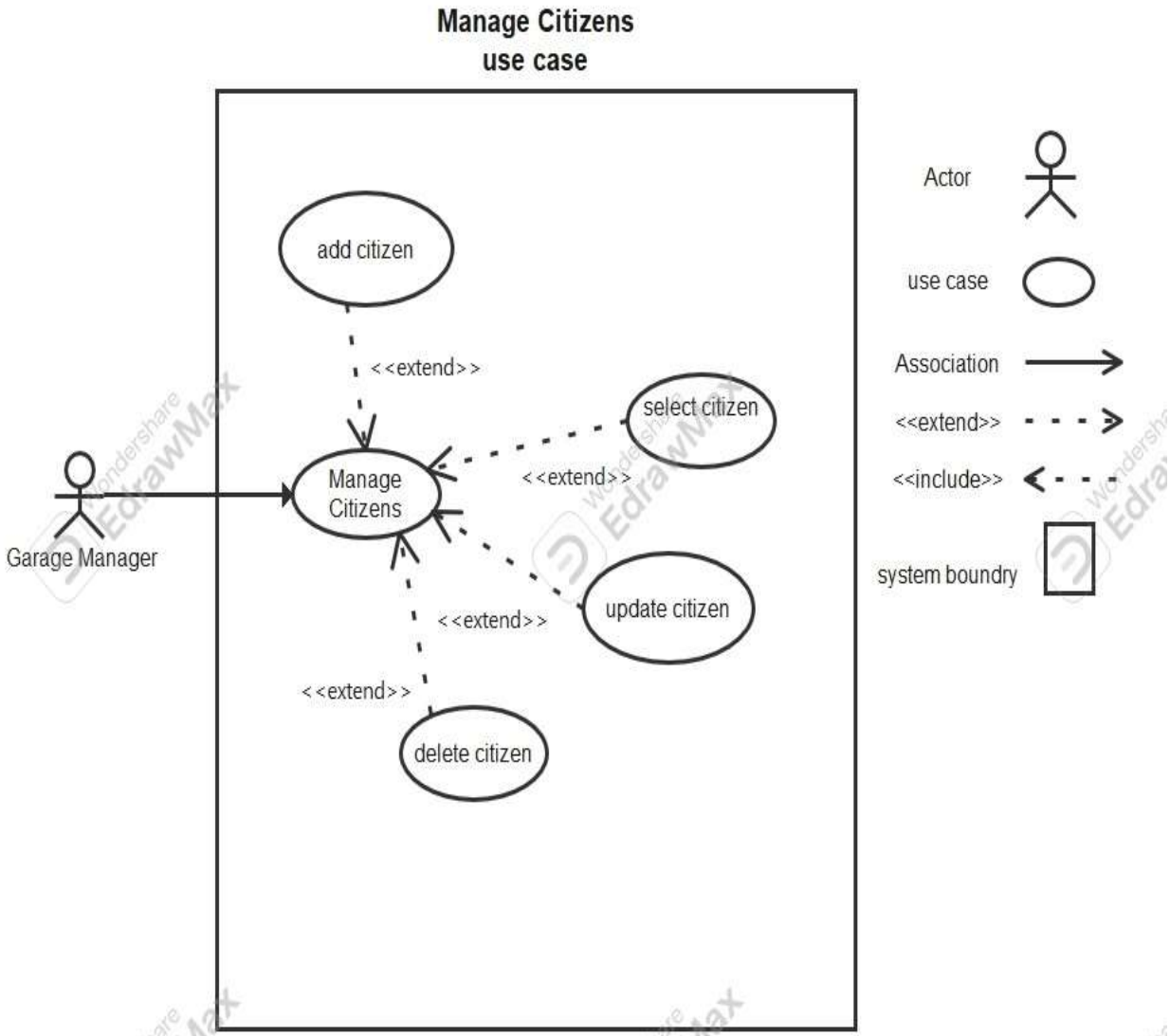


Table 7 use case mange citizens

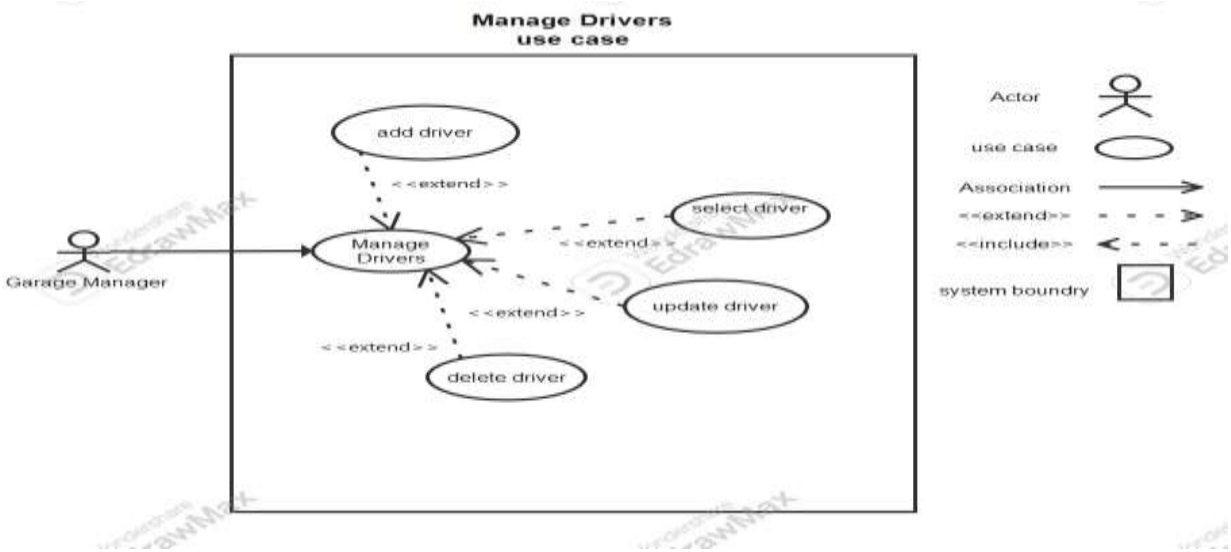


Table 8 use case mange drivers

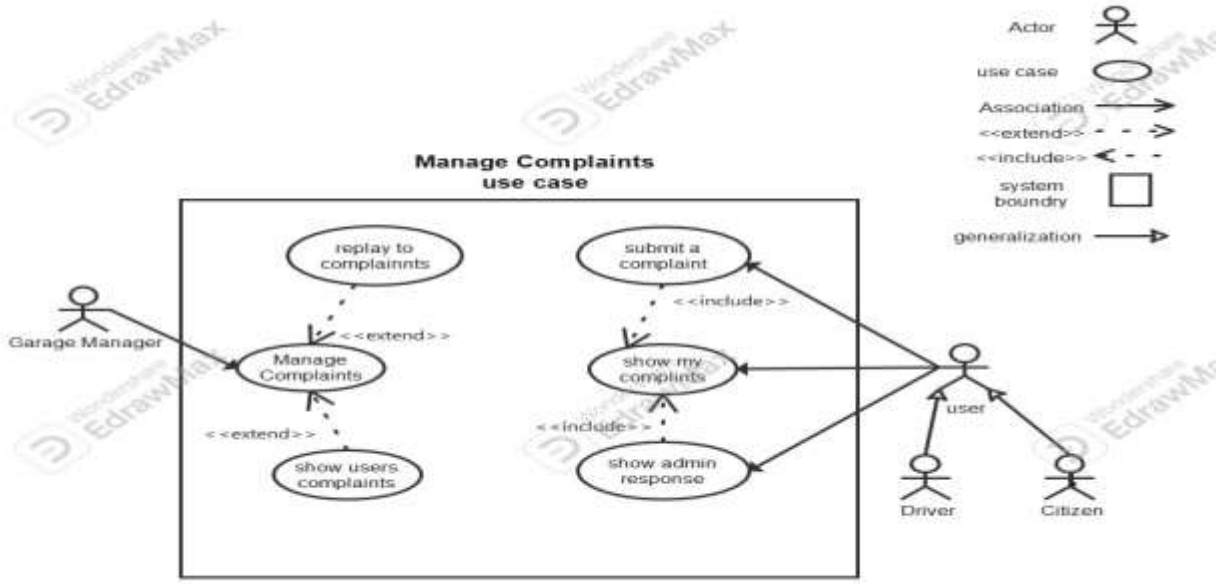


Table 9 use case mange complaints

7-2- use case Descriptions

Field	Description
Use Case ID	UC-01
Name	Add Driver
Actor(s)	Garage Manager
Preconditions	- User is logged into the system - Dashboard is loaded- User is on "Manage Drivers" page
Postconditions	- New driver is created and stored in the system - OR Error message is displayed
Main Flow	1. User clicks "Manage Drivers" 2. System displays drivers page 3. User clicks "New Driver" 4. System displays driver form 5. User enters required information: - Username, Email, Phone Number - License Number, Issuing Authority - Vehicle Plate, Model, Color - License Expiry, Registration Expiry - License Category - Initial Password 6. User clicks "Create Driver" 7. System validates data 8. System saves driver 9. System displays success message: "Driver created successfully"
Alternative Flow	A1: Cancel Operation - User clicks "Cancel"- System returns to Manage Drivers page A2: Validation Error - Invalid or duplicate data (e.g., email exists) - System displays error message (e.g., "Email already in use")

Table 10 desciption UC-01

Field	Description
Use Case ID	UC-02
Name	Update Driver
Actor(s)	Garage Manager
Preconditions	– User is logged in – User is on Manage Drivers page – Driver already exists
Postconditions	– Driver information is updated OR – No changes are made
Main Flow	1. User clicks "Edit" on a driver 2. System displays edit form with existing data 3. User modifies driver information: – Username, Phone Number – License details – Vehicle details – Expiry dates – License category 4. User clicks "Update Driver" 5. System asks for confirmation 6. User confirms update 7. System saves changes 8. System displays: "Driver updated successfully"
Alternative Flow	A1: Cancel Update – User clicks "Cancel"– System discards changes A2: Validation Error– Invalid data entered – System shows error message A3: User Rejects Confirmation – User cancels confirmation dialog– No update occurs

Table 11 desciption UC-02

Field	Description
Use Case ID	UC-03
Name	Delete Driver
Actor(s)	Garage Manager
Preconditions	- User is logged in - User is on Manage Drivers page - Driver exists
Postconditions	- Driver is deleted from the system OR - Driver remains unchanged
Main Flow	1. User clicks "Delete" on a driver 2. System displays confirmation message:"Are you sure you want to delete this driver?" 3. User clicks "Delete" 4. System removes driver from database 5. System displays: "Driver deleted successfully"
Alternative Flow	A1: Cancel Delete - User clicks "Cancel" - System keeps driver unchanged A2: Error During Deletion - System fails to delete driver - System displays error message

Table 12 descirption UC-03

Field	Description
Use Case ID	UC-04
Name	Show Driver Details
Actor(s)	Garage Manager
Preconditions	- User is logged in - User is on Manage Drivers page - Driver exists
Postconditions	- Driver details are displayed
Main Flow	1. User clicks "Show Details" on a driver 2. System navigates to driver details page 3. System displays full driver information
Alternative Flow	A1: Driver Not Found - Driver does not exist - System displays error message

Table 13 descirption UC-04

Field	Description
Use Case ID	UC-05
Name	Add Vehicle
Actor(s)	Garage Manager
Preconditions	– User is logged into the system – Dashboard is loaded – User is on "Manage Vehicles" page
Postconditions	– New vehicle is added to the system OR – Error message is displayed
Main Flow	1. User navigates to "Manage Vehicles" 2. System displays vehicles page 3. User clicks "New Vehicle" 4. System displays vehicle form 5. User enters vehicle information: – Name – Type – Capacity – Plate Number – Account Status (Active / Inactive) 6. User clicks "Create Vehicle" 7. System validates input 8. System saves vehicle 9. System displays success message: "Vehicle created successfully"

Alternative Flow	A1: Cancel Operation – User clicks "Cancel" – System returns to Manage Vehicles page A2: Validation Error – Invalid or duplicate data (e.g., plate number exists) – System displays error message

Table 14 descirption UC-05

Field	Description
Use Case ID	UC-06
Name	Update Vehicle
Actor(s)	Garage Manager
Preconditions	– User is logged in – User is on Manage Vehicles page – Vehicle exists
Postconditions	– Vehicle information is updated OR – No changes are made
Main Flow	1. User clicks "Edit" on a vehicle 2. System displays edit form with existing data 3. User updates vehicle information: – Name – Type – Capacity – Plate Number – Account Status 4. User clicks "Update Vehicle" 5. System displays confirmation message:"Confirm Update: Are you sure you want to update this vehicle?" 6. User clicks "Update" 7. System saves changes 8. System displays: "Vehicle updated successfully"

Alternative Flow	A1: Cancel Delete – User clicks "Cancel" – System keeps vehicle unchanged A2: Error During Deletion – System fails to delete vehicle – System displays error message

Table 15 descirption UC-06

Field	Description
Use Case ID	UC-06
Name	Update Vehicle
Actor(s)	Garage Manager
Preconditions	– User is logged in – User is on Manage Vehicles page – Vehicle exists
Postconditions	– Vehicle information is updated OR – No changes are made
Main Flow	1. User clicks "Edit" on a vehicle 2. System displays edit form with existing data 3. User updates vehicle information: – Name – Type – Capacity – Plate Number – Account Status 4. User clicks "Update Vehicle" 5. System displays confirmation message:"Confirm Update: Are you sure you want to update this vehicle?" 6. User clicks "Update" 7. System saves changes 8. System displays: "Vehicle updated successfully"

Alternative Flow	A1: Cancel Delete – User clicks "Cancel" – System keeps vehicle unchanged A2: Error During Deletion – System fails to delete vehicle – System displays error message

Table 16 uc-06

Field	Description
Use Case ID	UC-07
Name	Delete Vehicle
Actor(s)	Garage Manager
Preconditions	– User is logged in – User is on Manage Vehicles page – Vehicle exists
Postconditions	– Vehicle is deleted from the system OR – Vehicle remains unchanged

Main Flow	1. User clicks "Delete" on a vehicle 2. System displays confirmation message:"Confirm Delete: Are you sure you want to delete this vehicle?" 3. User clicks "Delete" 4. System removes vehicle from database 5. System displays: "Vehicle deleted successfully"
Alternative Flow	A1: Cancel Delete – User clicks "Cancel" – System keeps vehicle unchanged A2: Error During Deletion – System fails to delete vehicle – System displays error message

Table 17 uc-07

Field	Description
Use Case ID	UC-08
Name	Show Vehicle Details
Actor(s)	Garage Manager

Preconditions	– User is logged in – User is on Manage Vehicles page – Vehicle exists
Postconditions	– Vehicle details are displayed
Main Flow	1. User clicks "Show Details" on a vehicle 2. System navigates to vehicle details page 3. System displays full vehicle information
Alternative Flow	A1: Vehicle Not Found – Vehicle does not exist – System displays error message

Table 18 uc-08

Field	Description
Use Case ID	UC-09
Name	Add Citizen
Actor(s)	Garage Manager
Preconditions	– User is logged into the system – Dashboard is loaded

	– User is on "Manage Users (Citizens)" page
Postconditions	– New citizen is created and stored in the system OR – Error message is displayed
Main Flow	1. User clicks "Manage Users" 2. System displays citizens page 3. User clicks "New Citizen" 4. System displays citizen form 5. User enters required information: – Username, Email, Password, Phone – Account Status (Active / Pending / Suspended) – First Name, Last Name – National ID – Disability Status (None / Deaf / Blind / Wheelchair) – Gender, Date of Birth – City, Region, Current Address – Father Name, Mother Name – Birth Place – Card Number, Card Issue Date – Face Color, Eye Color 6. User clicks "Create Citizen" 7. System validates input 8. System saves citizen data 9. System displays: "Citizen created successfully"
Alternative Flow	A1: Cancel Operation – User clicks "Cancel"– System returns to citizens page A2: Validation Error – Invalid or duplicate data (e.g., email or national ID exists)– System displays error message

Table 19 uc-09

Field	**Description**
Use Case ID	UC-10
Name	Update Citizen
Actor(s)	Garage Manager

Preconditions	– User is logged in – User is on Manage Users page – Citizen exists
Postconditions	– Citizen information is updated OR – No changes are made
Main Flow	1. User clicks "Edit" on a citizen 2. System displays edit form with existing data 3. User updates citizen information 4. User clicks "Update Citizen" 5. System displays confirmation message:"Confirm Update: Are you sure you want to update this citizen?" 6. User clicks "Update" 7. System saves changes 8. System displays: "Citizen updated successfully"
Alternative Flow	A1: Cancel Update – User clicks "Cancel"– System discards changes A2: Validation Error – Invalid data entered– System displays error message A3: Cancel Confirmation – User cancels confirmation dialog– No update occurs

Table 20 uc-10

Field	Description
Use Case ID	UC-11
Name	Delete Citizen
Actor(s)	Garage Manager
Preconditions	- User is logged in - User is on Manage Users page - Citizen exists
Postconditions	- Citizen is deleted from the system OR - Citizen remains unchanged
Main Flow	1. User clicks "Delete" on a citizen 2. System displays confirmation message:"Confirm Delete: Are you sure you want to delete this citizen?" 3. User clicks "Delete" 4. System removes citizen from database 5. System displays: "Citizen deleted successfully"
Alternative Flow	A1: Cancel Delete - User clicks "Cancel" - System keeps citizen unchanged A2: Error During Deletion - System fails to delete citizen - System displays error message

Table 21 uc-11

Field	Description
Use Case ID	UC-12
Name	Show Citizen Details
Actor(s)	Garage Manager
Preconditions	- User is logged in - User is on Manage Users page - Citizen exists
Postconditions	- Citizen details are displayed
Main Flow	1. User clicks "Show Details" on a citizen 2. System navigates to citizen details page 3. System displays full citizen information
Alternative Flow	A1: Citizen Not Found - Citizen does not exist - System displays error message

Table 22 uc-12

Field	Description
Use Case ID	UC-13
Name	Submit Complaint
Actor(s)	Citizen, Driver
Preconditions	- User is logged into the system - Dashboard is loaded
Postconditions	- Complaint is stored in the system with status "Pending"
Main Flow	1. User clicks "Complaints" from dashboard 2. System displays complaint form 3. User enters: - Subject - Description 4. User clicks "Submit Complaint" 5. System validates input 6. System saves complaint with status "Pending" 7. System displays message: "Complaint submitted successfully" 8. System displays the complaint under "My Complaints"
Alternative Flow	A1: Cancel Operation - User leaves the page or cancels- No complaint is submitted A2: Validation Error - Missing required fields- System displays error message

Table 23 uc-13

Field	Description
Use Case ID	UC-14
Name	View My Complaints
Actor(s)	Citizen, Driver
Preconditions	- User is logged in
Postconditions	- User sees list of their complaints and responses
Main Flow	1. User navigates to "Complaints" page 2. System displays user's complaints list 3. Each complaint includes: - Subject - Description - Created Date - Status (Pending / In Review / Resolved / Rejected) - Admin Response (if available) 4. If no response exists, system shows: "No admin response yet"

Alternative Flow	**A1: No Complaints Found** – System displays empty state message (e.g., "No complaints submitted yet")

Table 24 uc-14

Field	Description
Use Case ID	UC-15
Name	View Users Complaints
Actor(s)	Garage Manager
Preconditions	– Garage Manager is logged in – Dashboard is loaded
Postconditions	– Complaints list is displayed

Main Flow	1. Garage Manager clicks "Complaints" 2. System navigates to complaints page 3. System displays all users complaints 4. Each complaint shows: – Subject – Description – Status (e.g., Pending) – Submitted By (Citizen/Driver) – Date and Time
Alternative Flow	A1: No Complaints Available – System displays message: "No complaints found"

Table 25 uc-15

Field	Description
Use Case ID	UC-16
Name	Reply to Complaint
Actor(s)	Garage Manager
Preconditions	– Garage Manager is logged in – Complaints list is displayed – Complaint exists
Postconditions	– Complaint status is updated – Admin response is saved and visible to user
Main Flow	1. Manager selects a complaint 2. Manager clicks "Response" 3. System displays response form with: – Subject (read-only) – Status (In Review / Resolved / Rejected) – Admin Response – Description 4. Manager enters response details 5. Manager clicks "Send Response" 6. System validates input

	7. System saves response and updates status 8. System displays confirmation message 9. Response becomes visible in user's "My Complaints"
Alternative Flow	A1: Cancel Response – Manager clicks "Cancel" – No response is saved A2: Validation Error – Missing or invalid input – System displays error message

Table 26 uc-16

Field	Description
Use Case ID	UC-17
Name	View Trip History
Actor(s)	Driver
Preconditions	– Driver is logged into the system – Dashboard is loaded
Postconditions	– Driver can view active trips and trip history
Main Flow	1. Driver clicks "My Trips" from dashboard 2. System displays trips page 3. System shows Active Trips including: – Trip capacity (e.g., 20 seats) – Status (Scheduled) – Departure date and time – Seats booked (e.g., 20/20) 4. Driver clicks "Show History" 5. System displays Trip History (Last 7 Days) 6. System shows finished trips with: – Trip capacity – Status (Finished) – Departure date and time – Seats information 7. Driver can click "Hide History" to return to active trips view

Alternative Flow	A1: No Trips Available – System displays message: "No trips available" A2: No History Found – System displays: "No trip history available"

Table 27 uc-17

Field	Description
Use Case ID	UC-18
Name	View Booking History
Actor(s)	Citizen
Preconditions	– Citizen is logged into the system – Dashboard is loaded
Postconditions	– Citizen can view active bookings and booking history

Main Flow	1. Citizen clicks "My Bookings" 2. System displays bookings page 3. System shows Active Bookings including: – Full Name – Booking Date – Departure Date – Number of Seats – Booking Code 4. Citizen clicks "History" button 5. System displays booking history (finished trips) 6. Each record shows: – Full Name – Booking Date – Departure Date – Seats – Status (Confirmed, Finished)
Alternative Flow	A1: No Bookings Found – System displays message: "No bookings available" A2: No History Available – System displays message: "No booking history found"

Table 28 uc-18

Use Case ID	**UC-19**
Name	View Profile
Actor(s)	Citizen, Driver, Garage Manager
Preconditions	– User is logged into the system – Dashboard is loaded
Postconditions	– User profile information is displayed

Main Flow	1. User clicks "My Profile" 2. System navigates to profile page 3. System displays user information based on role: Citizen: – Full Name, Email, Phone Number – Disability Status, Address Driver: – Full Name, Email, Phone Number – License Info (Number, Category, Authority, Expiry) – Vehicle Info (Plate, Model, Color, Registration Expiry) Garage Manager (Admin): – Full Name, Email, Role, Phone – First Name, Last Name, Gender – Date of Birth, National ID – City, Region, Account Status – Admin Level, Created Date, Last Updated
Alternative Flow	A1: Profile Not Found – System displays error message

Table 29 uc-19

Field	Description
Use Case ID	UC-20
Name	Update Profile
Actor(s)	Citizen, Driver, Garage Manager
Preconditions	– User is logged in – Profile page is displayed
Postconditions	– Profile is updated OR

	– No changes are made
Main Flow	1. User clicks "Edit Profile" 2. System enables editable fields based on role: Citizen can edit: – Full Name – Phone Number – Password (optional) – Disability Status – Address -Cannot edit Email Driver can edit: – Full Name – Phone Number – Password (optional) - Cannot edit Email - Cannot edit License Info - Cannot edit Vehicle Info Garage Manager can edit: – Full Name, Phone – First Name, Last Name – Gender, Date of Birth – City, Region - Cannot edit Email - Cannot edit Role / Admin Level 3. User modifies allowed fields 4. User clicks "Save" 5. System validates input 6. System updates profile data 7. System displays: "Profile updated successfully"
Alternative Flow	A1: Cancel Update – User clicks "Cancel"– System discards changes A2: Validation Error – Invalid input data– System displays error message

Table 30 uc-20

Field	**Description**
Use Case ID	UC-21
Name	Search with Filters
Actor(s)	User (Driver, Citizen, Garage Manager)
Preconditions	– User is logged into the system – User is on a page that supports search functionality

Postconditions	– Filtered results are displayed based on user criteria
Main Flow	1. User navigates to a system page (e.g., drivers, trips, bookings) 2. System displays data list with search/filter fields 3. User enters one or more search criteria (e.g., name, ID, date, etc.) 4. User triggers search (typing or clicking search button) 5. System validates input 6. System filters data according to entered criteria 7. System displays matching results
Alternative Flow	A1: No Results Found – System displays message: "No results found" A2: Empty Input – System displays full data list A3: Invalid Input – System displays validation error message

Table 31 uc-21

7-3-Activity Diagrams

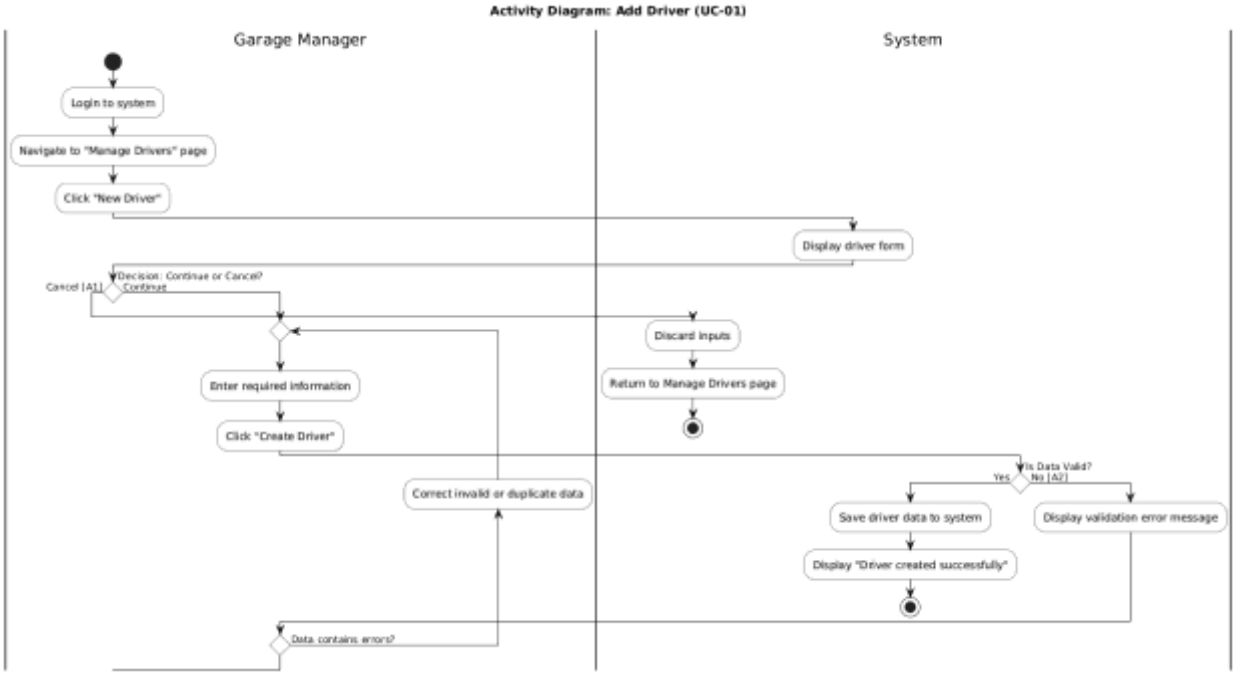


Figure 2 activity UC-1

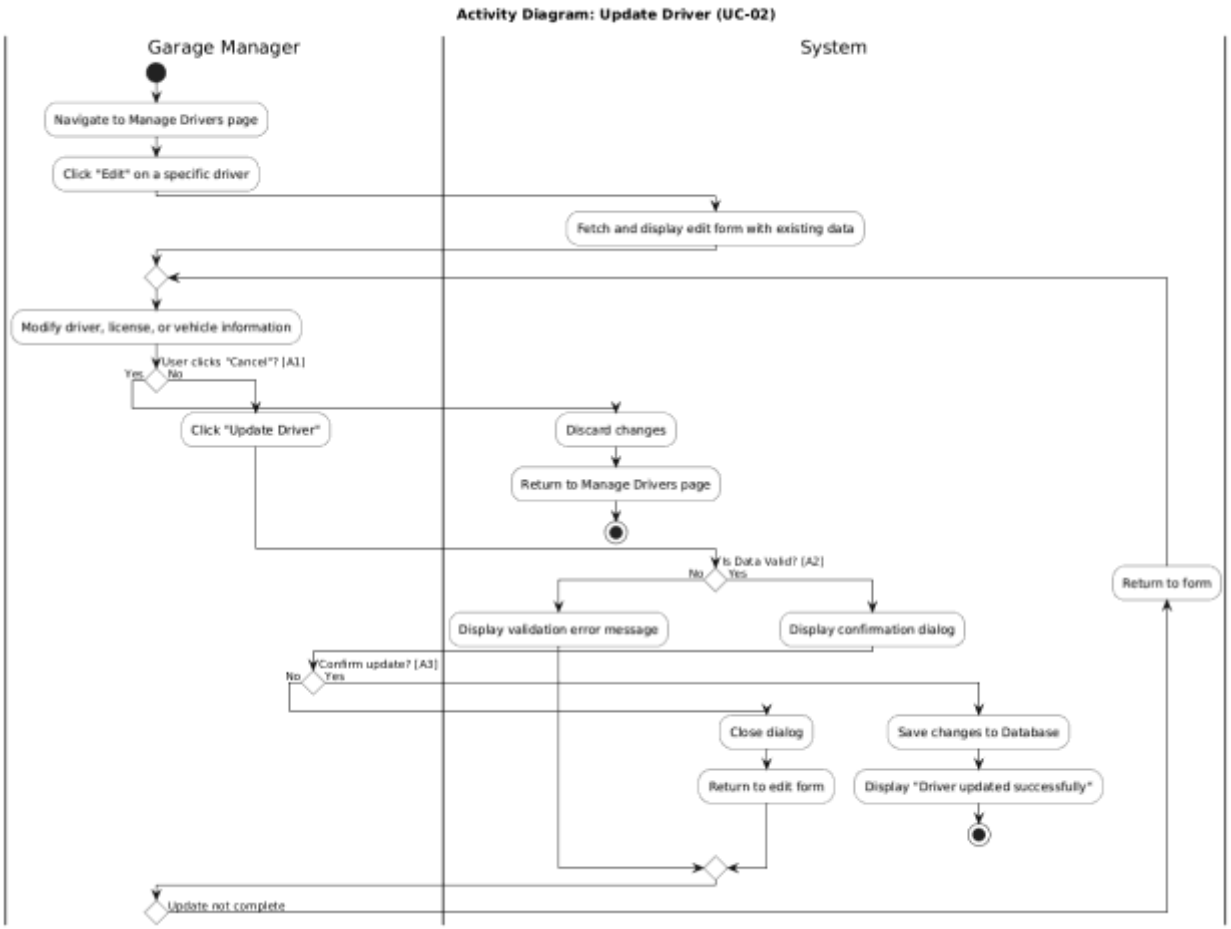


Figure 3 activity UC 2

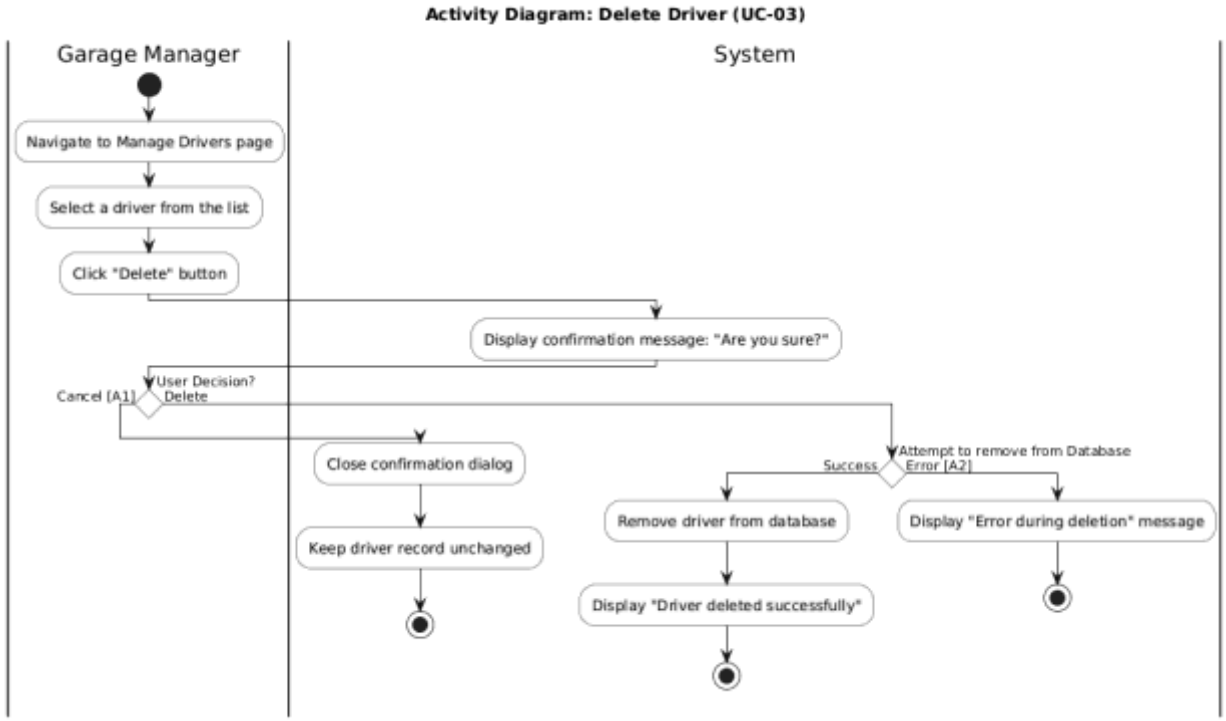


Figure 4activity UC-03

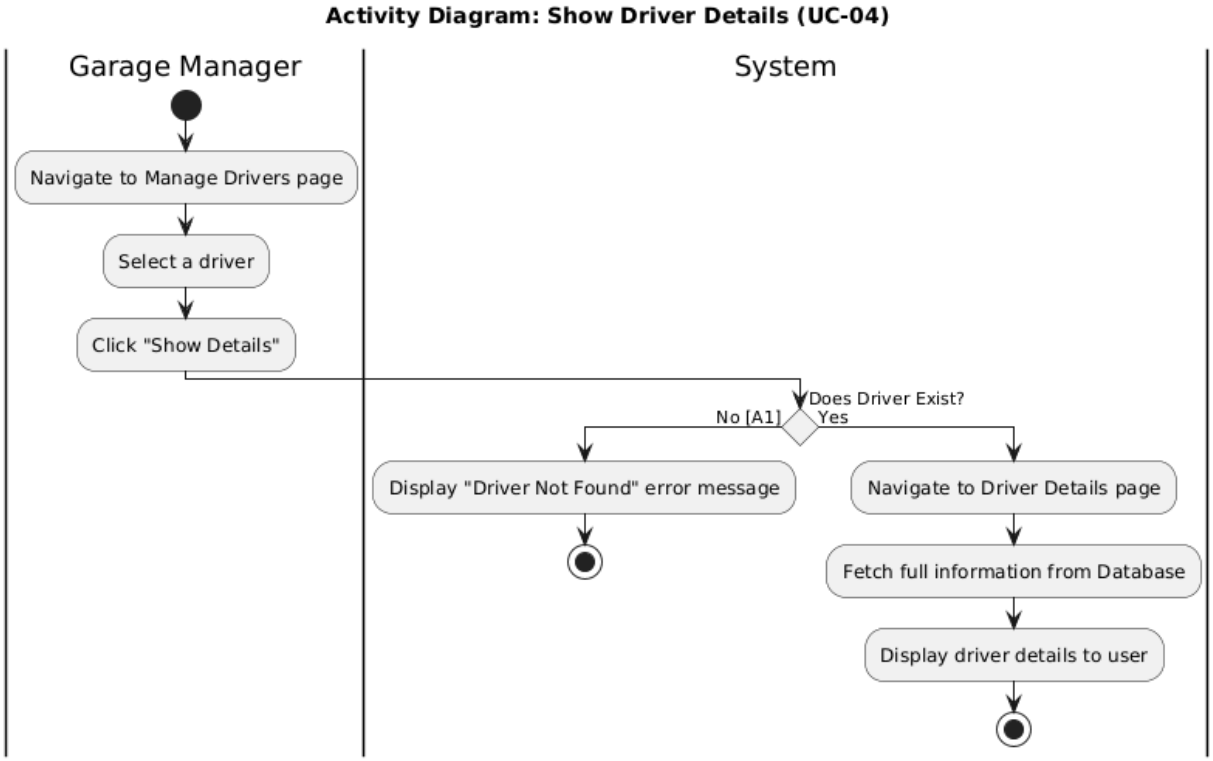


Figure 5 activity UC-4

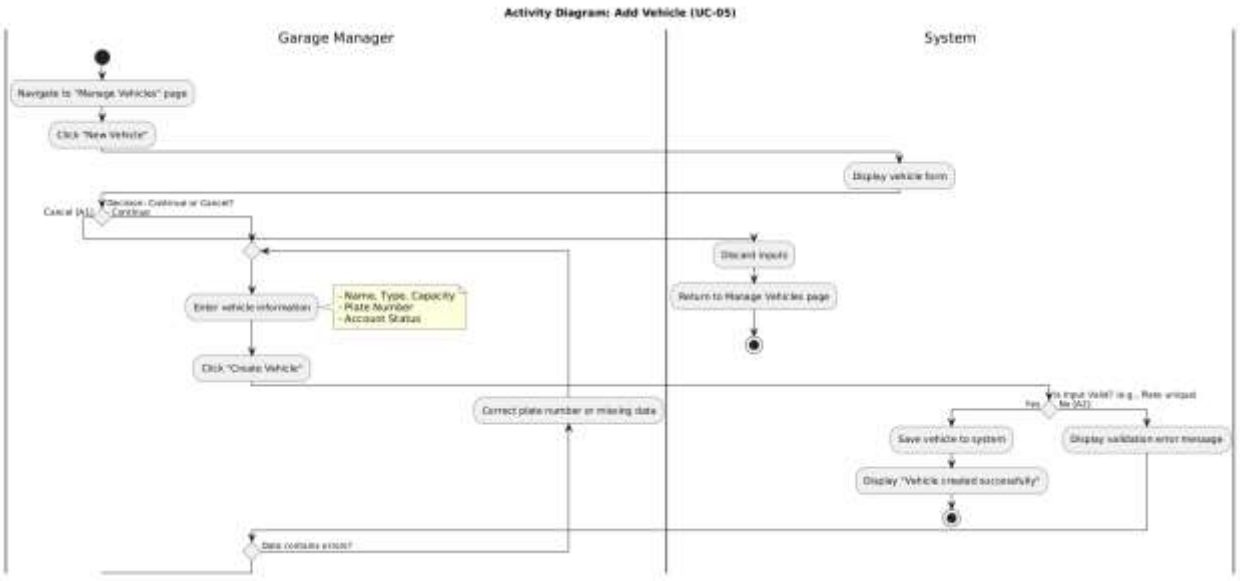


Figure 6 activity UC-05

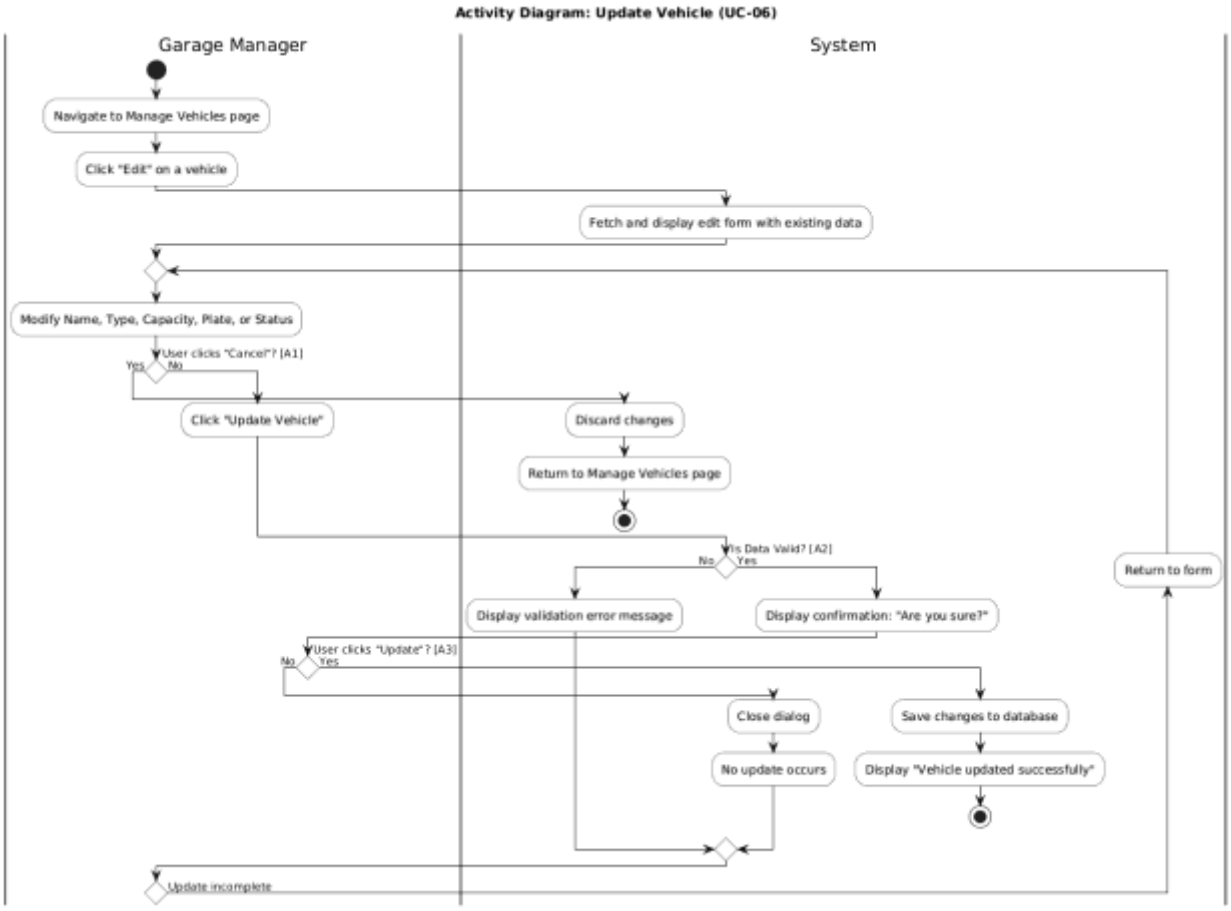


Figure 7 activity UC-06

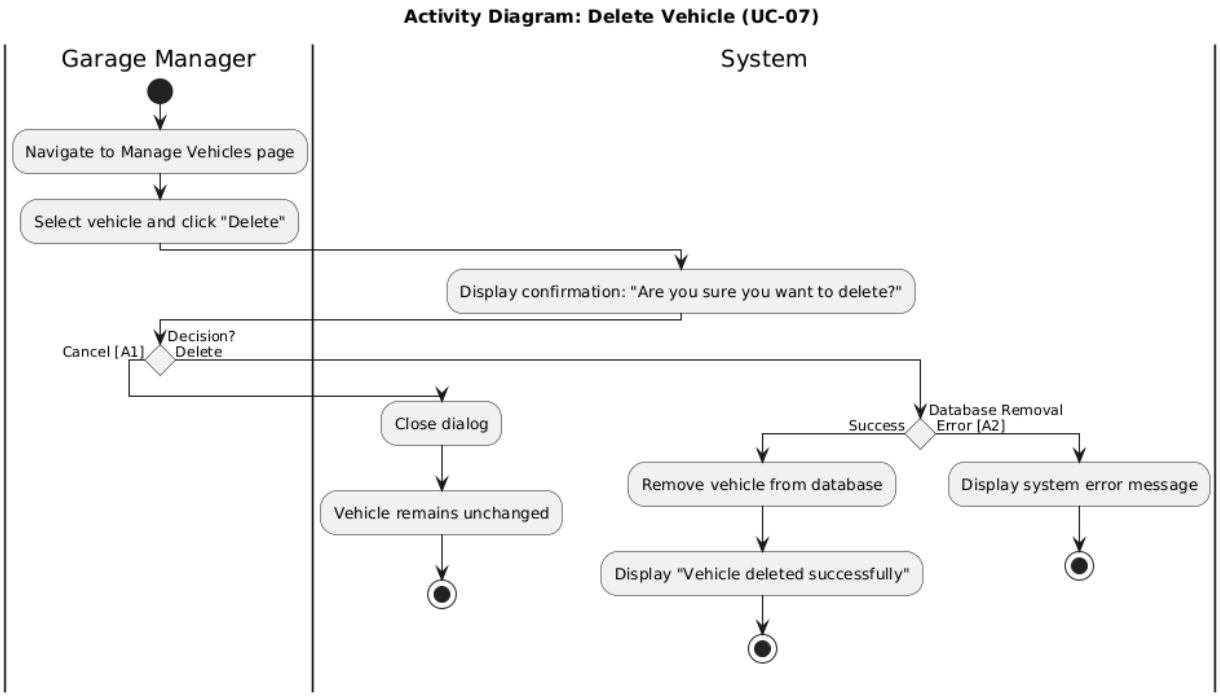


Figure 8 activity UC=7

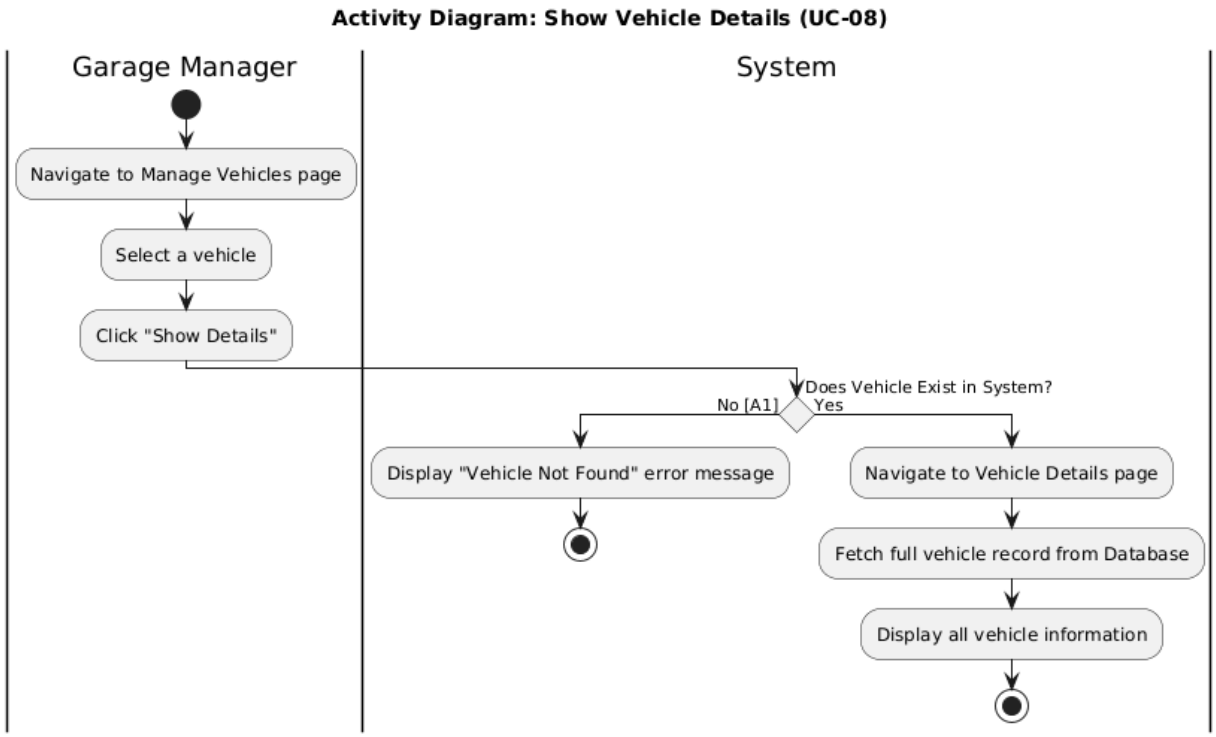
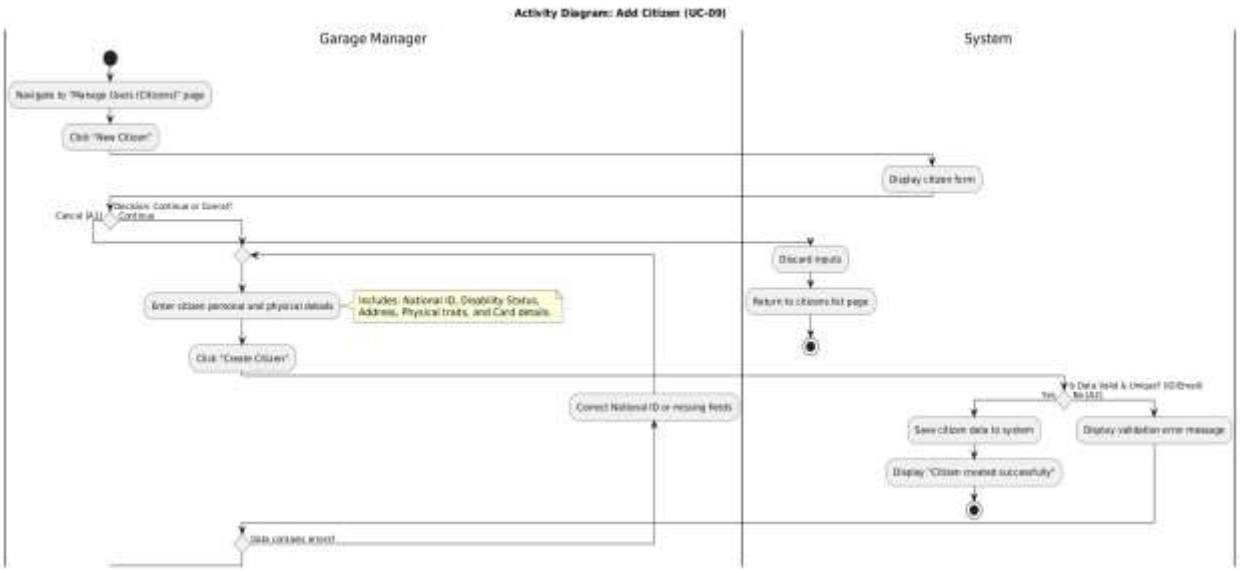


Figure 9 activity UC-08

Figure 10 activity UC-09



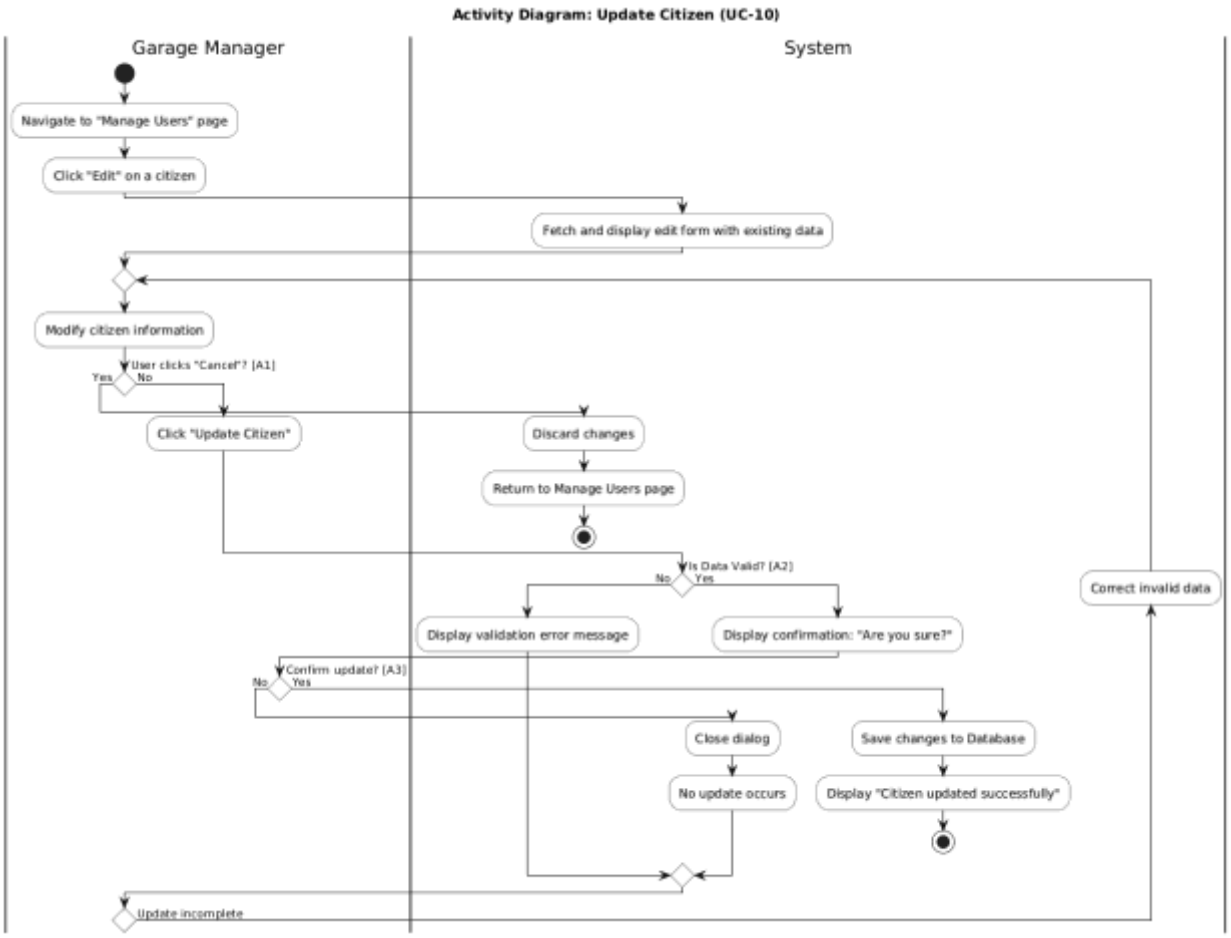


Figure 11 activity UC-10

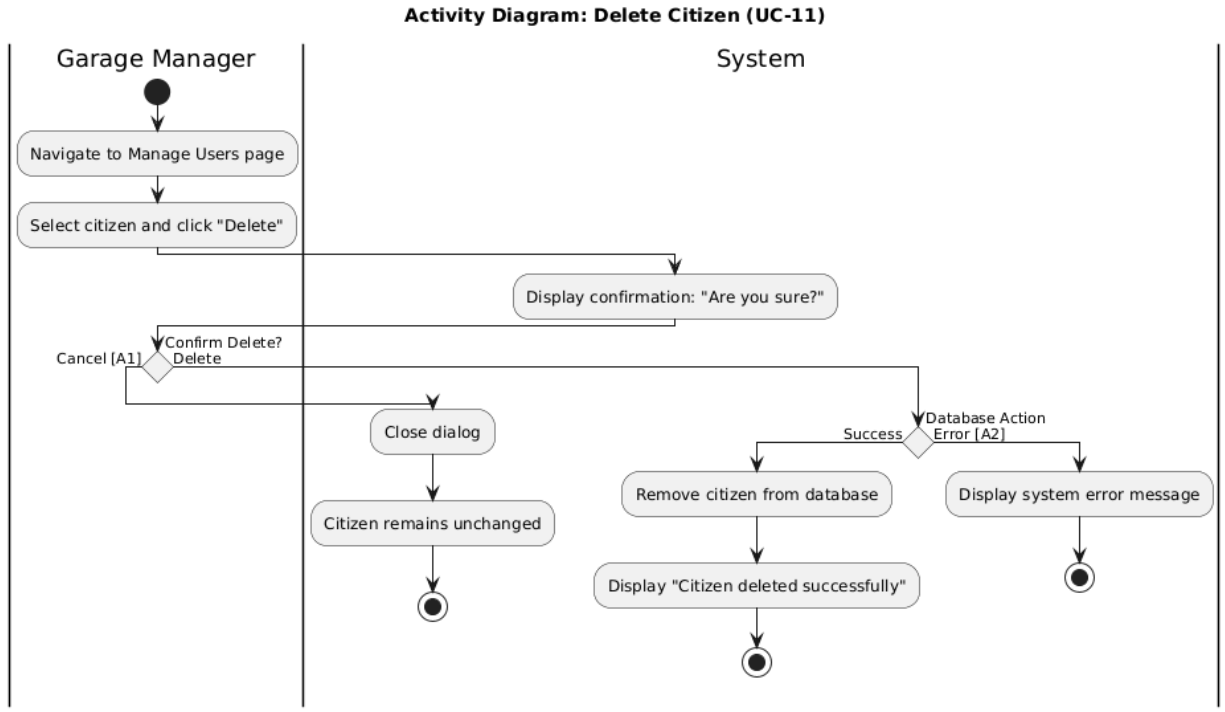


Figure 12 activity UC-11

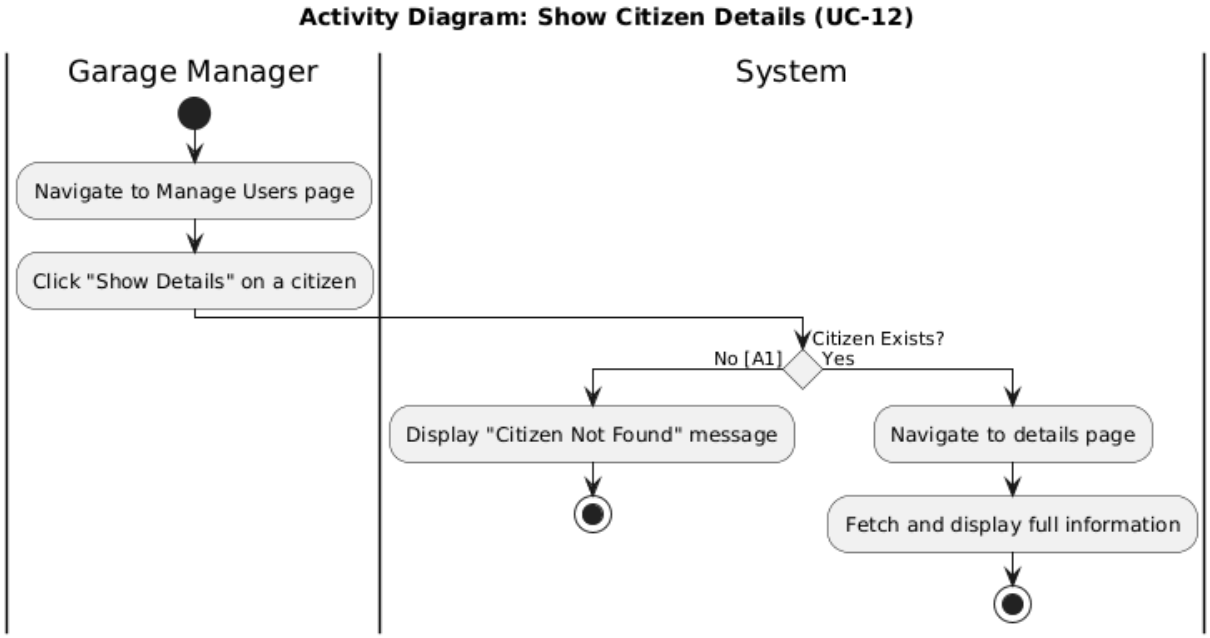


Figure 13 activity UC-12

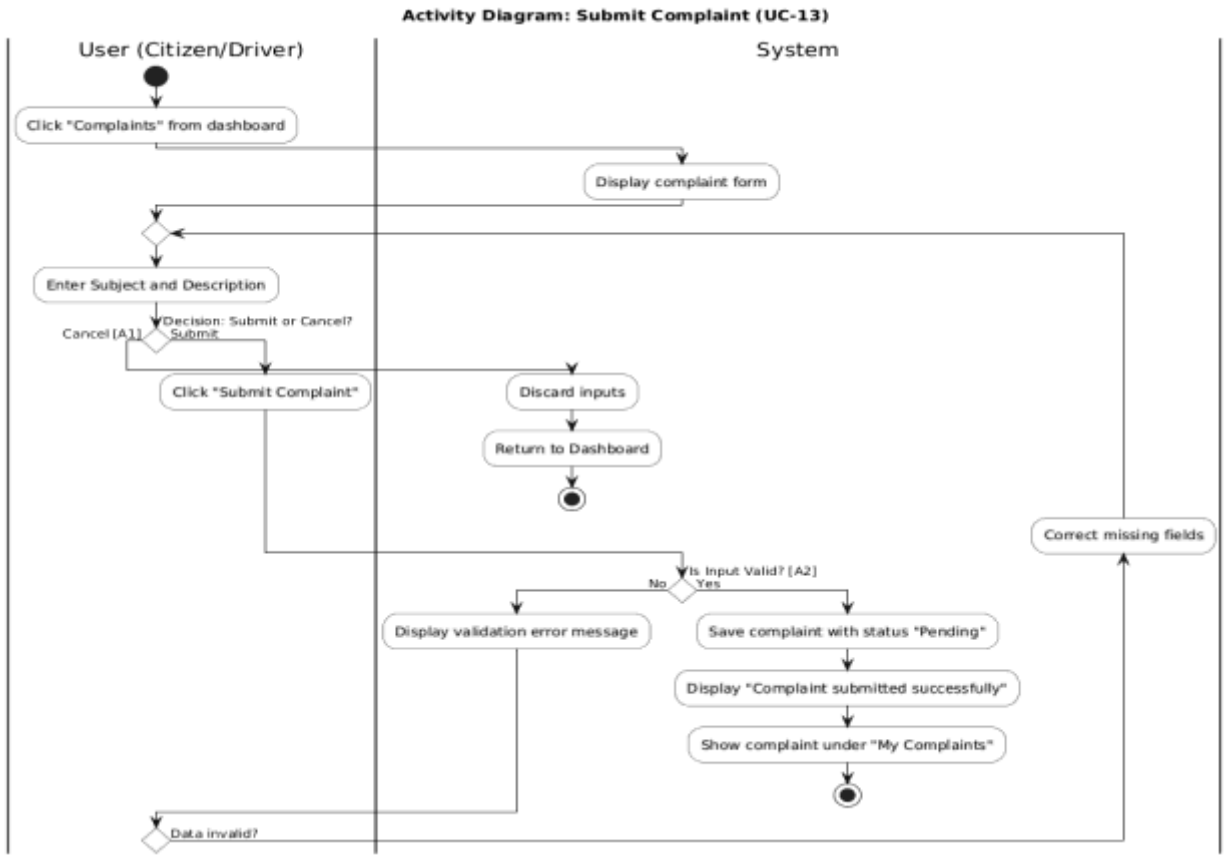


Figure 14 activity UC-13

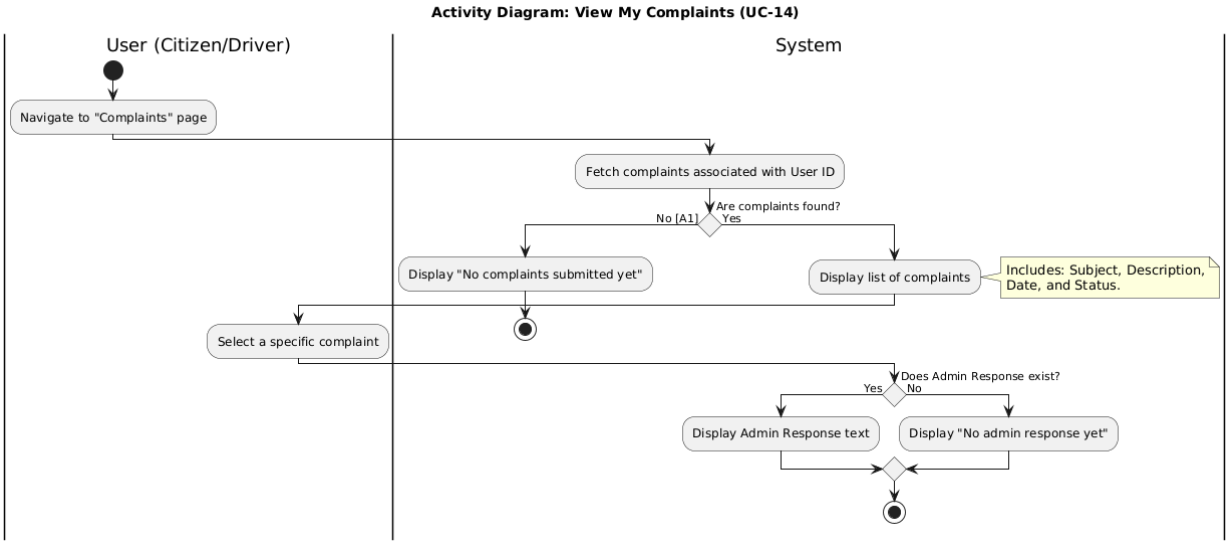


Figure 15 activity UC-14

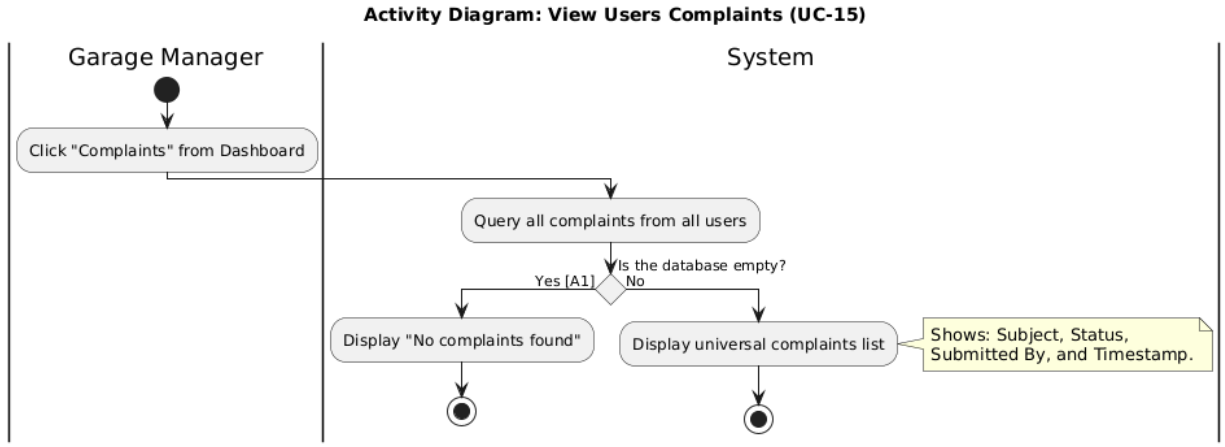


Figure 16 activity UC-15

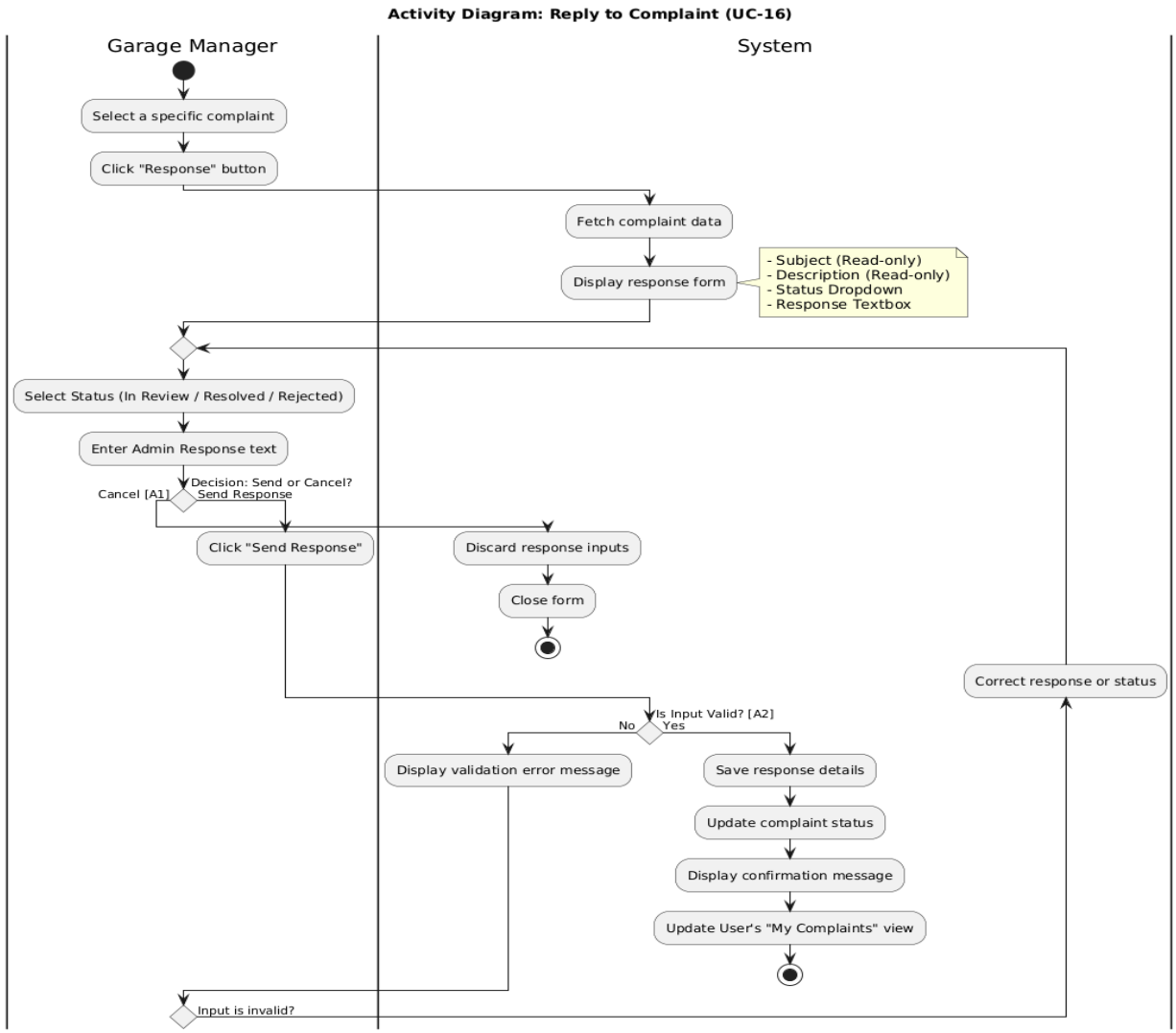


Figure 17 activity UC-16

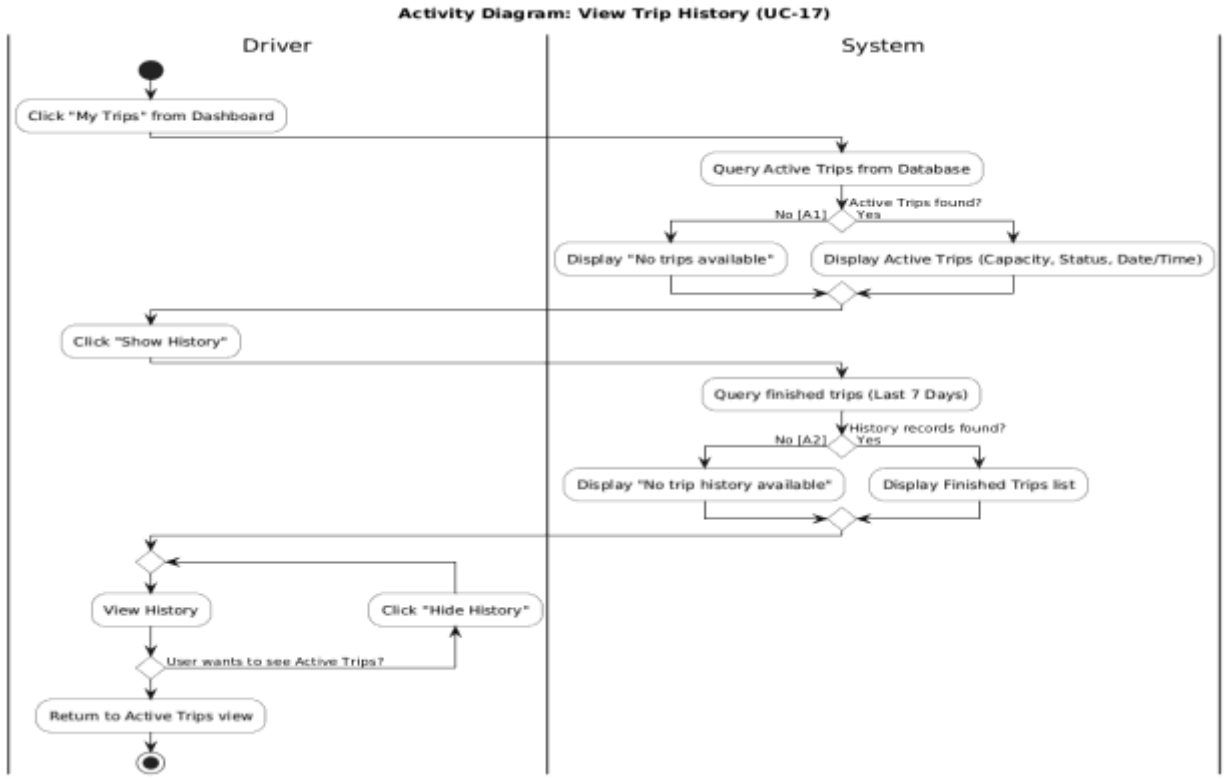


Figure 18 activity UC-17

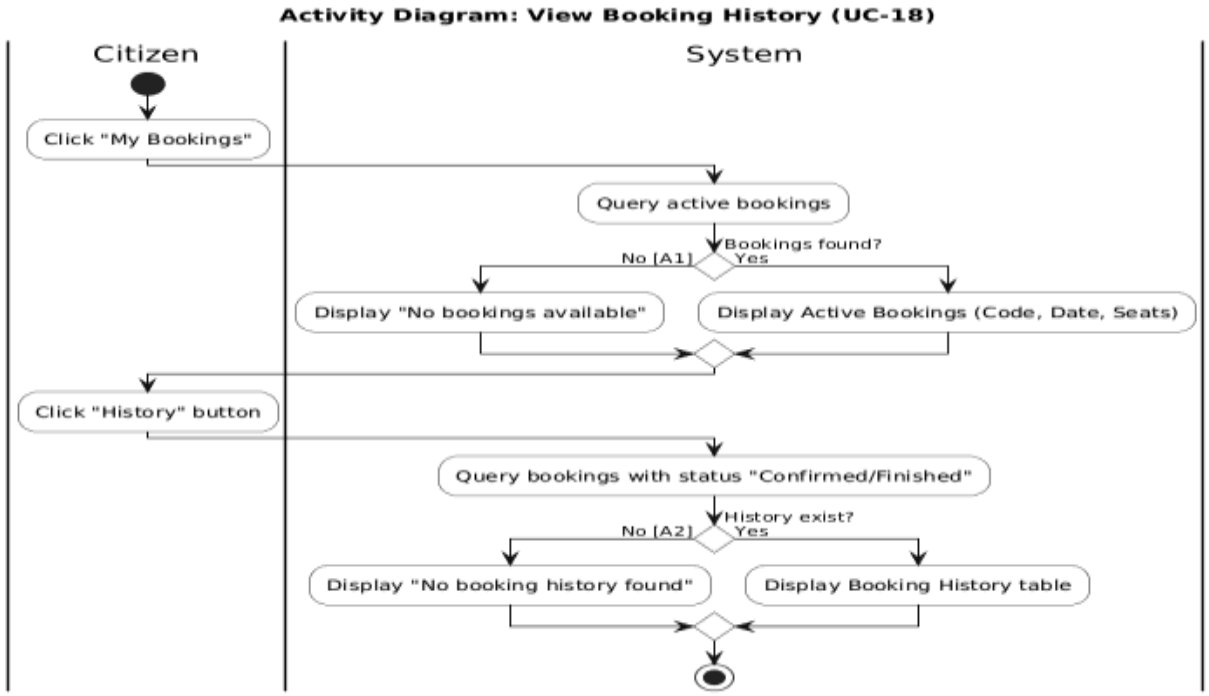


Figure 19 activity UC-18

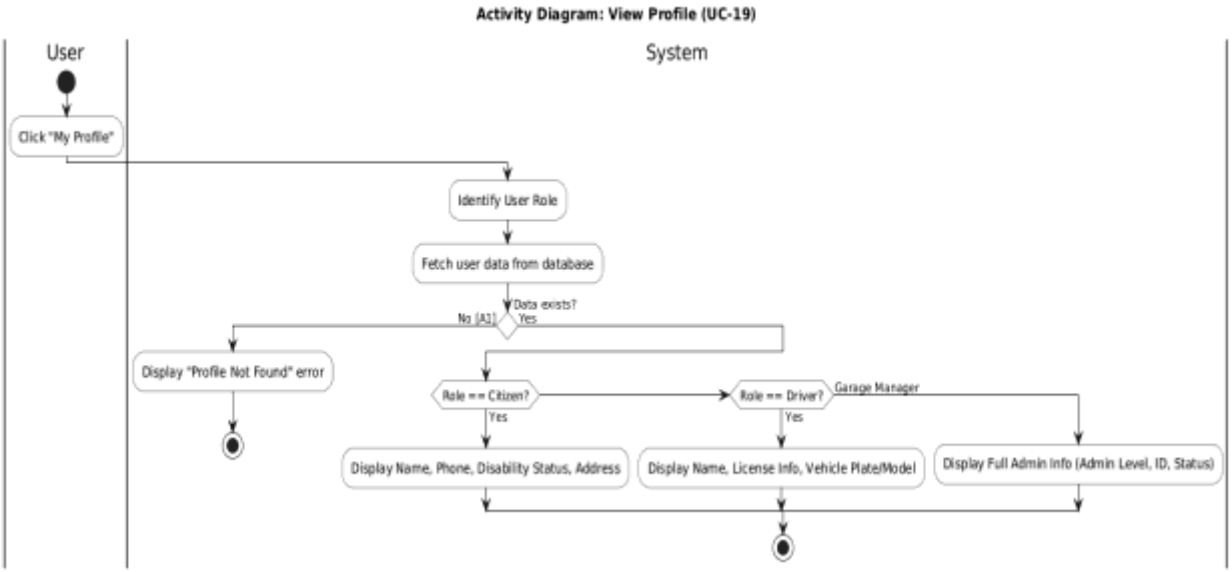


Figure 20 activity UC -19

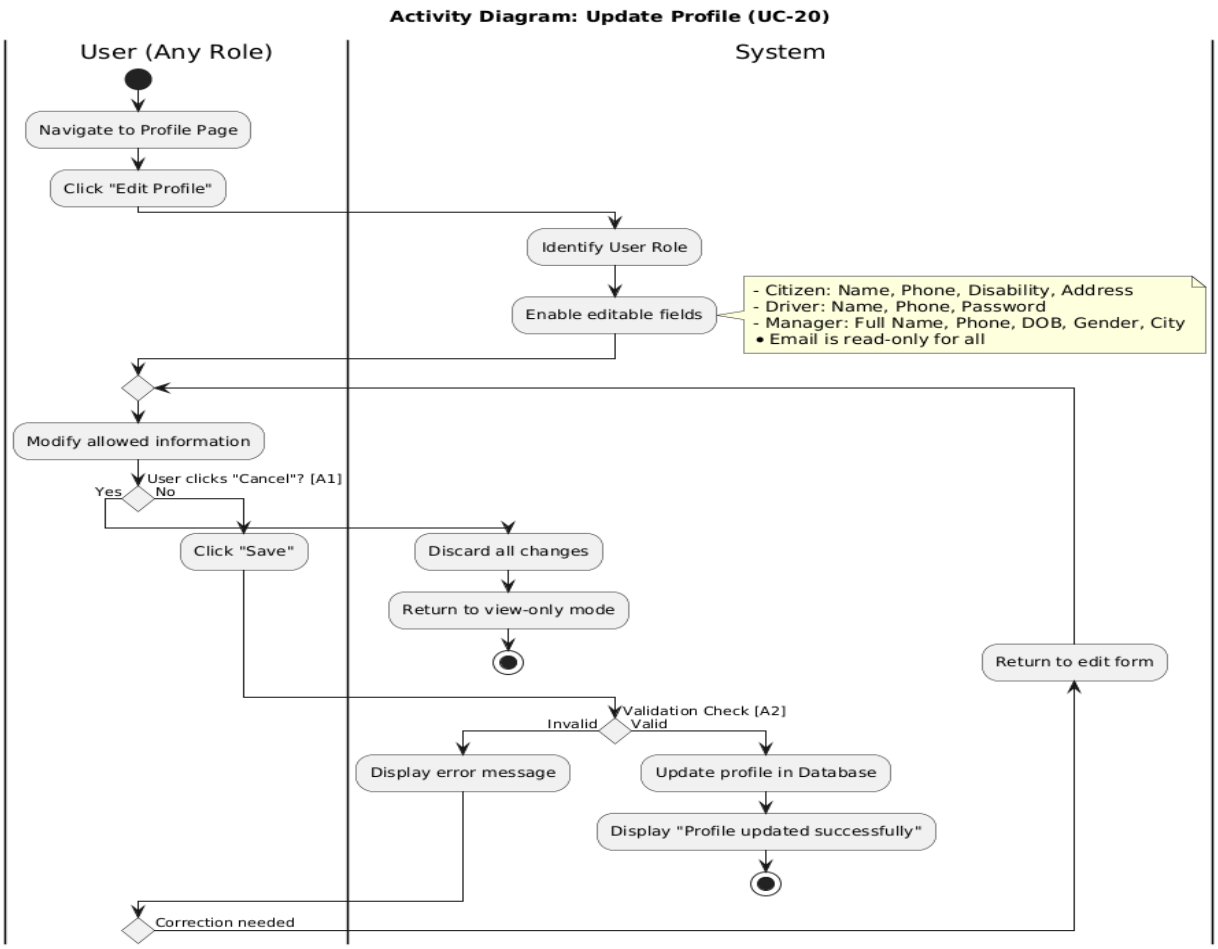


Figure 21 activity UC-20

Activity Diagram: Search with Filters (UC-21)

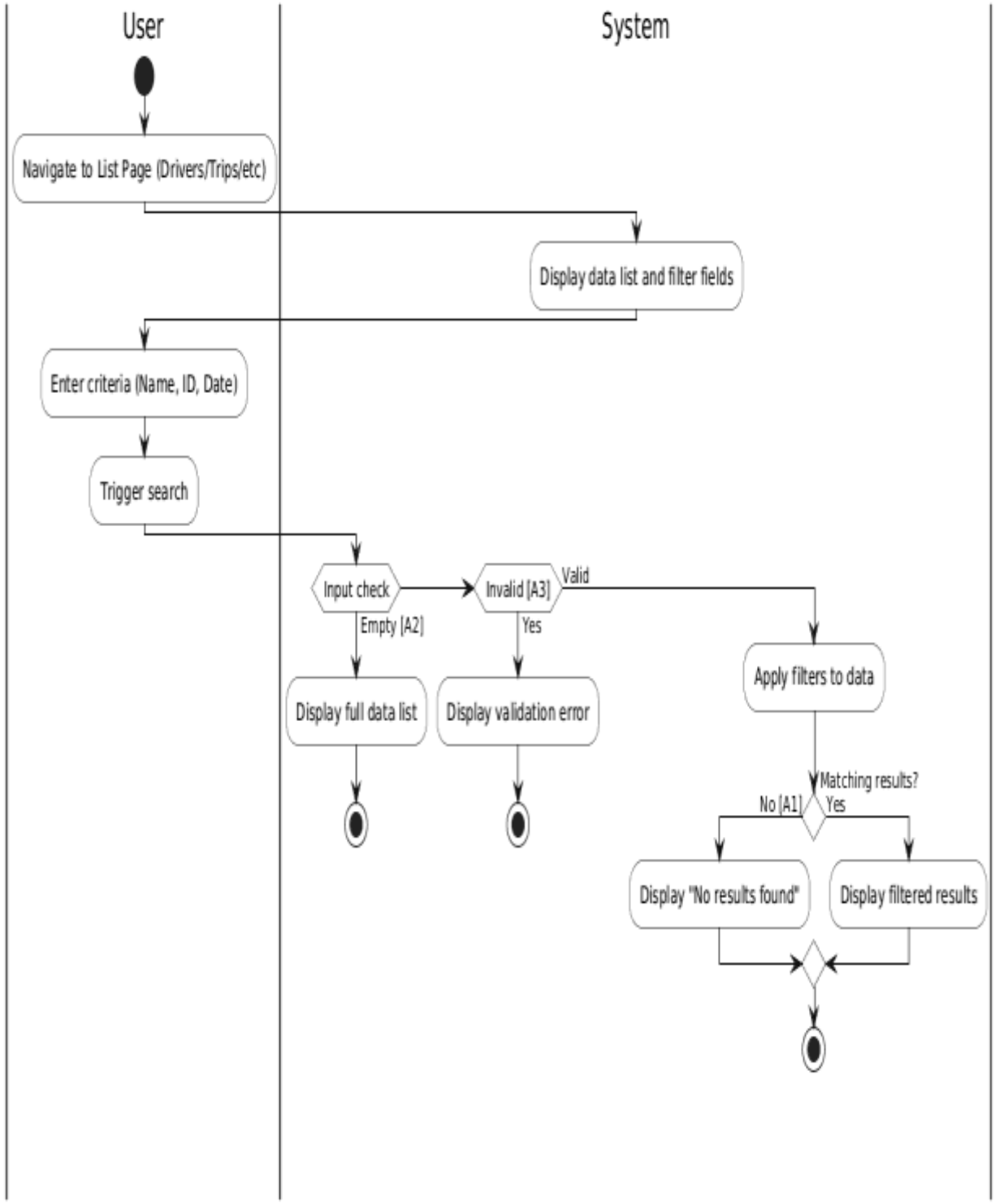


Figure 22 activity UC-21

7-4-Sequence Diagrams

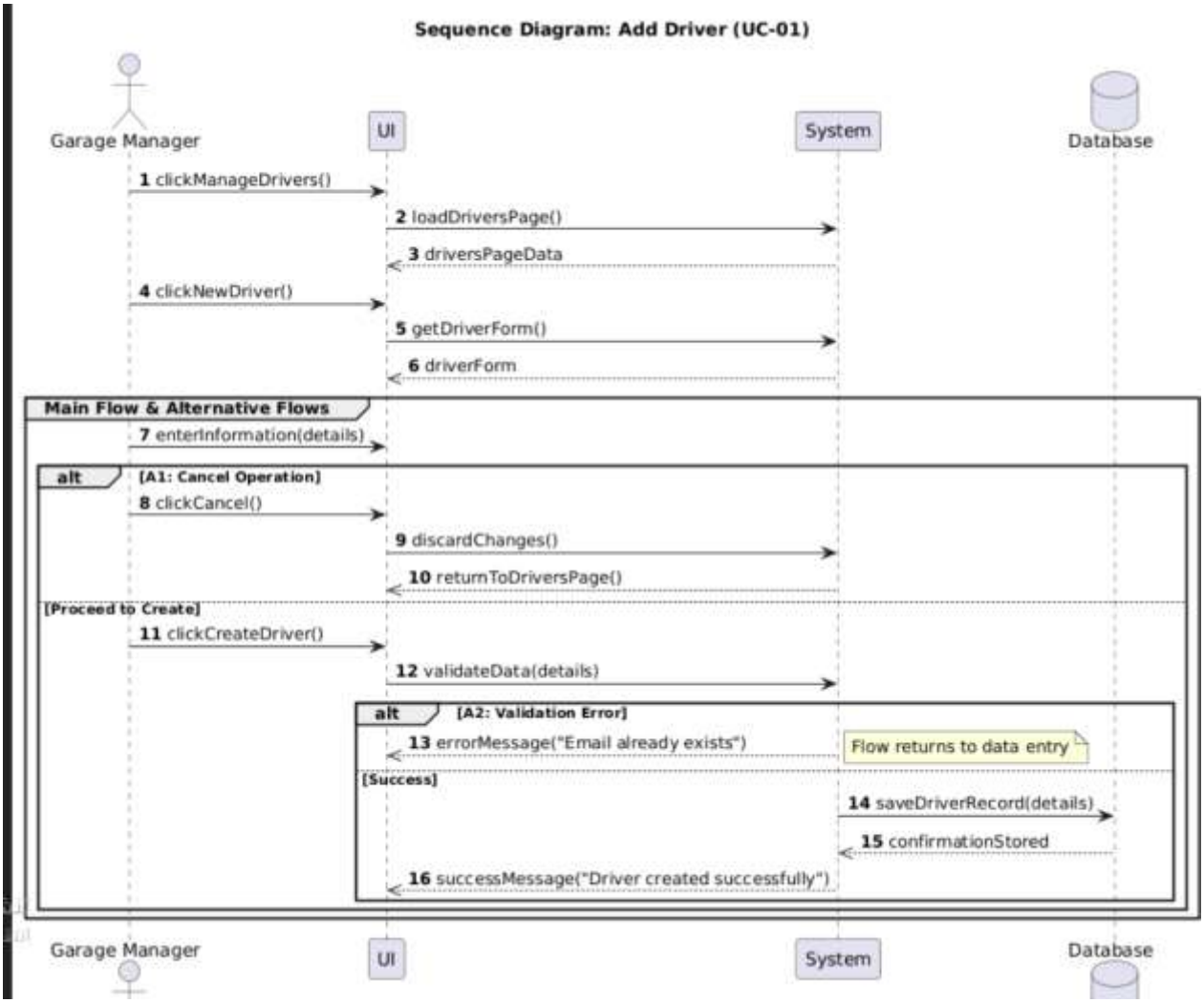


Figure 23 sequence UC-1

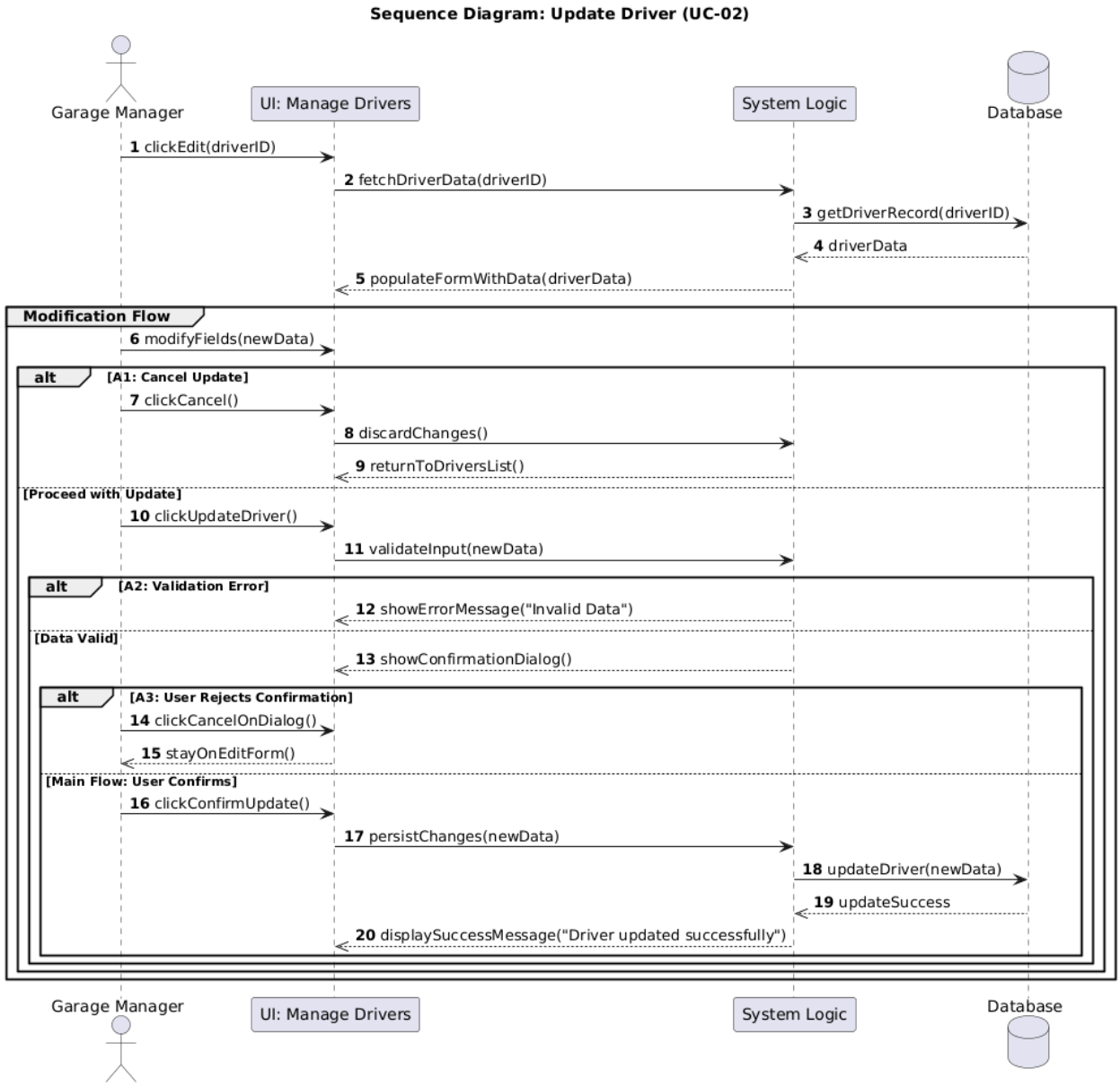


Figure 24 sequence UC-02

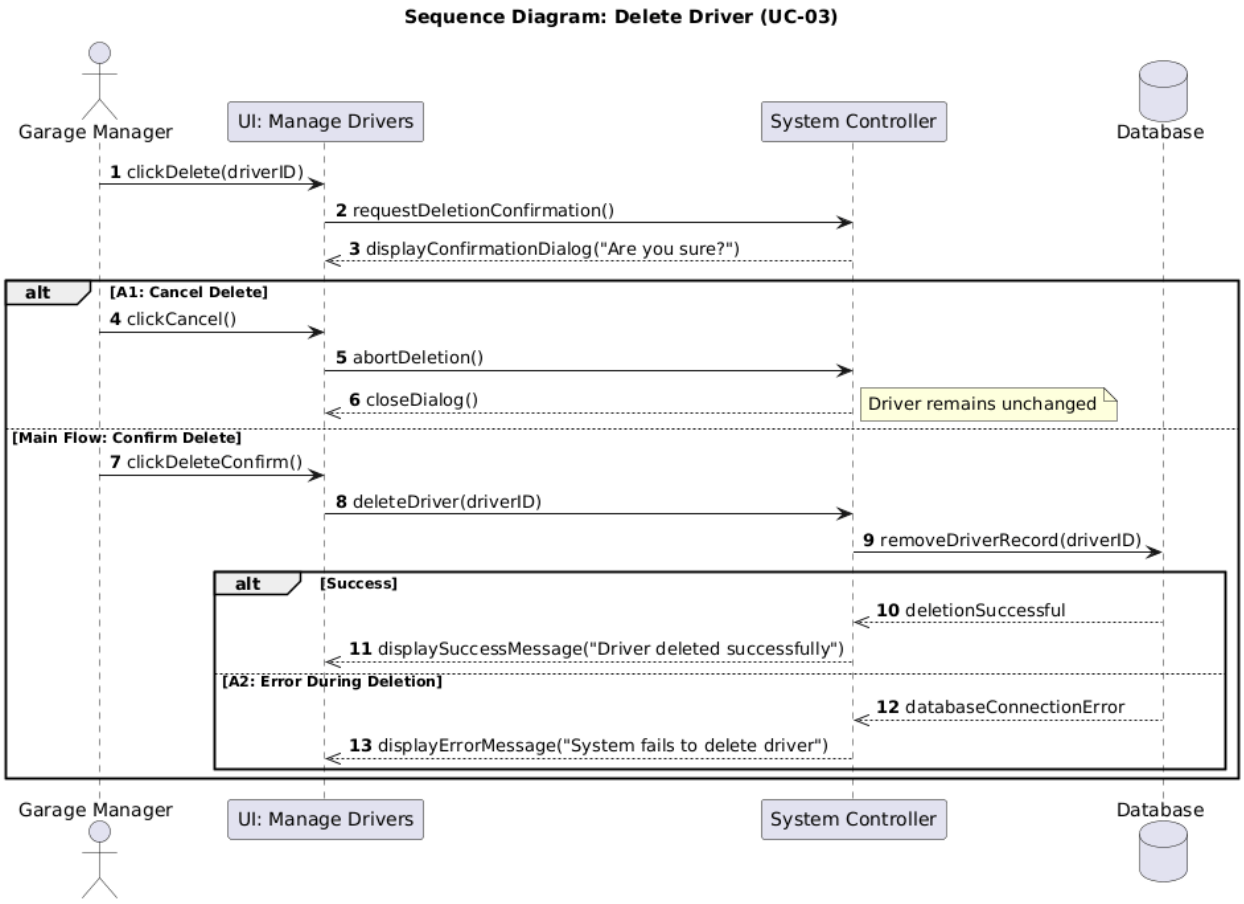


Figure 25 sequence UC-3

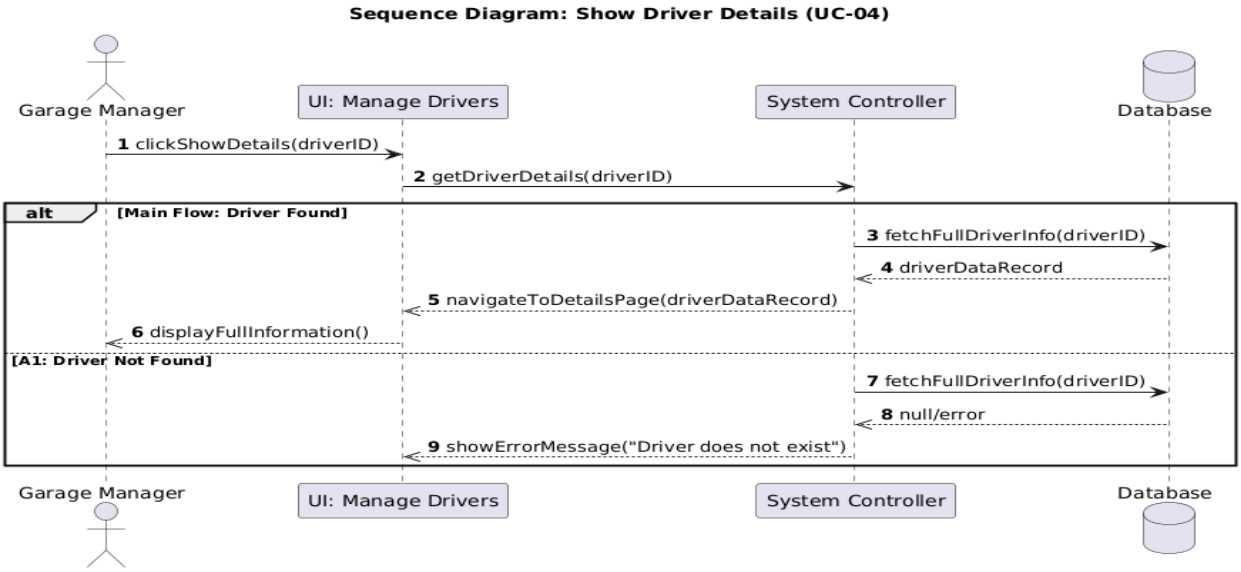


Figure 26 sequnce UC-04

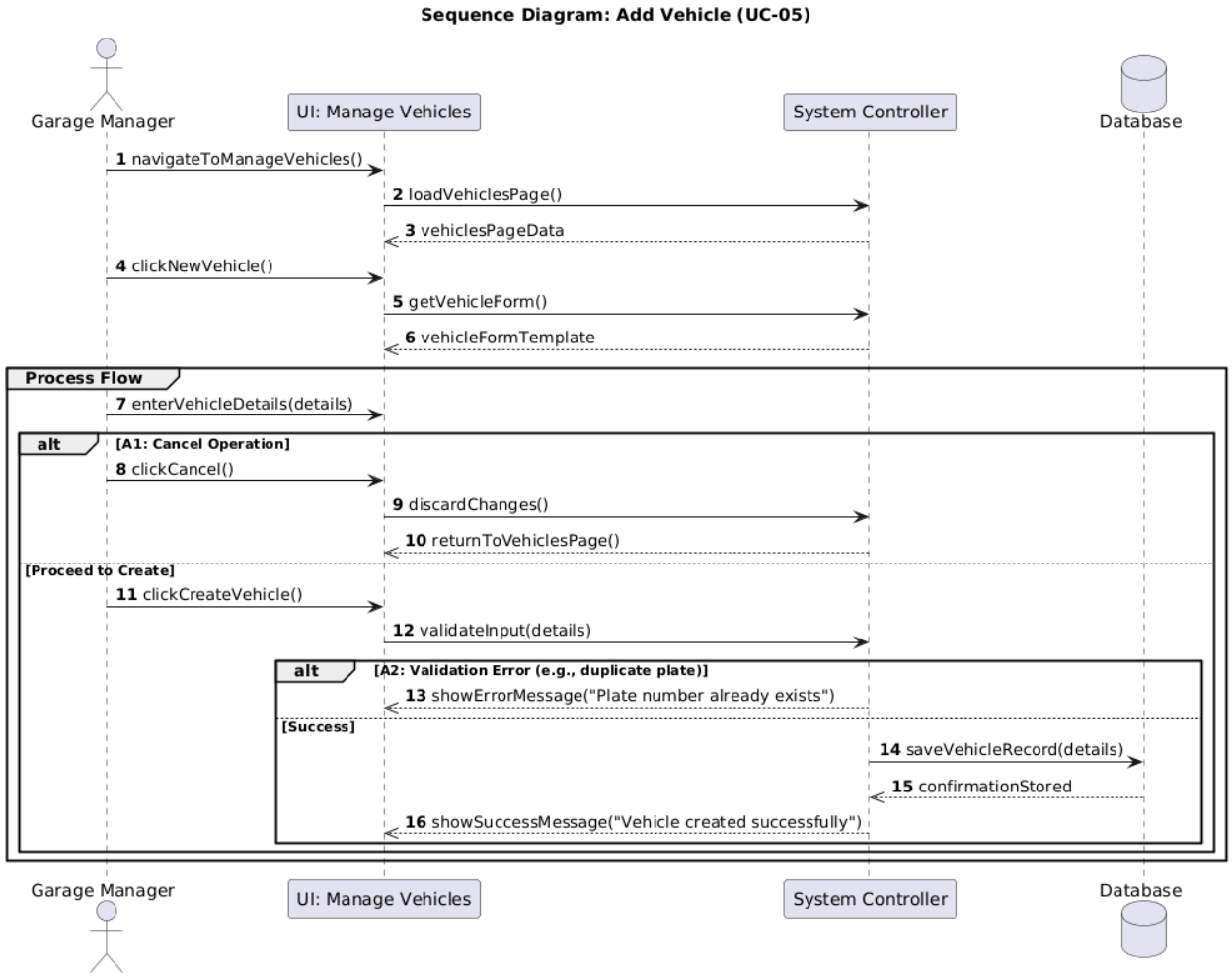


Figure 27 sequence UC-05

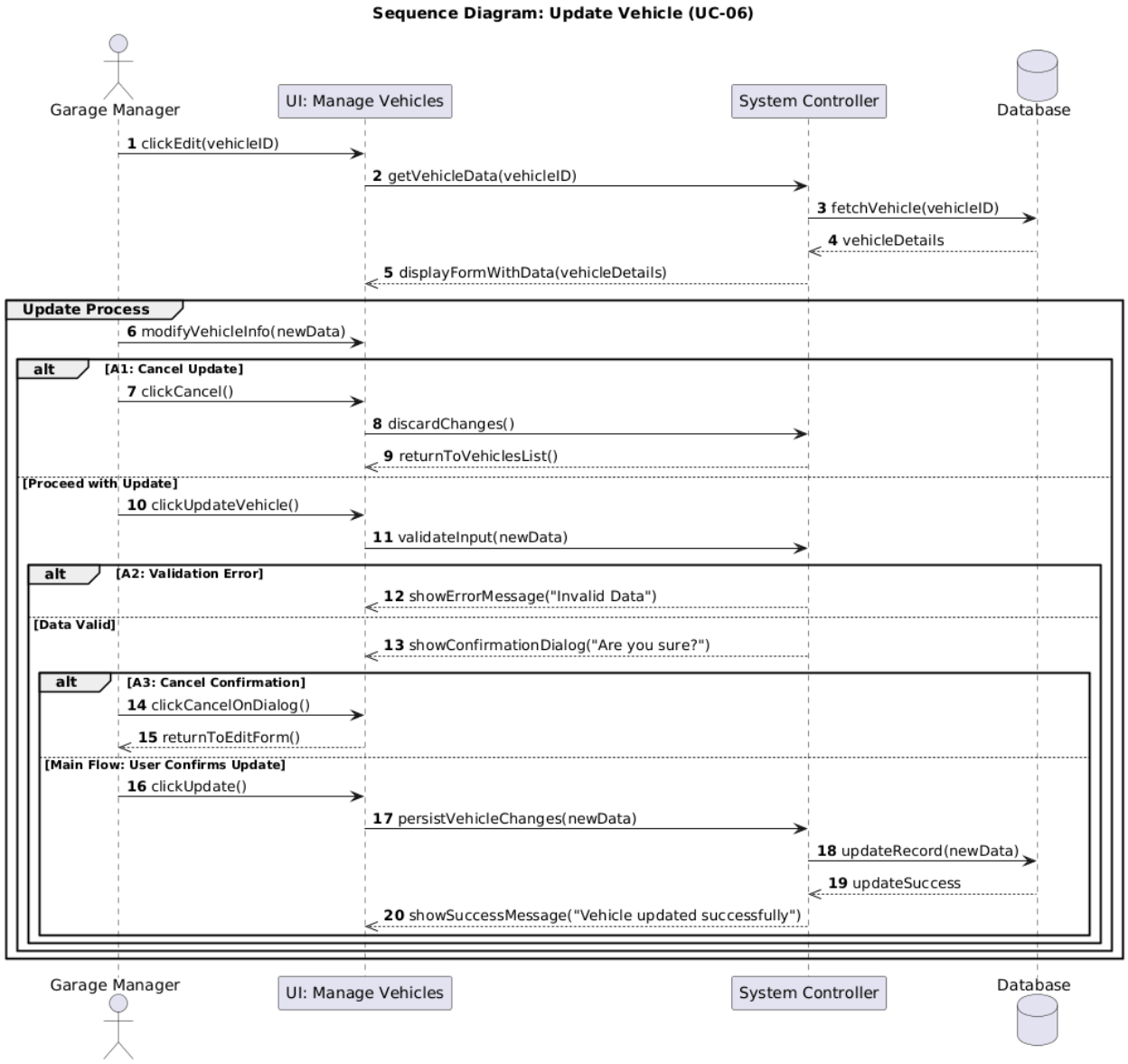


Figure 28 sequence UC-6

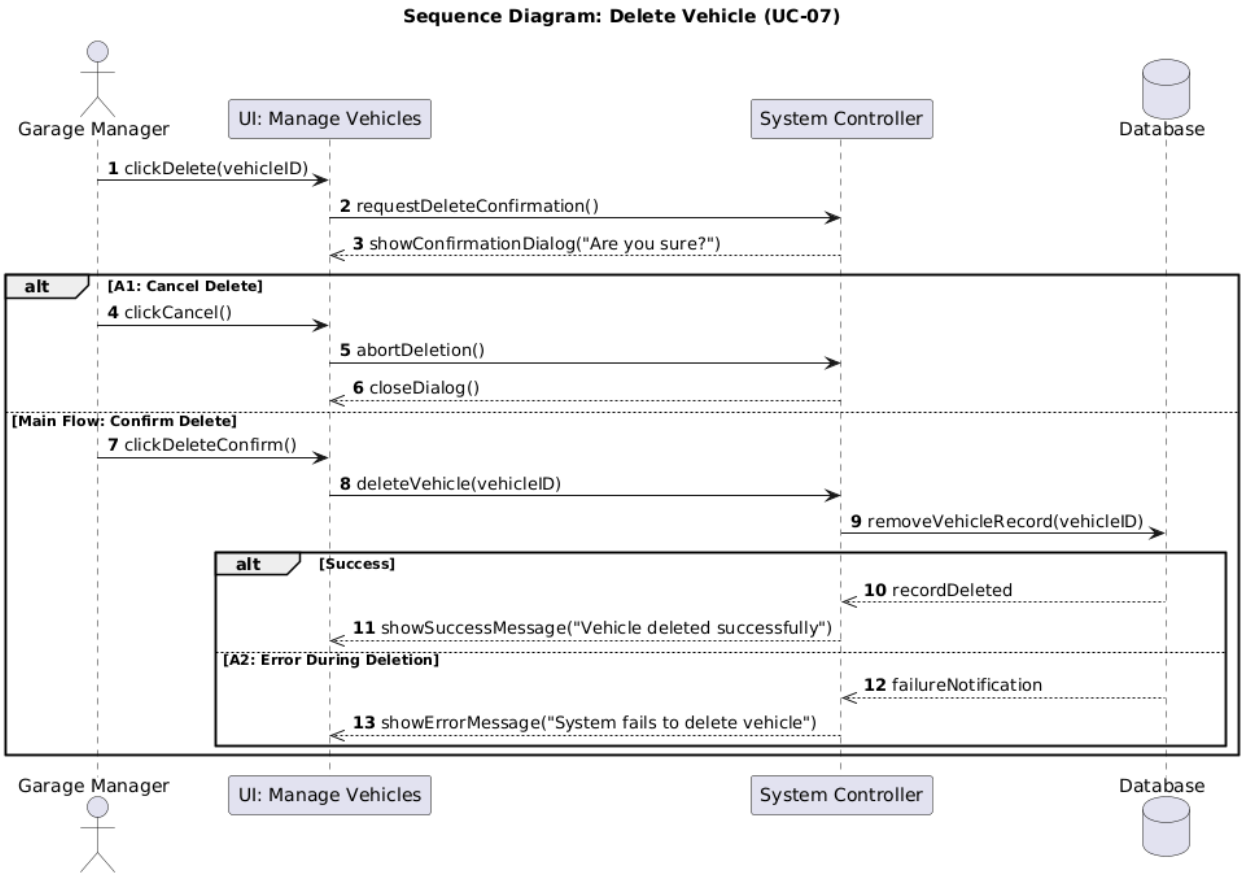


Figure 29 sequence UC-07

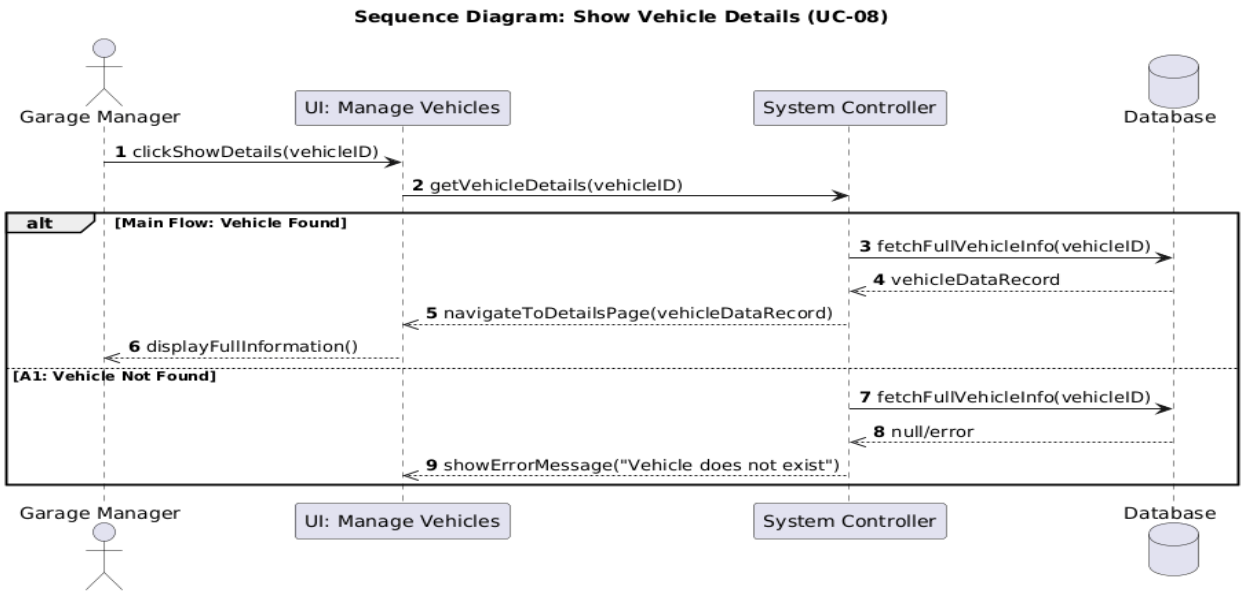


Figure 30 sequence UC-08

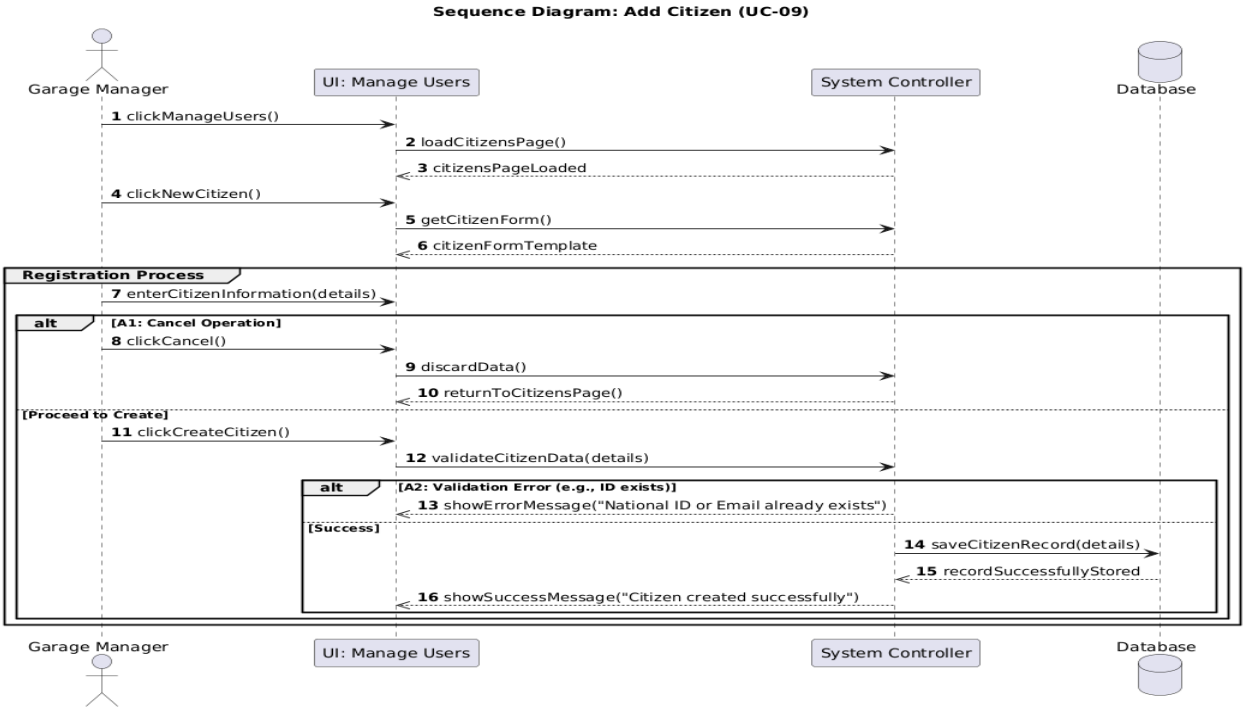


Figure 31 sequence UC-9

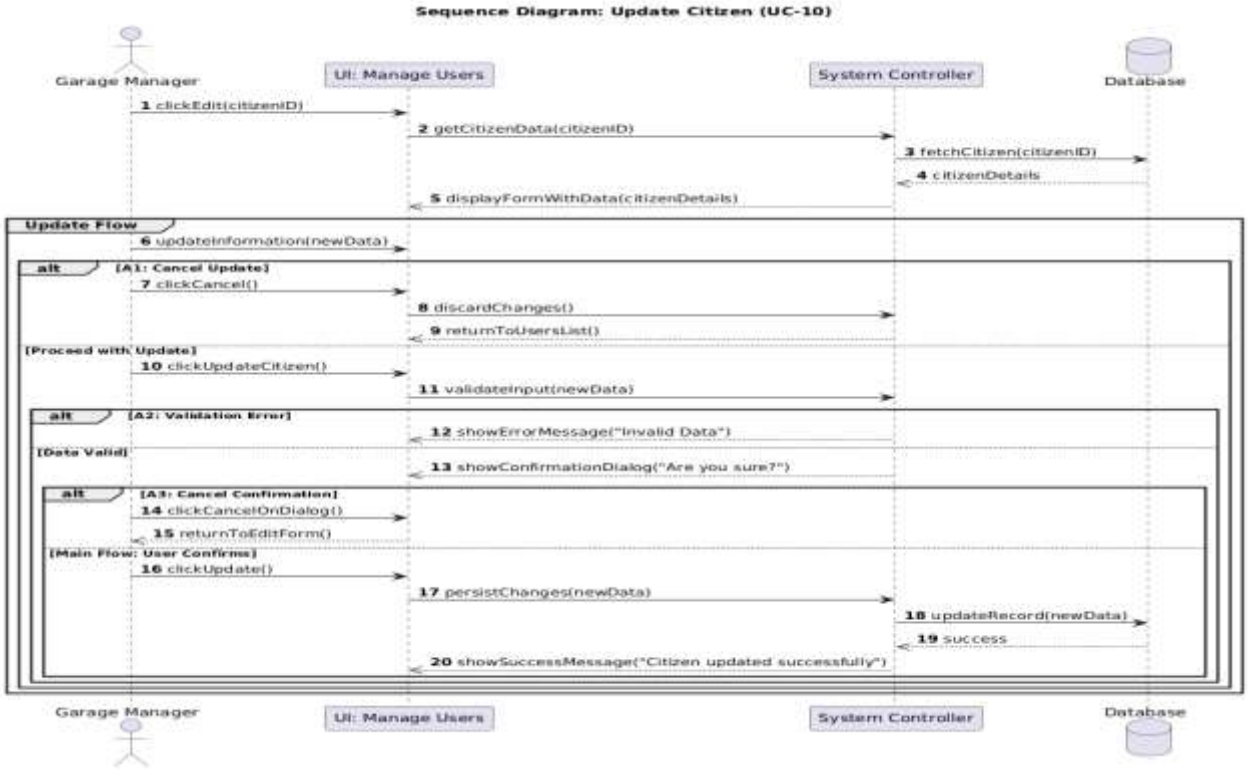


Figure 32 sequence UC-10

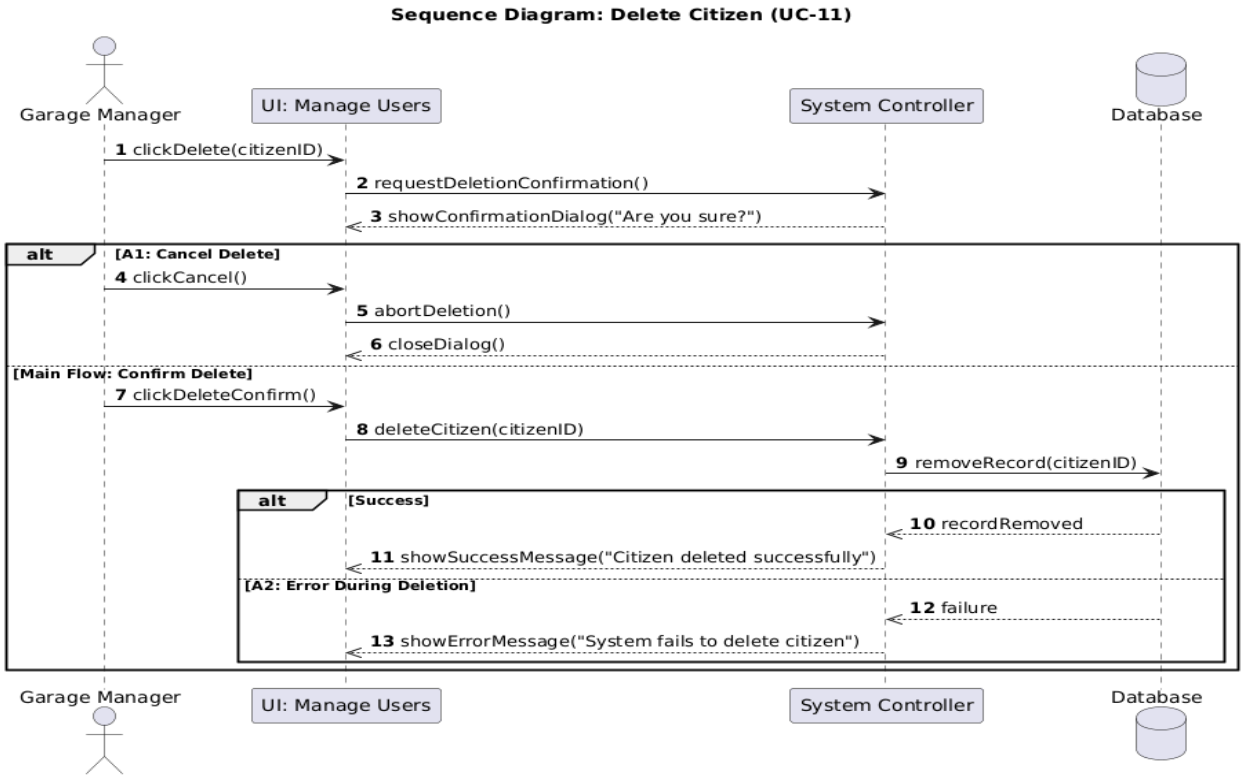


Figure 33 sequence UC-11

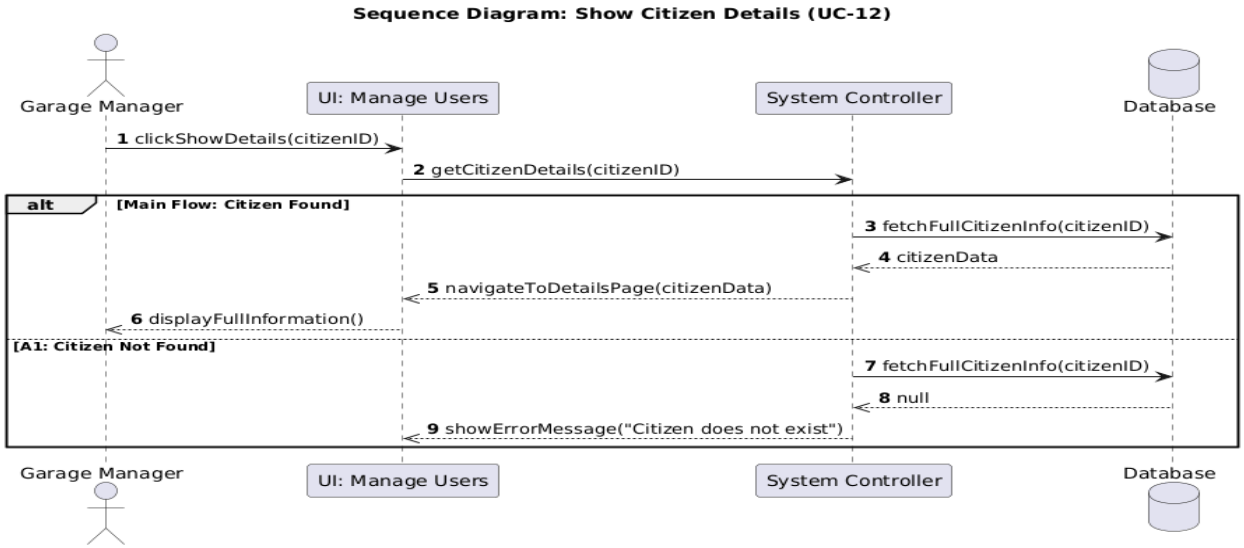


Figure 34 sequence UC-12

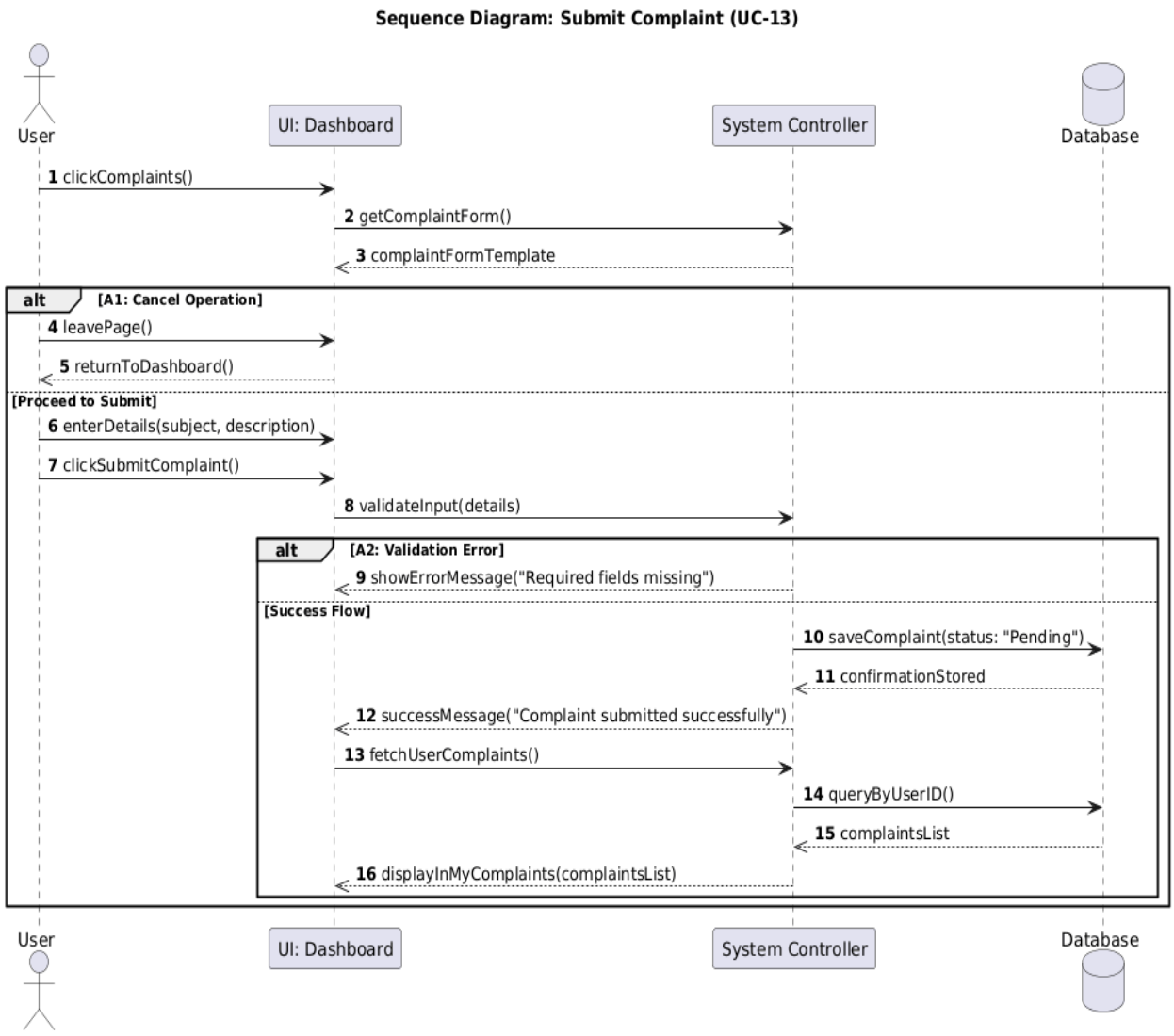


Figure 35 sequence UC-13

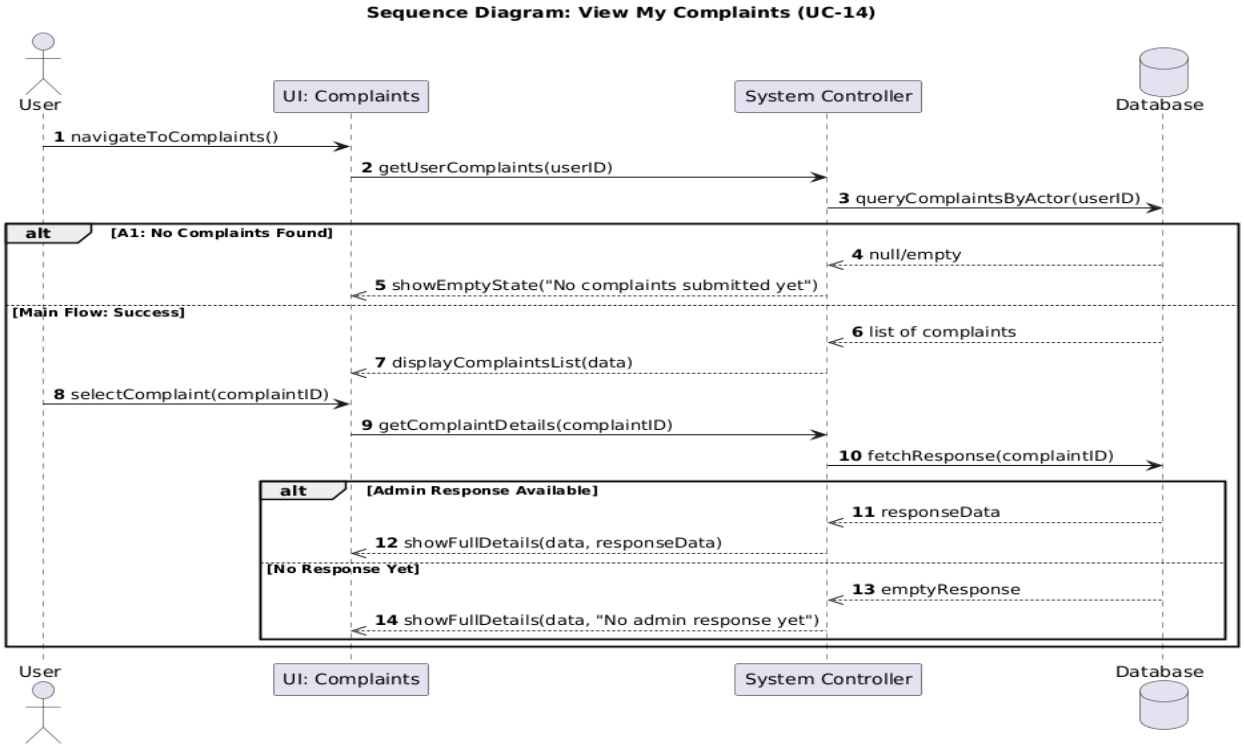


Figure 36 seguce UC-14

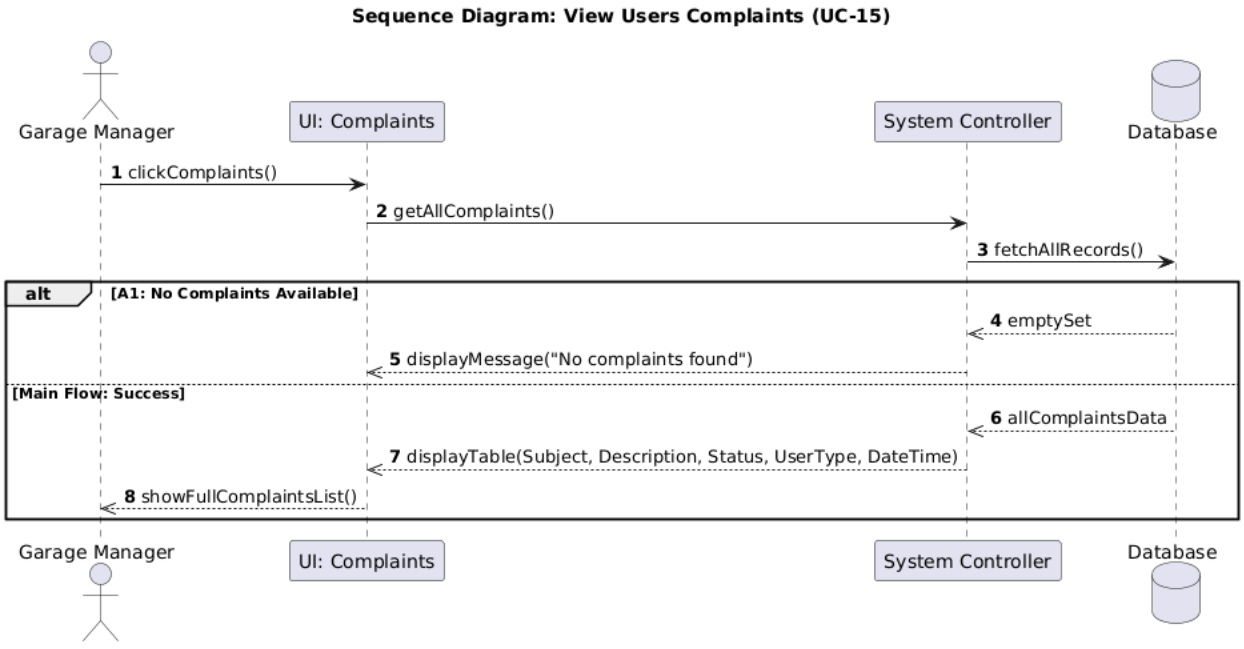


Figure 37 sequence UC-15

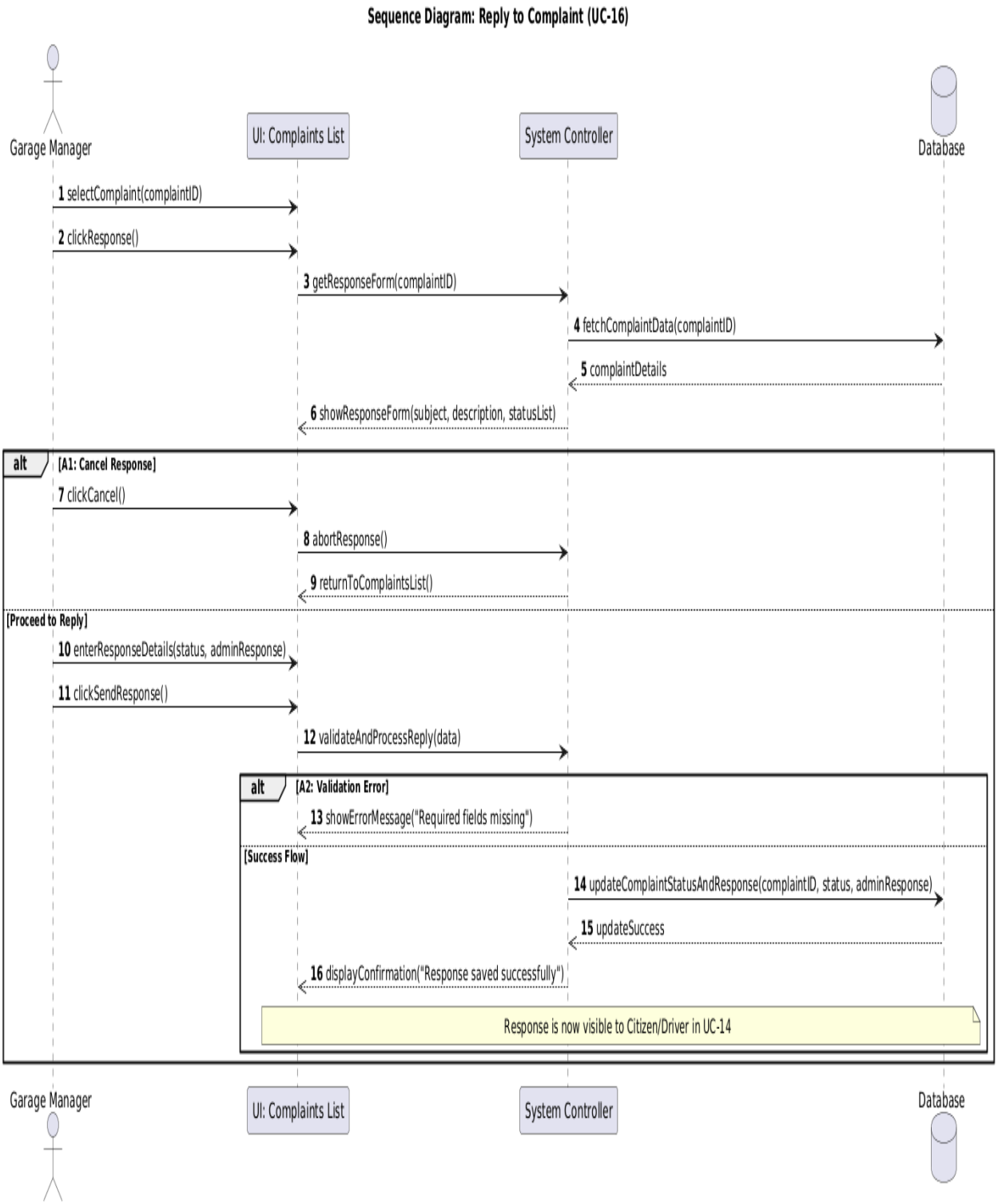


Figure 38 sequence UC-16

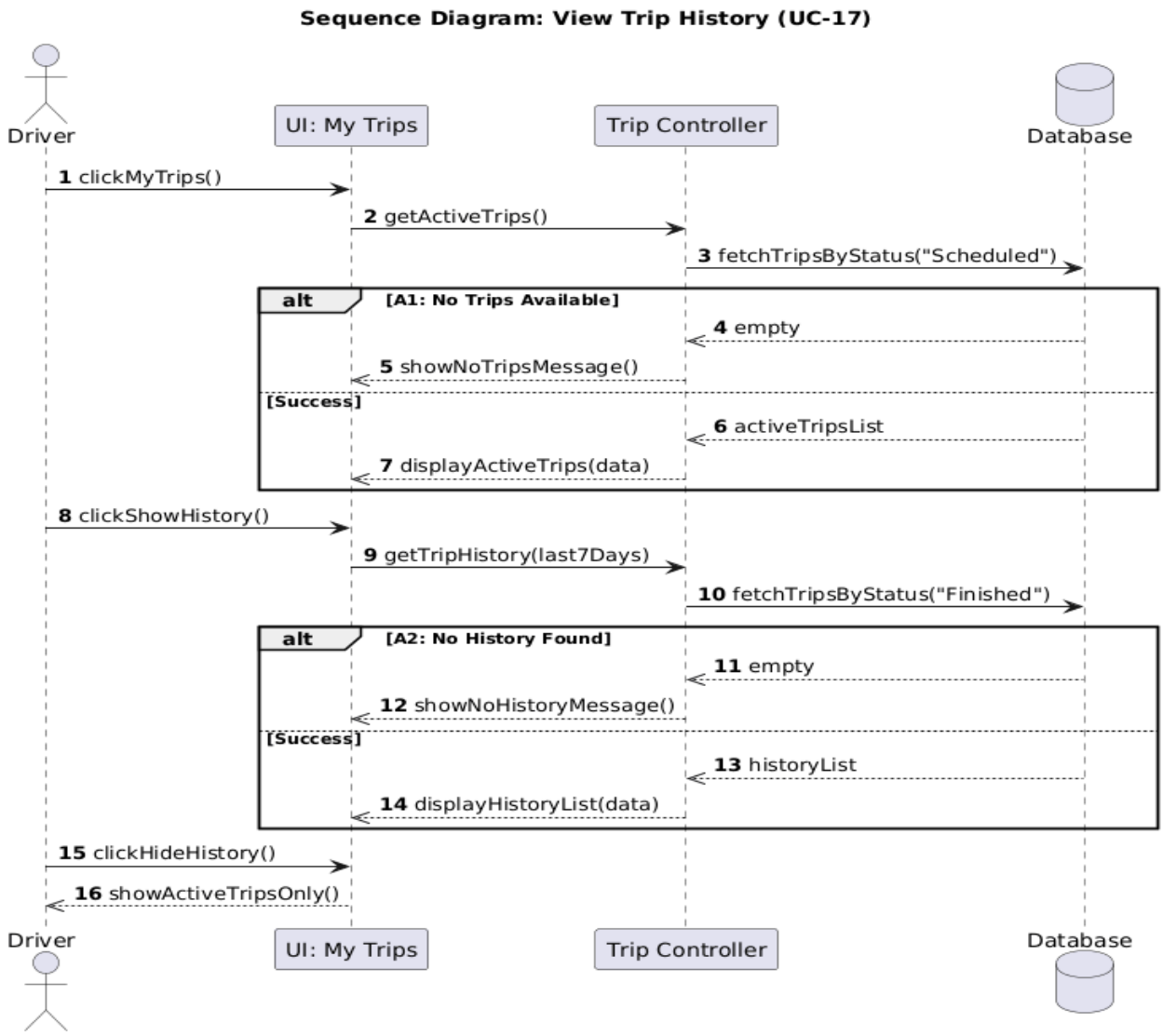


Figure 39 sequence UC-17[

Sequence Diagram: View Booking History (UC-18)

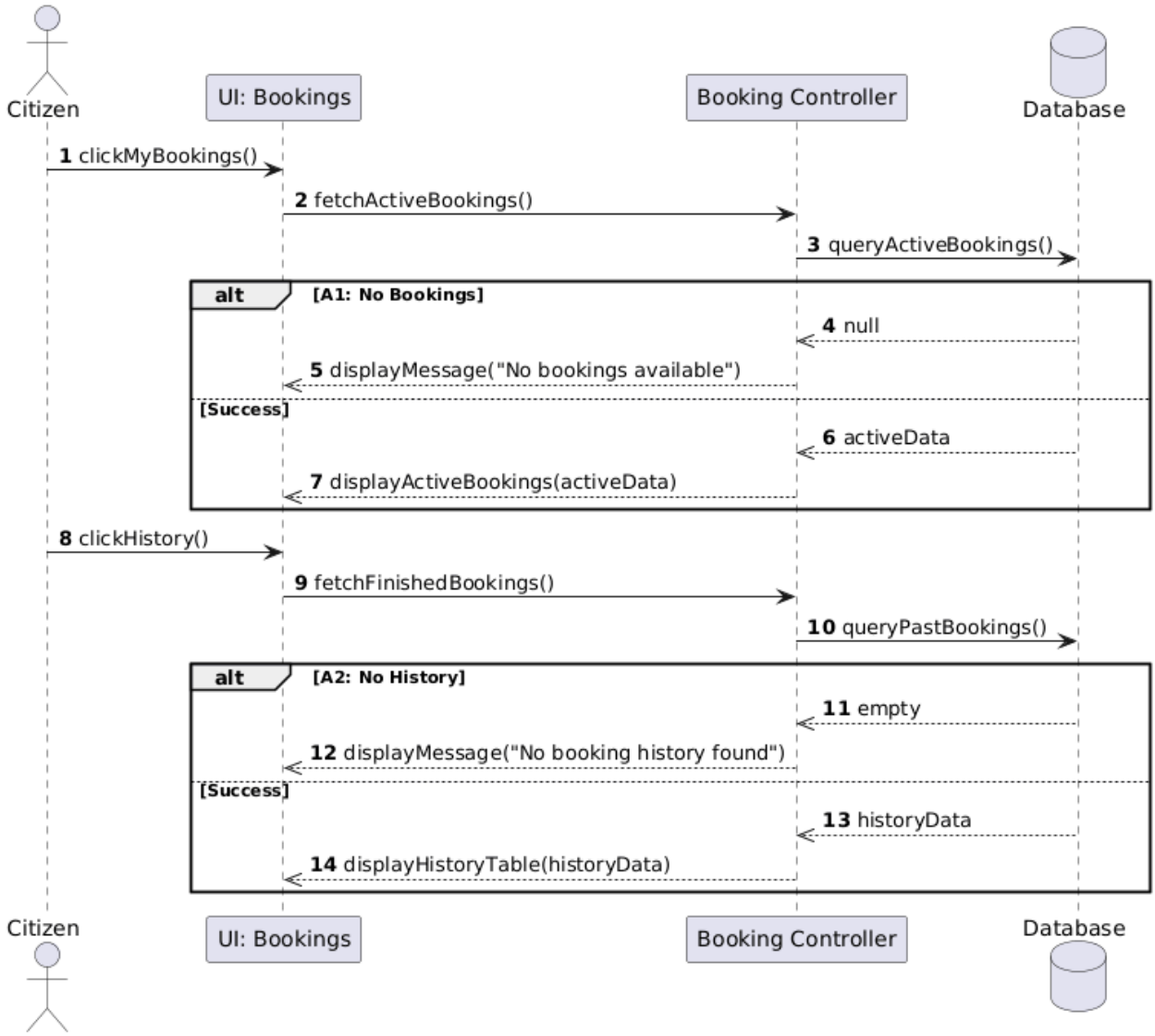


Figure 40 sequence UC-18

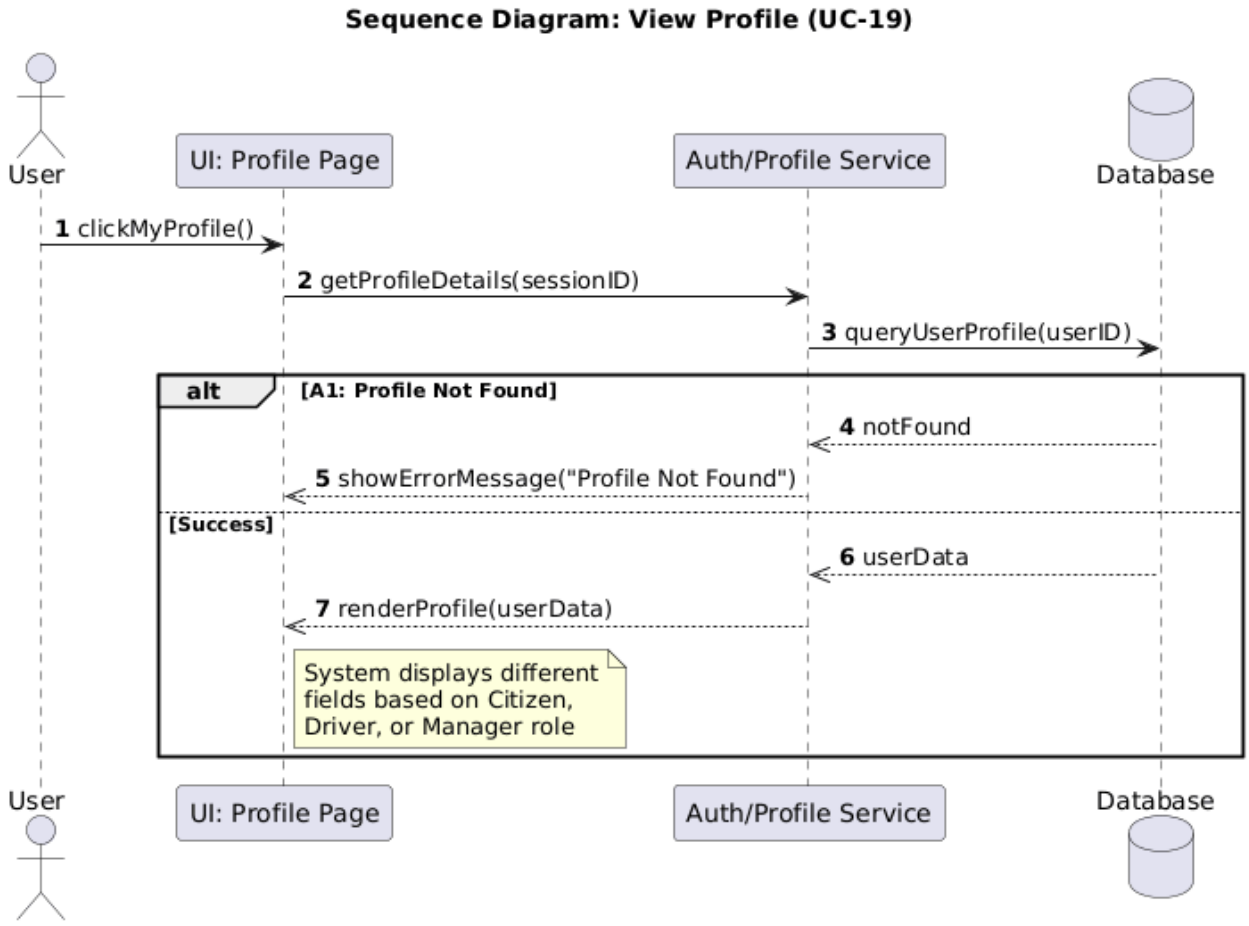


Figure 41 sequence UC-19

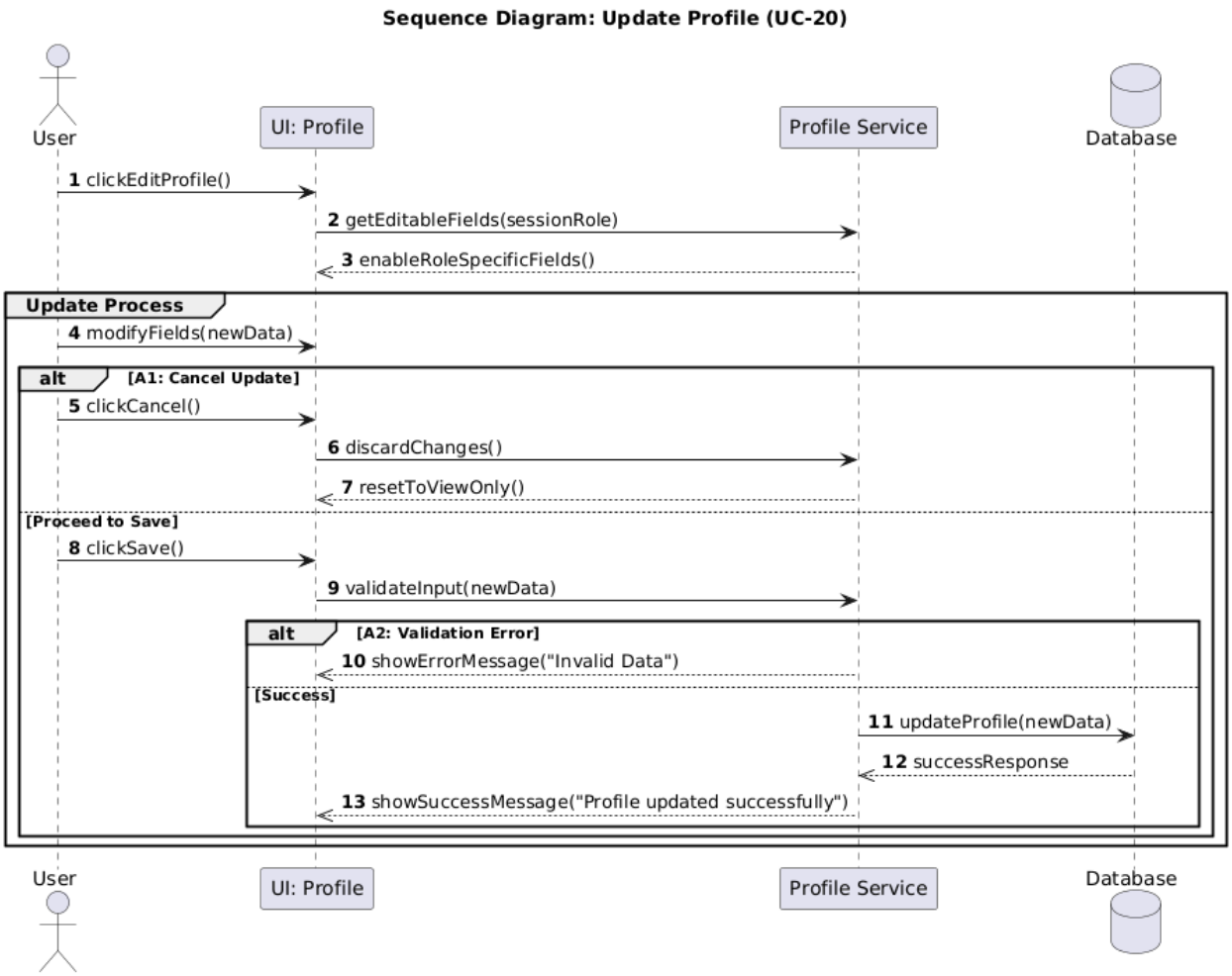


Figure 42 sequence UC-20

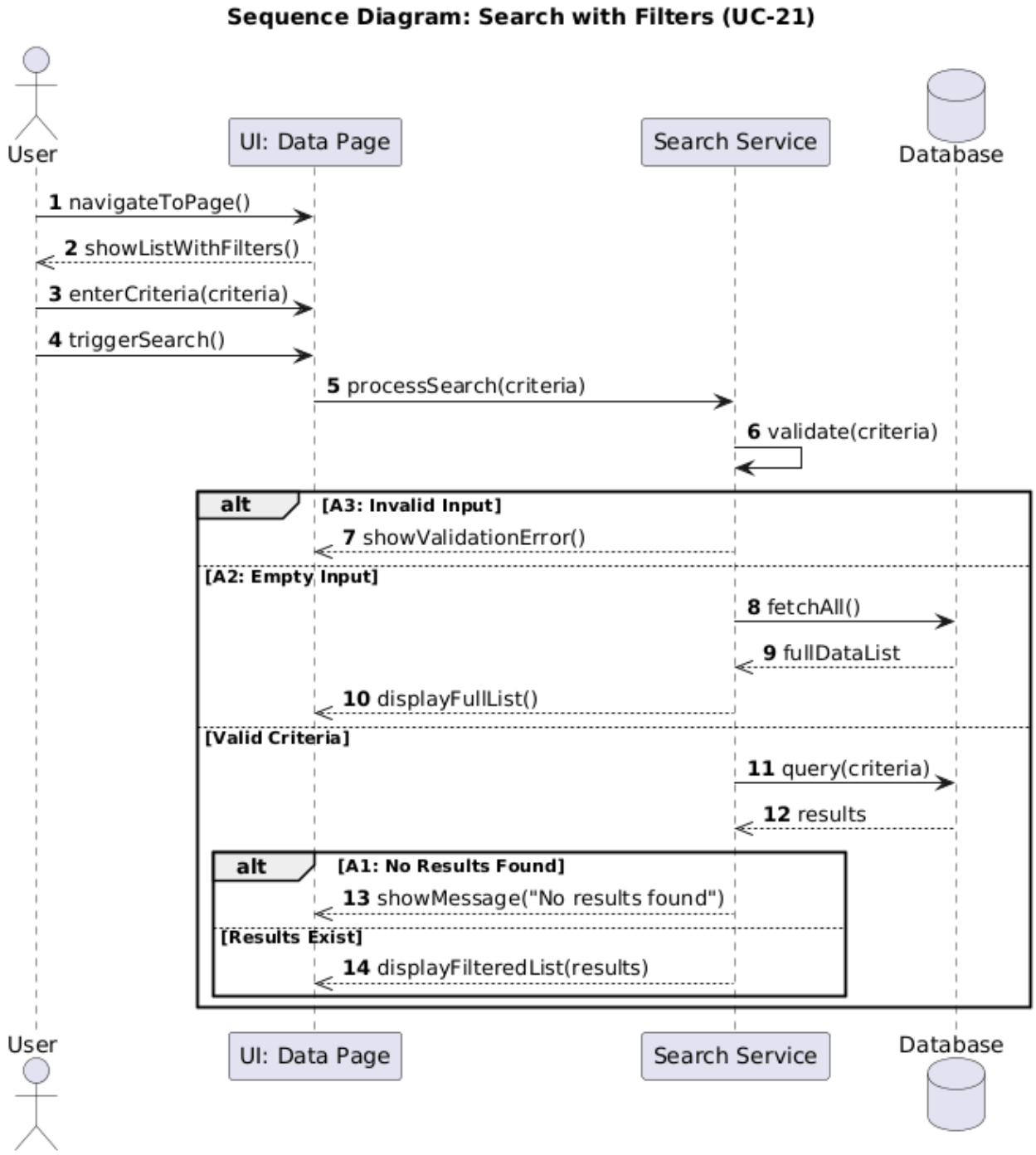


Figure 43 sequence UC-21

7-5-Requirements Traceability Matrix

ID	Title	Associated Dependency	SRS Section	Design Section	Coding	integration Testntgretion	Acceptance test
FR 1	Garage manager can be add new driver	None	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1.1 Add Driver Test Case Table	يجب تعبئة جميع الحقول الالزامية مثل (المعلومات الشخصية، معلومات الرخصة).
FR 2	Garage manager can be update driver	FR1	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1.2 Update Driver Test Case Table	يجب ان تكون التعديلات متوافقة والا تخلق تعارضات في بيانات السائق.
FR 3	Garage manager can be delete driver	FR1	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1 Functional Testing	يجب ان لا يكون السائق مرتبط برحلة قائمة أو نشطة عند الحذف.
FR 4	Garage manager can be show driver	FR1	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1 Functional Testing	يجب ان يبحث عن سائق بالاسم او بالرقم الوظيفي لعرض بياناته.

FR 5	Garage manager can be add new vehicle	None	Analytical study	Architectural Designn	Vehicle Service	8.2.1.3 Add Vehicle Test Case Table	يجب ان تكون الحافلة المراد اضافتها غير موجودة مسبقاً بالنظام.
FR 6	Garage manager can be update information vehicle	FR5	Analytical study	Architectural Design	Vehicle Service	8.2.1 Functional Testing	يجب ان تكون التعديلات متوافقة والا تخلق تعارضات مع أسطول الحافلات.
FR 7	Garage manager can be delete vehicle	FR5	Analytical study	Architectural Design	Vehicle Service	8.2.1.4 Delete Vehcile Test Case Table	يجب ان يكون الحافلة موجودة وان لا تكون مرتبطة بخط مرتبطة برحلات.
FR 8	Garage manager can be show vehicle	FR5	Analytical study	Architectural Design	Vehicle Service	8.2.1 Functional Testing	يجب ان يكون الحافلة متاحة مسبقا ومسجلة في الكراج ليتم عرضها.
FR 9	Garage manager can be add new citizen	None	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1 Functional Testing	يجب تعبئة الحقول الإلزامية للمواطن والتأكد من عدم تكرار الحساب.
FR 10	Garage manager can be update information citizen	FR9	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1 Functional Testing	يجب أن تكون البيانات المحدثة للمواطن صحيحة ولا تسبب تعارضات.

FR 11	Garage manager can be delete citizen	FR9	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1.9 Delete Citizen Test Case Table	يجب عدم وجود تذاكر فعالة أو رحلات قائمة للمواطن عند الحذف.
FR 12	Garage manager can be show citizen	FR9	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1 Functional Testing	يتيح النظام عرض بيانات الحساب والتحقق من وجوده قبل أي عملية.
FR 13	Driver and citizen can be send(submit) complaints	None	Analytical study	Architectural Design	complaint service	8.2.1.5 Submit Complaint Test Case Table	يجب على المواطن أو السائق تسجيل دخول لإضافة شكوى على النظام.
FR 14	Driver and citizen can be show complaints	FR13	Analytical study	Architectural Design	complaint service	8.2.1.5 Submit Complaint Test Case Table	يتيح عرض وتتبع حالة الشكاوى المقدمة ومراقبة الردود عليها.
FR 15	Garage manager can be view user complaints	FR13	Analytical study	Architectural Design	complaint service	8.2.1.5 Submit Complaint Test Case Table	يتيح لمدير الكراج استعراض كافة الشكاوى الواردة من المستخدمين.
FR 16	Garage manager can be respond complaints	FR15	Analytical study	Architectural Design	complaint service	8.2.1.6 Reply to Complaint Test Case Table	يجب على المدير كتابة رد واغلاق الشكوى وتغيير حالتها برمجياً.
FR 17	Drive can be view trip history	None	Analytical study	Architectural Design	7.2.4 Trip Service	8.2.1.8 View Trip History Test Case Table	يجب ان تكون الرحلات متاحة ومعتمدة من ادارة النظام لعرض تاريخها.

FR 18	Citizen can be view booking history	None	Analytical study	Architectural Design	7.2.3 Booking Service	8.2.1 Functional Testing	يتيح للمواطن استعراض سجل كافة الحجوزات السابقة والتذاكر الخاصة به.
FR 19	User can be view profile	None	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1 Functional Testing	يجب أن يكون المستخدم مسجل دخول لحسابه لعرض بيانات ملفه الشخصي.
FR 20	User can be edit profile	FR19	Analytical study	Architectural Design	7.2.2 Authentication Service	8.2.1.7 Update Profile Test Case Table	يجب أن تكون التعديلات متوافقة مع شروط حماية الحساب وتحديث البيانات.
FR 21	All actors can be search with filter	None	Analytical study	Architectural Design	7.6 User Interface Implementation	————	يجب ان يكون النظام متاح لتنفيذ المطلوب وتصفية البيانات بدقة عالية.

7-6-Test Case Descriptions

Add New Driver

Test Case ID	Test Case Name	Precondition	Steps	Expected Result
TC-01	Successfully Add New Driver	Active account exists	1. Open add interface 2. Enter data 3. Click Save	Message 'Saved successfully' appears and driver is listed
TC-02	Duplicate License Number		1. Open add interface 2. Enter data 3. Click Save	Error message (Unique Constraint Error)

Edit Driver Data

Test Case ID	Test Case Name	Precondition	Steps	Expected Result
TC-03	Edit Driver Successfully	Admin logged in	1. Open driver management 2. Enter new data 3. Save	Data updated in database
TC-04	Invalid Data Entry		1. Open driver management 2. Enter new data 3. Save	Invalid data message
TC-05	Empty Fields		Enter incomplete data	Error: All fields required

Bus Management

Test Case ID	Test Case Name	Precondition	Steps	Expected Result
TC-06	Add Bus		1. Open buses interface 2. Click add 3. Save	Bus added with status Available
TC-07	Delete Bus	Bus is available	1. Select bus 2. Click Delete 3. Confirm	Bus removed from list

Complaint

Test Case ID	Test Case Name	Steps	Expected Result
TC-08	Submit Complaint	1. Open complaints 2. Add complaint 3. Confirm	Saved as Pending with tracking number
TC-09	Reply to Complaint	1. Open complaints list 2. Write reply	Status becomes Replied and notification sent

Profile Management

Test Case ID	Test Case Name	Steps	Expected Result
TC-10	Change Password	1. Open profile 2. Enter old password 3. Enter new	Password updated if old correct

		password twice 4. Confirm	

Part Three: Design and Architecture

Chapter 5

Architectural Design

5.1 High-Level System Design

This chapter presents the architectural and technical design of the proposed system. The purpose of this chapter is to describe how the system components interact together in order to satisfy the functional and non-functional requirements identified in the previous chapters.

The system was designed using a modular and scalable architecture to ensure maintainability, flexibility, and ease of future expansion. The overall architecture follows the Microservices Architectural Pattern, where each service is independently developed, deployed, and maintained.

5.1.1 System Block Diagram

The following figure illustrates the high-level architectural design of the proposed system and the interaction between the major system components.

The system architecture consists of four primary layers:

- Presentation Layer
- API Gateway Layer
- Microservices Layer
- Database Layer

The Presentation Layer represents the user interface developed using React and Vite technologies. This layer allows administrators, drivers, and citizens to interact with the system through a modern web interface.

The API Gateway layer acts as a centralized entry point for all client requests. The gateway is implemented using Ocelot and is responsible for request routing, authentication forwarding, and communication management between the frontend and backend services.

The Microservices Layer contains several independent services, each responsible for a specific business domain such as authentication, booking management, complaint handling, notification delivery, route planning, trip management, user management, and vehicle management.

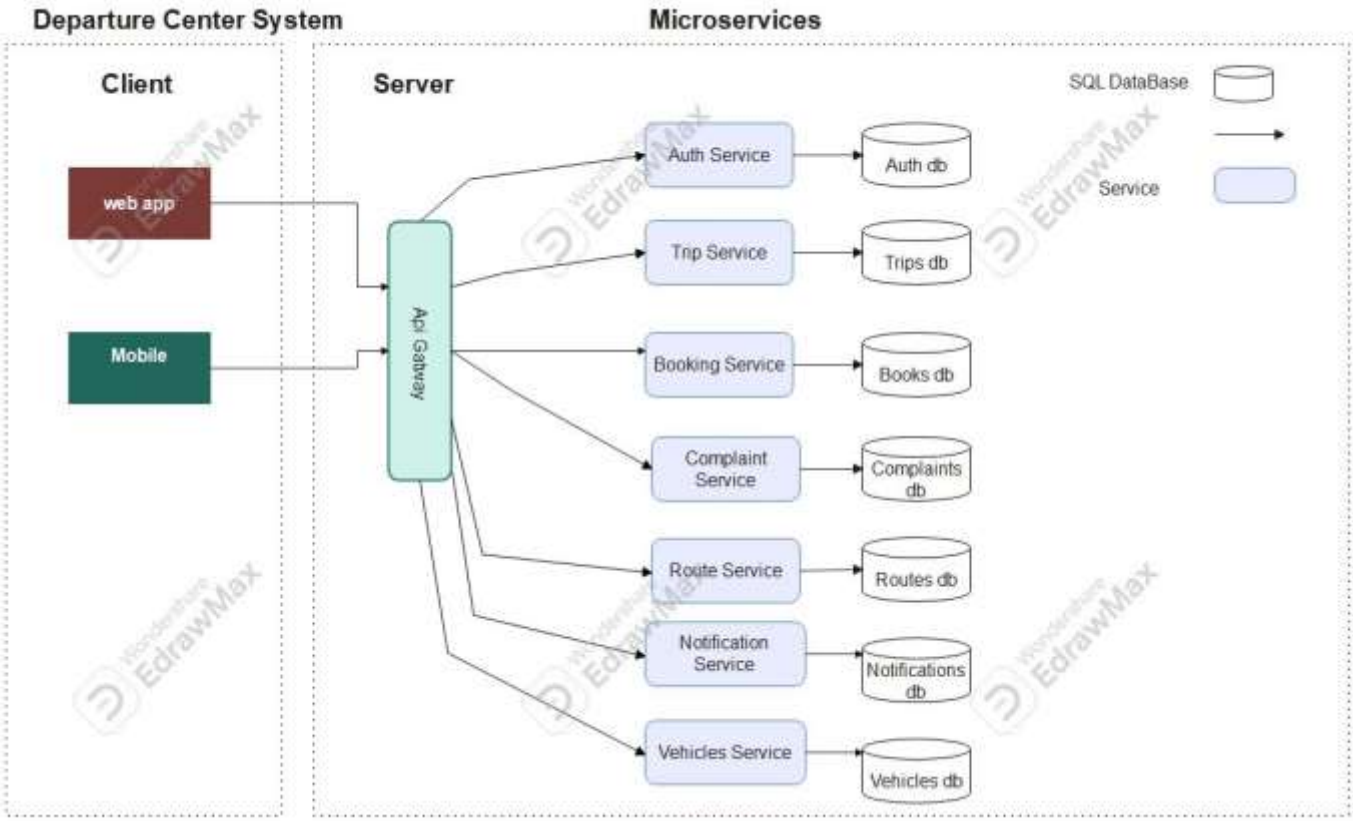
Each microservice owns its dedicated database in order to achieve loose coupling and independent scalability. This approach improves system reliability and follows modern distributed system design principles.

5.1.2 Architectural Pattern

The proposed system adopts the **Microservices** Architecture pattern. This architectural style divides the system into a collection of small, independent services that communicate through lightweight APIs.

The decision to use microservices was motivated by several factors:

- Improved scalability
- Independent deployment of services
- Easier maintenance and debugging
- Better fault isolation
- Support for distributed development teams
- Flexibility in future system expansion



5.1.3 System Architecture Description

The frontend application communicates with the backend services through RESTful APIs exposed by the API Gateway. After receiving a request, the gateway forwards the request to the appropriate microservice.

Authentication and authorization operations are handled by the AuthService using JWT–based authentication mechanisms. Once authenticated, users can access different system functionalities according to their assigned roles and permissions.

The BookingService manages trip reservations and scheduling operations, while the RouteService handles route planning and optimization functionalities. The NotificationService is responsible for sending alerts and notifications to users regarding bookings, complaints, or system events.

The VehicleService manages fleet information and vehicle–related operations, while the ComplaintService enables users to submit and track complaints or feedback related to transportation services.

The architecture was designed to ensure high cohesion within services and loose coupling between services, which improves maintainability and system performance.

5.2 Database Design

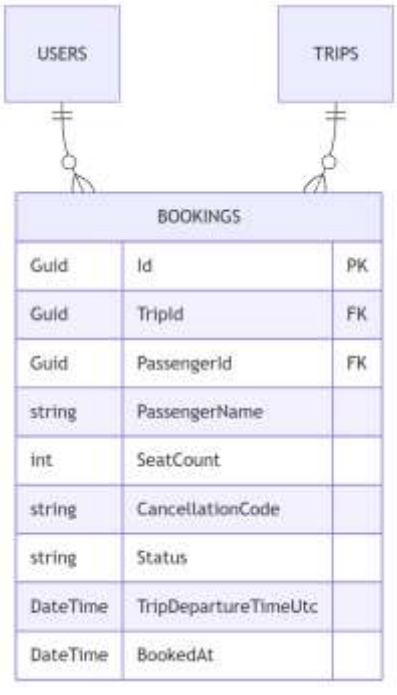
The database design defines how system data is structured, stored, and managed. Each microservice maintains its own independent database following the Database-per-Service pattern commonly used in microservices architecture.

This separation improves service autonomy, enhances scalability, and minimizes dependencies between system components.

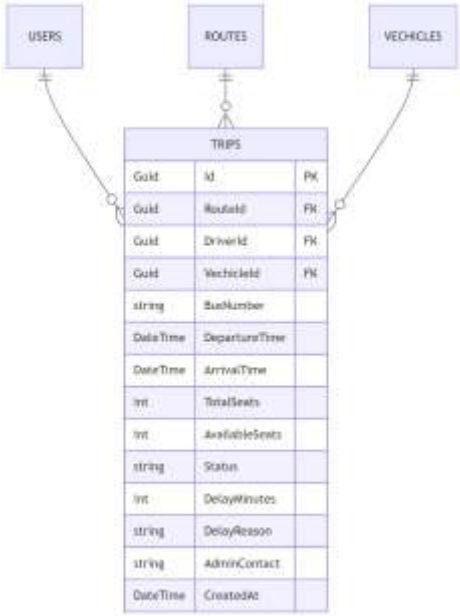
5.2.1 Entity Relationship Diagram (ERD)

The following diagram illustrates the relationships between the main entities used within the system database.

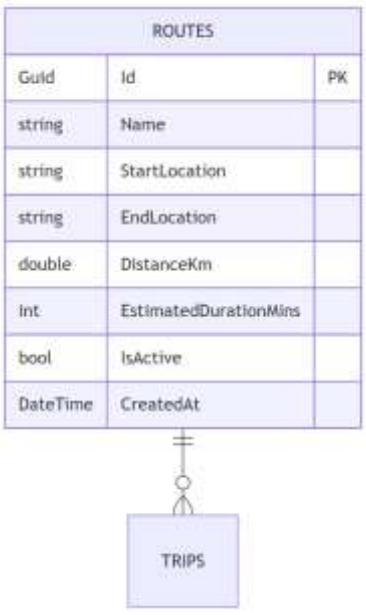
Bookings service



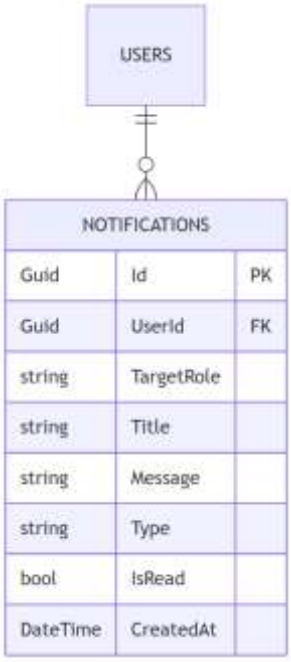
Trips service



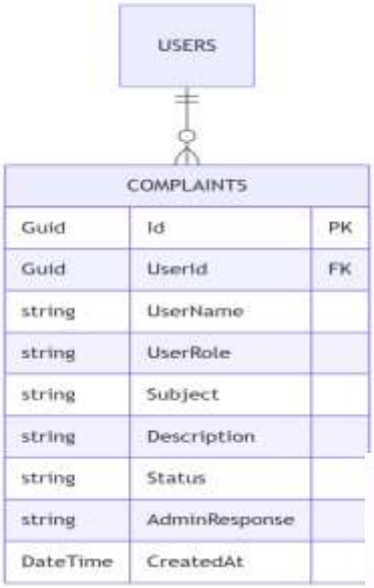
Routes service



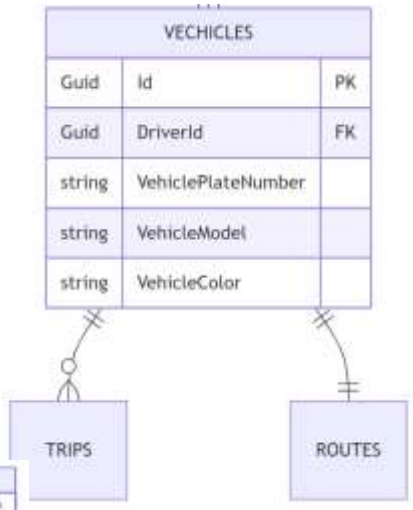
Notifications Service



Complaints service



Vehicles service



Auth service

*5.2.*2 Relational Schema

The relational schema defines the database tables, attributes, and relationships used by the system.

5.2.3 Database Constraints(primary key, Foreign Key, Unique, Check)

Database constraints are rules applied to the database schema to ensure data integrity, consistency, and validity. In the proposed system, Entity Framework Core enforces these constraints at the model level and translates them into corresponding database–level constraints.

These constraints play a critical role in maintaining correct relationships between entities and preventing invalid data operations.

Primary Key Constraint

The Primary Key uniquely identifies each record within a table. Every entity in the system has a primary key, typically represented using a Guid field named Id.

- Ensures each record is uniquely identifiable
- Cannot contain NULL values
- Must be unique for each row

Example:

- Users.Id
- Trips.Id
- Bookings.Id

Foreign Key Constraint

Foreign Keys are used to establish relationships between tables and maintain referential integrity across the system.

They ensure that a record in one table correctly references a valid record in another table.

Examples from the system:

- Bookings.PassengerId → Users.Id
- Trips.DriverId → Users.Id
- Trips.RouteId → Routes.Id
- Complaints.UserId → Users.Id
- Notifications.UserId → Users.Id
- DriverProfiles.DriverId → Users.Id

These relationships ensure consistency across microservices at the logical data level.

Unique Constraint

The Unique constraint ensures that no duplicate values are stored in specific fields where uniqueness is required.

Examples:

- Users.Email → ensures no two users share the same email address
- Routes.Name → ensures each route has a unique name

- Bookings.CancellationCode → ensures each booking cancellation code is unique
- DriverProfiles.DriverId → ensures one driver has only one profile

Check Constraint

Check constraints are used to enforce domain-level validation rules on column values.

They ensure that values stored in the database meet predefined conditions.

Examples:

- AvailableSeats >= 0 in Trips table
- TotalSeats > 0
- Enum fields (e.g., Status, Role) restricted to predefined valid values

These constraints help prevent invalid business logic from being stored in the database.

5.3 API Design

The system exposes RESTful APIs to enable communication between the frontend application and backend microservices.

Each API endpoint follows REST architectural principles and uses standard HTTP methods such as: (GET – POST – PUT – DELETE)

Authentication APIs

Endpoint	Method	Description	Parameters	Return Value	Error Codes
/api/auth/register	POST	Register a new user	User registration data	JWT token and user information	400, 409
/api/auth/login	POST	Authenticate user login	Email, password	JWT access token	400, 401

User Management APIs

Endpoint	Method	Description	Parameters	Return Value	Error Codes
/api/users/drivers	GET	Retrieve all drivers	JWT token	List of drivers	401, 403

/api/users/drivers	POST	Create a new driver	Driver information	Created driver object	400, 401
/api/users/drivers/{id}	PUT	Update driver information	Driver ID, updated data	Updated driver object	400, 404
/api/users/drivers/{id}	DELETE	Delete a driver	Driver ID	Deletion confirmation	404
/api/users	GET	Retrieve all users	JWT token	List of users	401, 403
/api/users/by-ids	POST	Retrieve users by IDs	List of user IDs	User objects	400
/api/users/{id}	GET	Retrieve user by ID	User ID	User information	404
/api/users/{id}	PUT	Update user information	User ID, updated data	Updated user object	400, 404
/api/users/{id}	DELETE	Delete user	User ID	Deletion confirmation	404
/api/users/me	GET	Retrieve current user profile	JWT token	Current user profile	401

/api/users/me	PUT	Update current user profile	Updated profile data	Updated profile	400, 401
/api/users/admin-contact	GET	Retrieve administrator contact information	JWT token	Contact information	404

5.3.2 BookingService APIs

Endpoint	**Method**	**Description**	**Parameters**	**Return Value**	**Error Codes**
/api/booking	GET	Retrieve all bookings	JWT token	List of bookings	401
/api/booking/{id}	GET	Retrieve booking by ID	Booking ID	Booking details	404
/api/booking/my	GET	Retrieve current user's bookings	JWT token	User bookings	401
/api/booking/my/active	GET	Retrieve active bookings	JWT token	Active bookings	401

/api/booking/my/history	GET	Retrieve booking history	JWT token	Historical bookings	401
/api/booking/trip/{tripId}	GET	Retrieve bookings for a trip	Trip ID	Trip bookings	404
/api/booking	POST	Create a new booking	Booking information	Booking details	400, 409
/api/booking/cancel	POST	Cancel a booking	Booking ID	Cancellation confirmation	400, 404

5.3.3 ComplaintService APIs

Endpoint	**Method**	**Description**	**Parameters**	**Return Value**	**Error Codes**
/api/complaint	GET	Retrieve all complaints	JWT token	List of complaints	401
/api/complaint/{id}	GET	Retrieve complaint by ID	Complaint ID	Complaint details	404

/api/complaint/my	GET	Retrieve current user's complaints	JWT token	User complaints	401
/api/complaint	POST	Create a new complaint	Complaint information	Complaint details	400
/api/complaint/{id}/status	PATCH	Update complaint status	Complaint ID, status	Updated complaint	400, 404
/api/complaint/{id}/respond	PATCH	Respond to a complaint	Complaint ID, response	Updated complaint	400, 404

5.3.4 RouteService APIs

Endpoint	Method	Description	Parameters	Return Value	Error Codes
/api/route	GET	Retrieve all routes	JWT token	List of routes	401
/api/route/{id}	GET	Retrieve route by ID	Route ID	Route details	404
/api/route	POST	Create a new route	Route information	Created route	400

/api/route/{id}	PUT	Update route information	Route ID, updated data	Updated route	400, 404
/api/route/{id}	DELETE	Delete route	Route ID	Deletion confirmation	404

5.3.5 TripService APIs

Endpoint	**Method**	**Description**	**Parameters**	**Return Value**	**Error Codes**
/api/trip	GET	Ret all trips	JWT token	List of trips	401
/api/trip/{id}	GET	Ret trip by ID	Trip ID	Trip details	404
/api/trip/driver/{driverId}	GET	Ret trips by driver	Driver ID	Driver trips	404
/api/trip/route/{routeId}	GET	Ret trips by route	Route ID	Route trips	404
/api/trip/active	GET	Retrieve active trips	JWT token	Active trips	401
/api/trip/history	GET	Ret trip history	JWT token	Historical trips	401

/api/trip/driver/{driverId}/history	GET	Ret driver's trip history	Driver ID	Trip history	404
/api/trip/route/{routeId}/exists	GET	Check route availability	Route ID	Boolean value	404
/api/trip	POST	Create a new trip	Trip information	Created trip	400
/api/trip/{id}	PUT	Update trip information	Trip ID, updated data	Updated trip	400, 404
/api/trip/{id}/status	PATCH	Update trip status	Trip ID, status	Updated trip	400, 404
/api/trip/{id}	DELETE	Delete trip	Trip ID	Deletion confirmation	404
/api/trip/{id}/decrement-seat	POST	Decrease available seats	Trip ID	Updated seat count	400

/api/trip/{id}/increment-seat	POST	Increase available seats	Trip ID	Updated seat count	400

Driver Profile APIs

Endpoint	Method	Description	Parameters	Return Value	Error Codes
/api/driver/profile/{driverId}	GET	Retrieve driver profile	Driver ID	Driver profile	404
/api/driver/profile/me	GET	Retrieve current driver's profile	JWT token	Driver profile	401
/api/driver/profile/admin/all	GET	Retrieve all driver profiles	JWT token	List of driver profiles	401, 403

/api/driver/profile	POST	Create driver profile	Profile information	Created profile	400
/api/driver/profile/{driverId}	PUT	Update driver profile	Driver ID, updated data	Updated profile	400, 404
/api/driver/profile/me	PUT	Update current driver's profile	Updated profile data	Updated profile	400
/api/driver/profile/{driverId}	DELETE	Delete driver profile	Driver ID	Deletion confirmation	404

5.3.6 VehicleService APIs

Endpoint	Method	Description	Parameters	Return Value	Error Codes
/api/vehicle	GET	Retrieve all vehicles	JWT token	List of vehicles	401
/api/vehicle/{id}	GET	Retrieve vehicle by ID	Vehicle ID	Vehicle details	404
/api/vehicle	POST	Create a new vehicle	Vehicle information	Created vehicle	400

/api/vehicle/{id}	PUT	Update vehicle information	Vehicle ID, updated data	Updated vehicle	400, 404
/api/vehicle/{id}	DELETE	Delete vehicle	Vehicle ID	Deletion confirmation	404

5.3.7 HTTP Response Codes

Status Code	Meaning
200 OK	Request completed successfully
201 Created	Resource created successfully
400 Bad Request	Invalid request data
401 Unauthorized	Authentication required
403 Forbidden	Access denied
404 Not Found	Requested resource not found
409 Conflict	Data conflict occurred
500 Internal Server Error	Unexpected server-side error

5.4 Security and Authorization Design

Security and access control were considered critical aspects during the design and implementation of the proposed system. The system was designed to ensure secure communication between users and backend services while protecting sensitive information and restricting unauthorized access.

The security architecture is based on authentication, authorization, role-based access control, and secure API communication mechanisms.

5.4.1 Authentication Mechanism

The system uses JSON Web Token (JWT) authentication to verify user identities and secure access to protected resources.

When a user successfully logs into the system, the AuthService generates a JWT access token containing the user's identity and assigned role. The generated token is then returned to the client application and attached to subsequent API requests using the Authorization header.

The token contains encoded claims such as:

- User Identifier
- User Role
- Token Expiration Time
- Security Metadata

This mechanism enables stateless authentication and improves scalability in the microservices architecture.

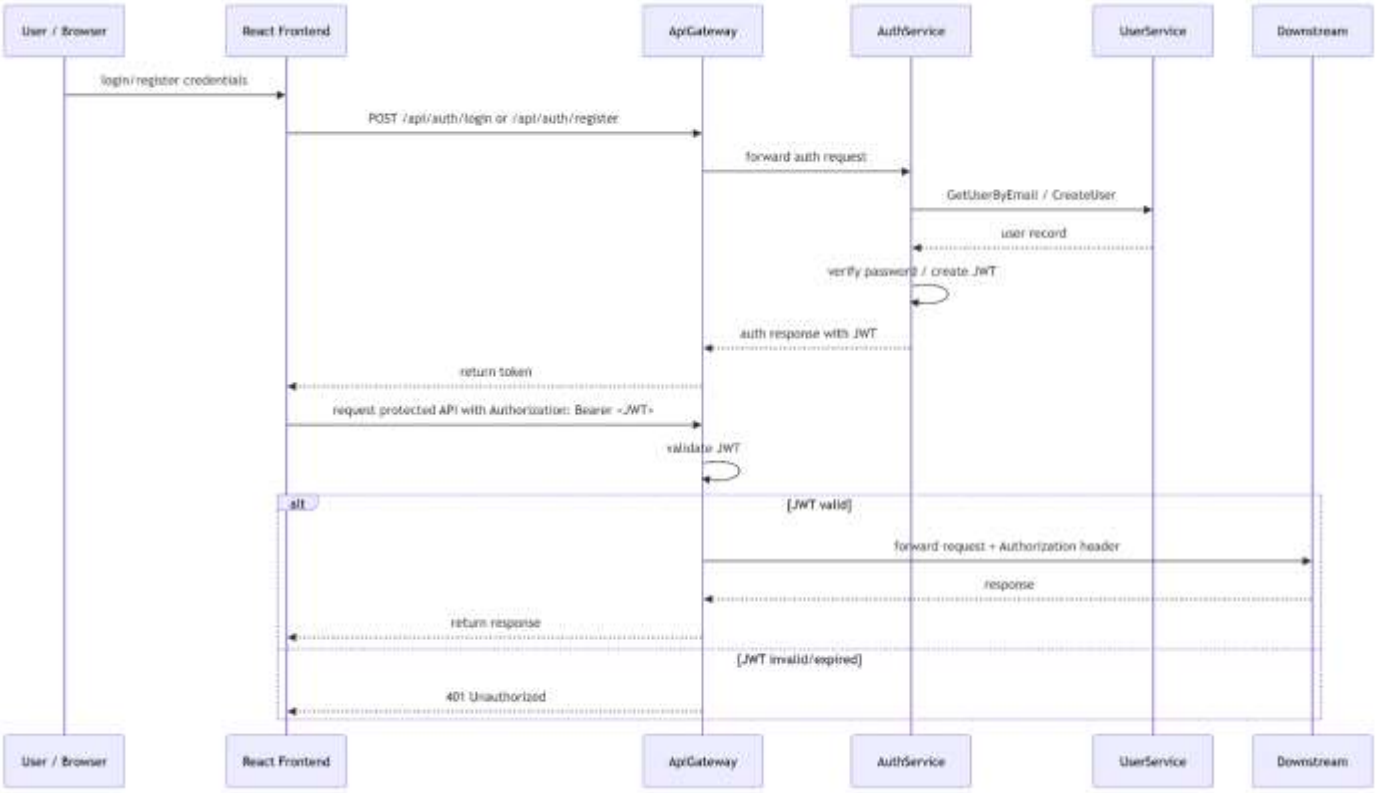


Figure 44 jwt Athentication and Authorization

The authentication workflow follows these steps:

1. The user submits login credentials through the frontend application.
2. The AuthService validates the provided credentials.
3. A JWT access token is generated upon successful authentication.
4. The frontend stores the token securely and includes it in future requests.
5. The API Gateway validates the token before forwarding requests to backend services.

This approach improves security and minimizes repeated authentication requests between services.

5.4.2 Authorization and Access Control

The system implements Role-Based Access Control (RBAC) to regulate access to system resources and functionalities.

Each authenticated user is assigned a specific role that determines the operations and resources they are permitted to access.

The main user roles implemented in the system are:

Role	Description
Admin	Has full access to system management functionalities
Driver	Manages trips, driver profile, and transportation-related operations
User	Can book trips, submit complaints, and manage personal information

Authorization rules are enforced at both the API Gateway and microservice levels to ensure secure access control.

Protected endpoints require valid authentication tokens and appropriate permissions before access is granted.

5.4.3 Access Levels

Different access levels were implemented to protect sensitive operations and system resources.

Administrative Access

Administrative users have elevated privileges that allow them to:

- Manage users and drivers
- Create and update routes
- Manage vehicles and trips
- View system–wide data and reports
- Handle complaints and notifications

Driver Access

Drivers are granted permissions related to transportation management, including:

- Viewing assigned trips
- Managing driver profiles
- Updating trip statuses
- Accessing route information

User Access

Regular users can perform standard platform operations such as:

- Booking trips
- Viewing booking history

- Submitting complaints
- Managing personal profile information

5.4.4 API Security Measures

Several additional security measures were implemented to improve system protection, including:

- Password hashing
- Secure JWT token validation
- HTTP based communication
- Input validation and request sanitization
- Protected API routes
- Token expiration handling
- Exception and error handling mechanisms

Sensitive operations such as updating passwords or deleting records require authenticated requests and proper authorization privileges.

5.4.5 Security Architecture Overview

The overall security architecture ensures confidentiality, integrity, and controlled access to system resources while maintaining scalability and performance within the distributed microservices environment.

The combination of JWT authentication, role–based authorization, API Gateway validation, and protected service communication provides a secure foundation for the proposed transportation management system.

Part Four:

Implmentation and Testing

Chapter 6

DEVELOPMENT ENVIRONMENT AND TOOLS

6.1 Introduction

This chapter presents the development environment, tools, frameworks, and technologies used in the implementation of the Departure Center System. The objective of this chapter is to provide a complete technical overview of the system setup in order to ensure reproducibility and maintainability.

6.2 Development Tools and Software

The following tools were used during system development:

- **Visual Studio Code (VS Code)**: Primary code editor used for both frontend and backend development.
- **Postman**: Used for testing and validating RESTful API endpoints.
- **Docker**: Used for containerizing microservices and managing system dependencies.
- **k6**: Used for performance and load testing of the system APIs.

These tools were essential in ensuring efficient development, testing, and deployment workflows.

6.3 Development Environment and System Structure

The system was developed in a hybrid environment using both Windows OS and Windows Subsystem for Linux (WSL). WSL was primarily used to run Docker containers efficiently.

The project structure follows a microservices–based architecture with a clear separation between frontend, backend, and testing components.

6.3.1 Root Directory Structure

The root directory of the project contains the main system components as follows:

DepartureCenterSystemCopy/

├── backend/ # All backend microservices

├── frontend/ # React frontend application

├── k6-tests/ # Performance testing scripts

This structure ensures clear separation between system layers and simplifies development and deployment processes.

6.3.2 Backend Structure

The backend is implemented using a microservices architecture. Each service is independently developed and deployed.

backend/

├── ApiGateway/

├── AuthService/

├── BookingService/

├── ComplaintService/

├── NotificationService/

├── RouteService/

├── TripService/

├── VehicleService/

Each service is responsible for a specific business domain, and all communication between services is handled through the API Gateway.

6.3.3 Microservice Internal Architecture

Each microservice follows a Clean Architecture pattern consisting of four main layers:

```
ServiceName/
├── ServiceName.Api/              # REST API layer (Controllers, Middleware)
├── ServiceName.Application/      # Business logic (Services, Interfaces, DTOs)
├── ServiceName.Domain/           # Domain models (Entities, Enums, Interfaces)
└── ServiceName.Infrastructure/   # Data access & external services (Persistence, HttpClients)
```

This layered architecture ensures:

- Separation of concerns
- High maintainability
- Testability of business logic
- Independent development of layers

6.3.4 Frontend Structure

The frontend is built using React with Vite and follows a modular component-based structure.

```
frontend/
├── public/                       # Static assets
├── src/
│   ├── components/               # Reusable React components
```

```
│   ├── context/              # React Context for state management
│   ├── data/                 # Mock/sample data
│   └── assets/               # Images, fonts, etc.
```

This structure improves code reusability and maintains a clean separation between UI components and application logic.

6.3.5 Performance Testing Structure

Performance testing is implemented using k6 and organized in a dedicated directory:

```
k6-tests/
├── utils/                    # Test utilities and helpers
└── scenarios/                # Test scenarios and configurations
```

This structure allows the definition of multiple load testing scenarios and reusable testing utilities to evaluate system performance under different conditions.

6.4 Programming Languages and Frameworks

The system was implemented using the following technologies:

Backend

- **Language**: C#
- **Framework**: .NET 8
- **Architecture**: Clean Architecture (4-layer structure)

Frontend

- **Languages**: HTML, CSS, JavaScript

- ***Framework***: React (Vite-based setup)

API Communication

- RESTful APIs
- JSON-based data exchange
- HTTP protocols

6.5 Database Server and Operating System

Database

- **SQL Server** was used as the primary database management system.
- Each microservice has its own dedicated database to ensure data isolation and scalability.

Operating System

- **Windows 10** was used as the main development environment.
- **WSL** was used to support Docker container execution and Linux-based tooling.

6.6 Dependencies and Libraries

The frontend project uses a modern JavaScript ecosystem managed through package.json.

Frontend Dependencies

```
{
  "react": "^19.2.0",
```

```
  "react-dom": "^19.2.0",
  "react-router-dom": "^7.13.1",
  "axios": "^1.13.6",
  "tailwindcss": "^4.2.1",
  "@tailwindcss/vite": "^4.2.1"
}
```

Development Dependencies

```
{
  "vite": "^7.3.1",
  "eslint": "^9.39.1",
  "@vitejs/plugin-react": "^5.1.1",
  "@types/react": "^19.2.7",
  "@types/react-dom": "^19.2.3"
}
```

These dependencies support:

- UI development (React)
- Styling (TailwindCSS)
- API communication (Axios)
- Routing (React Router)
- Code quality (ESLint)
- Build tooling (Vite)

6.7 Version Control and Source Code Management

The system uses:

- **Git** for version control
- **GitHub** for repository hosting and collaboration

The project follows a structured branching strategy to manage development and feature integration. Each microservice and frontend module is tracked independently to ensure modular development.

6.8 CI/CD Tools

Continuous Integration and Continuous Deployment (CI/CD) tools such as GitHub Actions or Jenkins are planned for future integration.

Currently, the system does not include a fully automated CI/CD pipeline.

6.9 Testing Environment

The system was tested in a local development environment.

Testing Setup

- Backend services were containerized using **Docker**
- Services were executed locally using **localhost**
- API testing was performed using **Postman**
- Load and performance testing were conducted using **k6**

Performance Testing

k6 scripts were used to simulate multiple concurrent users in order to evaluate system performance under load conditions.

Execution Environment

- Local machine (using WSL)
- Docker containers running microservices locally

6.10 Summary

The development environment was carefully selected to support scalability, modularity, and efficient microservices development. The combination of .NET 8, React, SQL Server, Docker, and modern development tools provided a robust foundation for implementing the Departure Center System.

Chapter 7

Software Implementation

7.1 Project and File Structure

The DepartureCenterSystem project follows a modular microservices architecture based on the Clean Architecture pattern. The system is divided into independent backend services, a frontend client application, and performance testing modules. This structure improves scalability, maintainability, and separation of concerns.

Root Directory Structure

```
DepartureCenterSystemCopy/
├── backend/                    # All backend microservices
├── frontend/                   # React frontend application
├── k6-tests/                   # Performance testing scripts
```

Backend Structure

The backend layer contains all microservices responsible for handling business operations and system functionalities.

```
backend/
├── ApiGateway/
├── AuthService/
├── BookingService/
├── ComplaintService/
├── NotificationService/
├── RouteService/
├── TripService/
├── VehicleService/
```

Each microservice follows a standardized 4-layer Clean Architecture structure:

```
ServiceName/
├── ServiceName.Api/                # REST API Layer
├── ServiceName.Application/        # Business Logic Layer
├── ServiceName.Domain/             # Domain Models Layer
└── ServiceName.Infrastructure/     # Data Access & External Services Layer
```

Layer Responsibilities

Layer	Responsibility
API Layer	Handles HTTP requests, controllers, middleware, and routing
Application Layer	Contains business logic, DTOs, interfaces, and services
Domain Layer	Defines entities, Enums, and core domain models
Infrastructure Layer	Manages database access, repositories, migrations, and external integrations

Frontend Structure

The frontend application was implemented using React with Vite.

```
frontend/
├── public/                         # Static files
├── src/
│   ├── components/                 # Reusable UI components
│   ├── context/                    # Global state management
│   ├── data/                       # Mock/sample data
│   └── assets/                     # Images and static resources
```

Performance Testing Structure

The system uses k6 for load and performance testing.

```
k6-tests/
├── utils/                          # Shared test utilities
└── scenarios/                      # Load testing scenarios
```

Service Communication Structure

The system uses an API Gateway pattern for centralized routing between the frontend and backend services.

```
User Request
    ↓
Frontend (React)
    ↓
ApiGateway (Ocelot)
    ├──→ AuthService
    ├──→ BookingService
    ├──→ ComplaintService
    ├──→ NotificationService
    ├──→ RouteService
    ├──→ TripService
    └──→ VehicleService
```

Each microservice owns a separate SQL Server database to ensure service isolation and independent deployment.

7.2 Modules Implementation

The system was implemented using independent microservices, where each service is responsible for a specific business domain. This modular design improves maintainability, scalability, and fault isolation.

7.2.1 API Gateway

The API Gateway was implemented using Ocelot to provide centralized request routing, authentication forwarding, and communication management between the frontend and backend services.

Main Responsibilities

- Request routing
- JWT authentication forwarding
- Reverse proxy functionality
- Unified API entry point
- Service abstraction from frontend clients

Example Configuration

```
"ReverseProxy": {
  "Routes": {
    "auth-route": {
      "ClusterId": "auth-cluster",
      "Match": {
        "Path": "/api/auth/{**catch-all}" }
      } },
"Clusters": {
    "auth-cluster": {
      "Destinations": {
```

```
        "destination1": {
          "Address": "http://authservice:8080/"
        }
      }
    }}
```

7.2.2 Authentication Service

The AuthService is responsible for user authentication and JWT token generation.

Main Functionalities

- User registration
- User login
- JWT token generation
- Role-based authorization
- Token validation

JWT Implementation Example

```
var tokenDescriptor = new SecurityTokenDescriptor
{
    Subject = new ClaimsIdentity(new[]
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Role, user.Role)
    }),
    Expires = DateTime.UtcNow.AddHours(2),
    SigningCredentials = new SigningCredentials(
```

```
        new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(jwtSettings.SecretKey)),
        SecurityAlgorithms.HmacSha256Signature)
};
```

7.2.3 Booking Service

The BookingService manages all booking-related operations within the system.

Main Functionalities

- Create bookings
- Cancel bookings
- Retrieve booking details
- Manage passenger reservations
- Validate booking states

Booking Creation Logic

```
public async Task<BookingDto> CreateBookingAsync(CreateBookingDto dto)
{
    var booking = new Booking
    {
        PassengerId = dto.PassengerId,
        TripId = dto.TripId,
```

```
        PassengerName = dto.PassengerName,
        Status = BookingStatus.Confirmed,
        BookedAt = DateTime.UtcNow
    };

    await _bookingRepository.AddAsync(booking);
    await _bookingRepository.SaveChangesAsync();

    return MapToDto(booking);
}
```

7.2.4 Trip Service

The TripService manages trip scheduling and trip-related operations.

Main Functionalities

- Create trips
- Manage trip schedules
- Assign drivers
- Retrieve trip information
- Validate trip capacity

Protected Endpoint Example

```
[HttpPost]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> Create(CreateTripDto request)
{
    var token = HttpContext.Request.Headers["Authorization"]
```

```
        .ToString()
        .Replace("Bearer ", "");
    var trip = await _tripService.CreateTripAsync(request, token);
    return CreatedAtAction(nameof(GetById), new { id = trip.Id }, trip);
}
```

7.2.5 Vehicle Service

The VehicleService is responsible for vehicle management and tracking.

Main Functionalities

- Vehicle registration
- Vehicle updates
- Vehicle assignment
- Capacity management

Controller Example

```
[HttpGet("{id}")]
public async Task<IActionResult> GetById(Guid id)
{
    var vehicle = await _vehicleService.GetVehicleByIdAsync(id);
    if (vehicle == null)
        return NotFound();
    return Ok(vehicle);
}
```

7.3 API Implementation

The system APIs were implemented using ASP.NET Core Web API following RESTful principles.

API Characteristics

- Attribute-based routing
- JWT authentication
- Role-based authorization
- Asynchronous operations
- Dependency injection
- Standard HTTP status codes

Controller Implementation Example

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IAuthenticationService _authService;
    public AuthController(IAuthenticationService authService)
    {
        _authService = authService;
    }
    [HttpPost("login")]
    [AllowAnonymous]
    public async Task<IActionResult> Login(LoginDto request)
    {
        var result = await _authService.LoginAsync(request);
```

```
        return Ok(result);
    }
}
```

HTTP Methods Used

Method	Purpose
GET	Retrieve resources
POST	Create resources
PUT	Update resources
DELETE	Remove resources

Common HTTP Status Codes

Status Code	Description
200 OK	Successful request
201 Created	Resource created successfully
400 Bad Request	Invalid request
401 Unauthorized	Authentication required
403 Forbidden	Access denied
404 Not Found	Resource not found
500 Internal Server Error	Unexpected server error

7.4 Data Access Layer Implementation

The data access layer was implemented using Entity Framework Core with SQL Server databases.

DbContext Example

```
public class BookingDbContext : DbContext
{
    public DbSet<Booking> Bookings { get; set; }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Booking>(entity =>
        {
            entity.HasKey(e => e.Id);
            entity.Property(e => e.PassengerName)
                .IsRequired()
                .HasMaxLength(150);
            entity.HasIndex(e => e.CancellationCode)
                .IsUnique();
        });
    }
}
```

Repository Pattern Implementation

```
public class BookingRepository : IBookingRepository
{
    private readonly BookingDbContext _dbContext;
    public BookingRepository(BookingDbContext dbContext)
    {
        _dbContext = dbContext;
    }
    public async Task<Booking?> GetByIdAsync(Guid id)
    {
        return await _dbContext.Bookings.FindAsync(id);
    }
    public async Task AddAsync(Booking booking)
    {
        await _dbContext.Bookings.AddAsync(booking);
    }
    public async Task SaveChangesAsync()
    {
        await _dbContext.SaveChangesAsync();
    }
}
```

EF Core Features Used

- DbContext and DbSet mapping
- Database migrations
- LINQ queries
- Asynchronous database operations

- Entity relationships and constraints

7.5 Business Logic Layer Implementation

The business logic layer was implemented inside the Application layer of each microservice.

Main Responsibilities

- Input validation
- Business rule enforcement
- DTO mapping
- Inter–service communication
- Exception handling
- Logging

Service Implementation Example

```
public class BookingManagementService : IBookingService
{
    private readonly IBookingRepository _bookingRepository;
    public BookingManagementService(
        IBookingRepository bookingRepository)
    {
        _bookingRepository = bookingRepository;
    }
    public async Task CancelBookingAsync(Guid id, string reason)
    {
        var booking = await _bookingRepository.GetByIdAsync(id);
        if (booking == null)
```

```
            throw new KeyNotFoundException("Booking not found");
        booking.Status = BookingStatus.Cancelled;
        booking.CancellationReason = reason;

        await _bookingRepository.UpdateAsync(booking);
        await _bookingRepository.SaveChangesAsync();
    }
}
```

7.6 User Interface Implementation

The frontend application was implemented using React and Vite to provide a responsive and interactive user interface.

Frontend Technologies

Technology	Purpose
React	User interface development
Vite	Frontend build tool
React Context API	State management
Axios	API communication
CSS	Styling and layout

Main Frontend Components

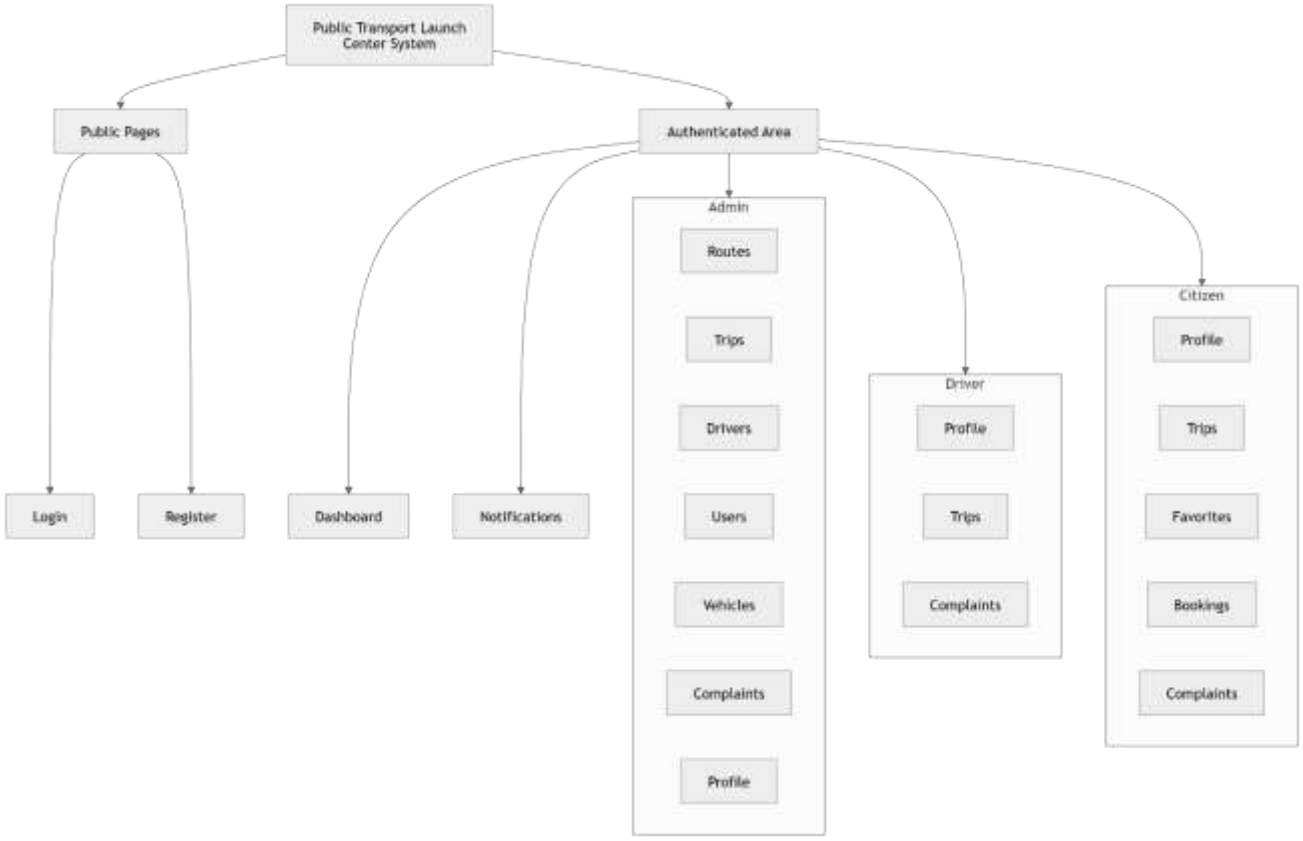
- Authentication pages
- Trip management pages
- Booking interfaces
- Complaint management
- Dashboard interfaces

Example React Component

```
function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const handleSubmit = async (e) => {
    e.preventDefault();
    await axios.post("/gateway/auth/login", {
      email,
      password
    });
  };
  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        onChange={(e) => setEmail(e.target.value)}
      />
      <input
        type="password"
        onChange={(e) => setPassword(e.target.value)}
```

```
      />
      <button type="submit">
        Login
      </button>
    </form>
  );
}
```

I.Site map


Public Transport Launch Center System
Public Pages
Authenticated Area
Login
Register
Dashboard
Notifications
Admin
Routes
Trips
Drivers
Users
Vehicles
Complaints
Profile
Driver
Profile
Trips
Complaints
Citizen
Profile
Trips
Favorites
Bookings
Complaints

Example React Component

1.register

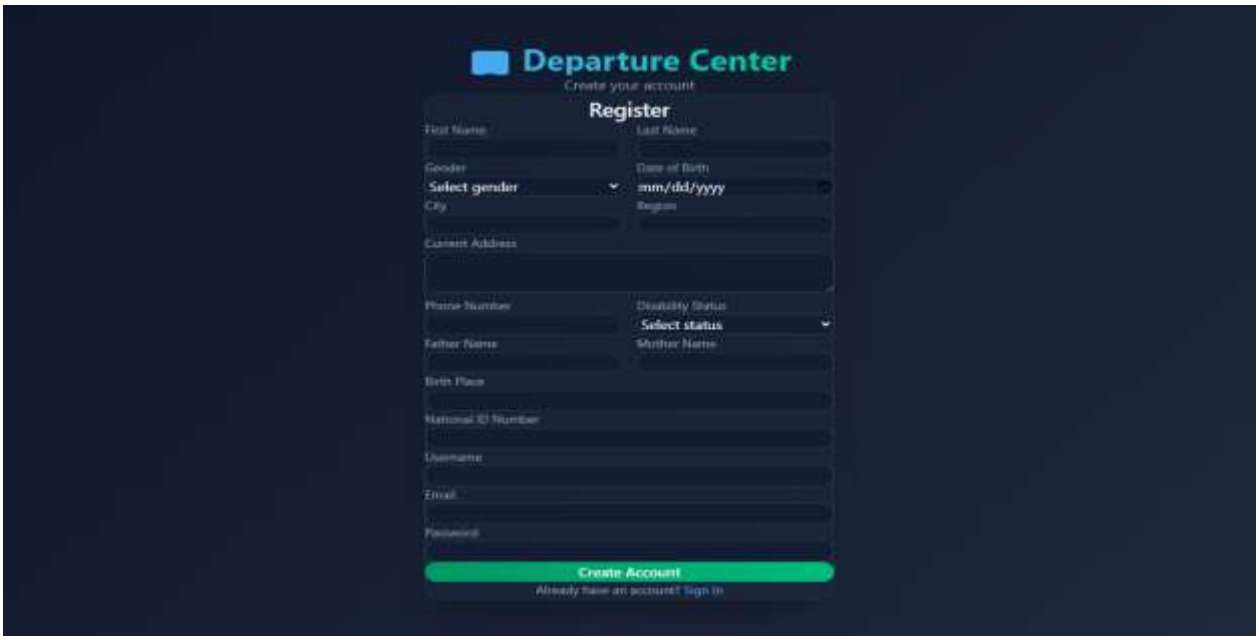


Figure 45 interface register

2.login

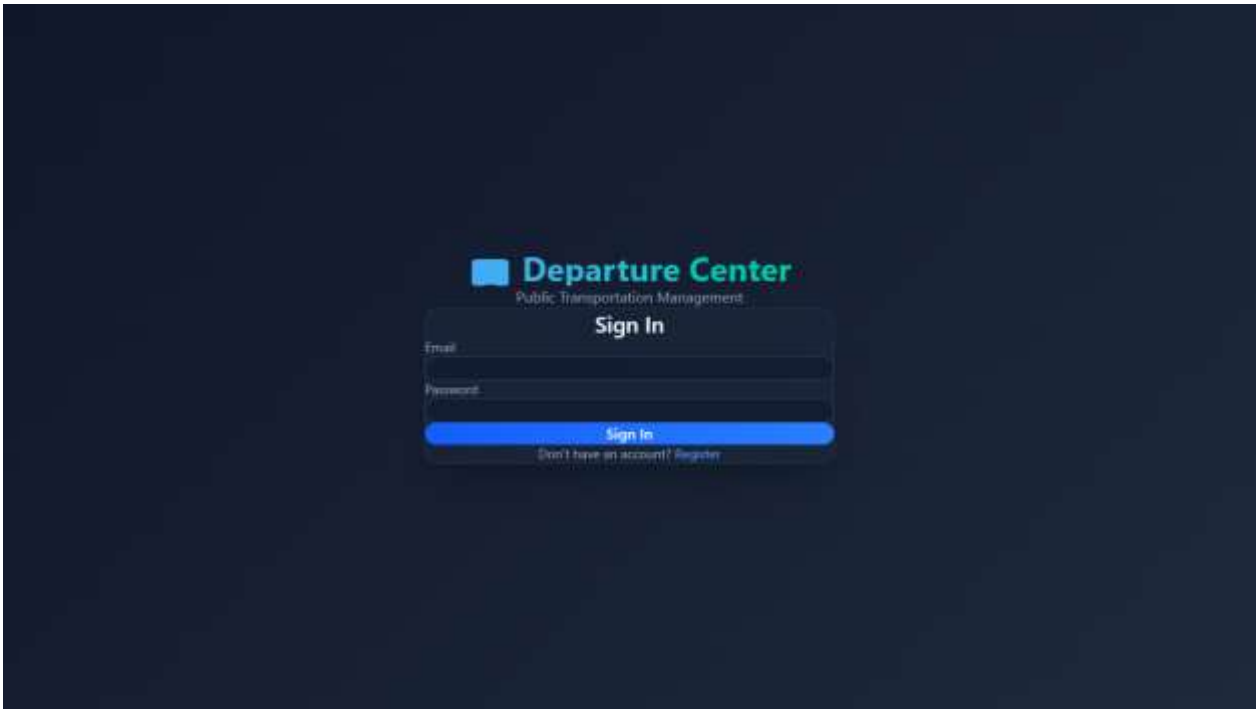


Figure 46 interface login

3.Admin Manage users

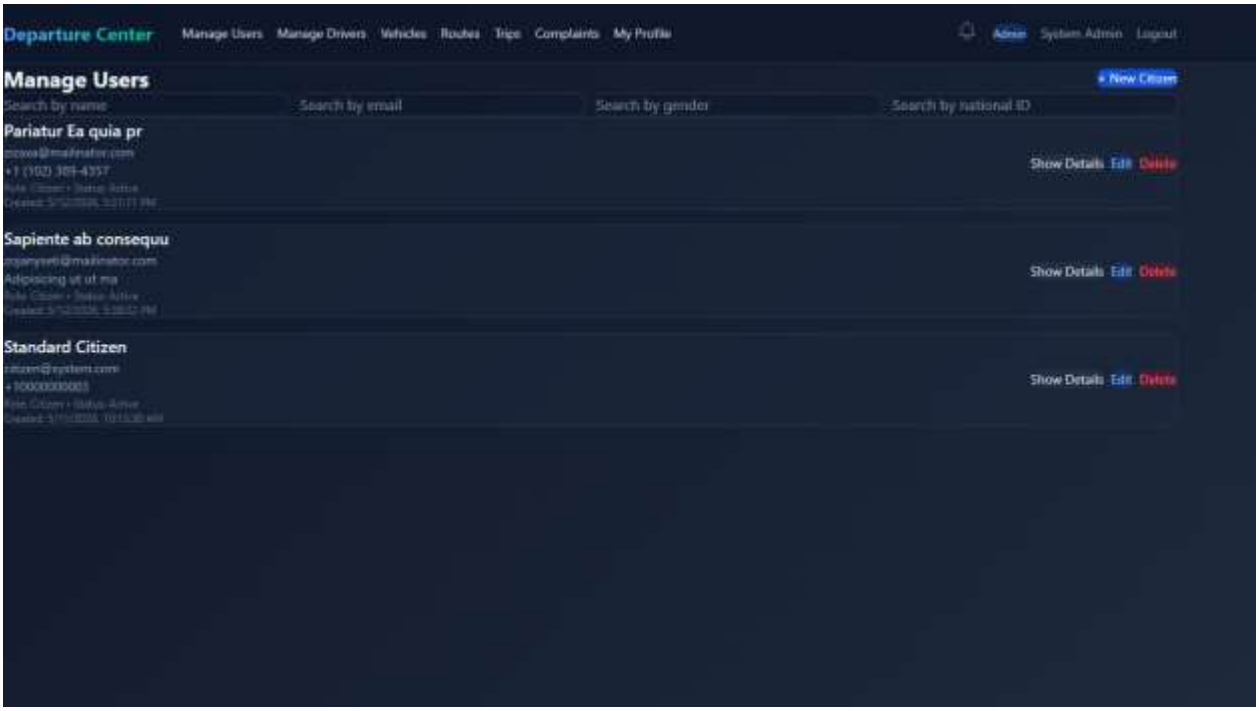


Figure 47 interfadce adman users

4.Admin add user

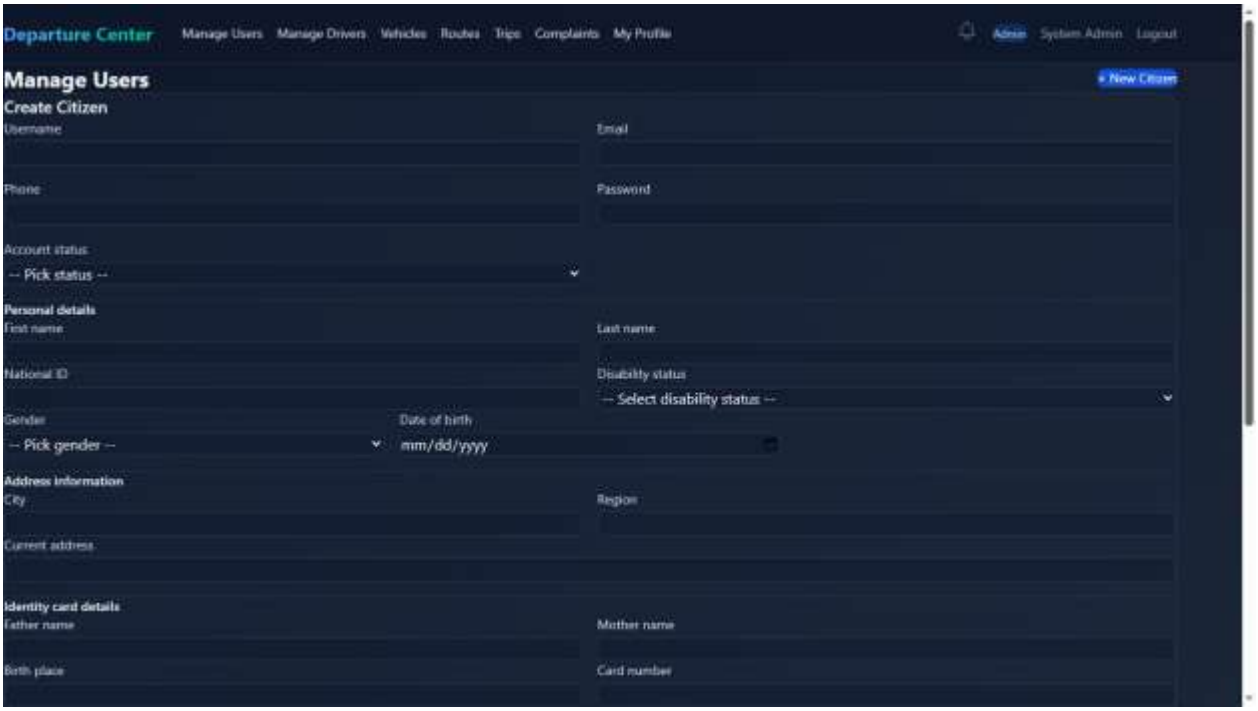


Figure 48 interface add user

5.Admin edit user

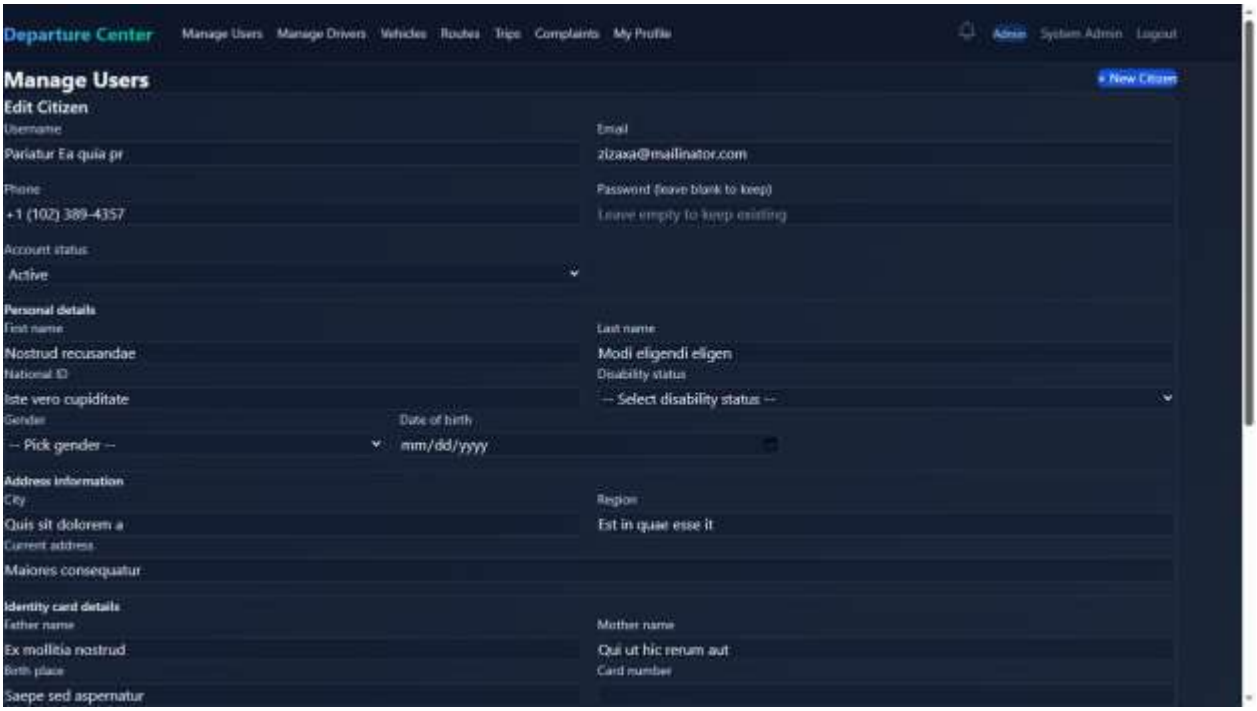


Figure 49 interface edit user

6.Admin select user (show user details)

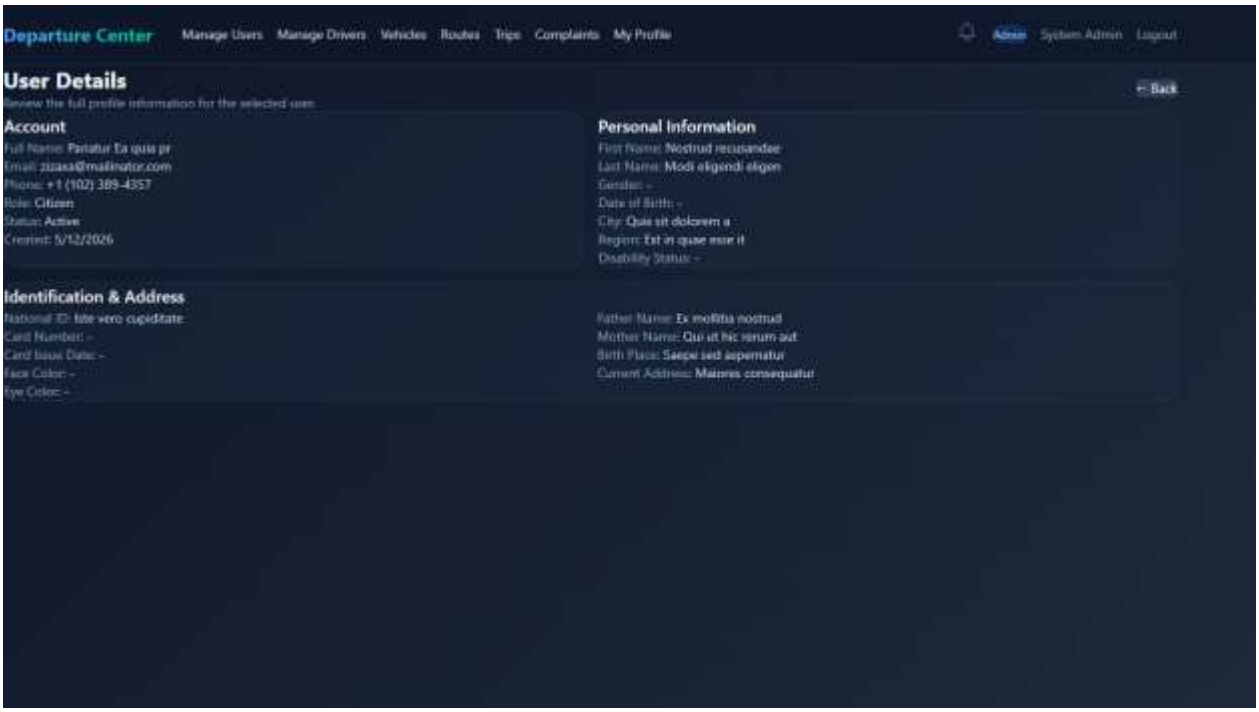


Figure 50 interface show user details

7.Admin Manage Drivers

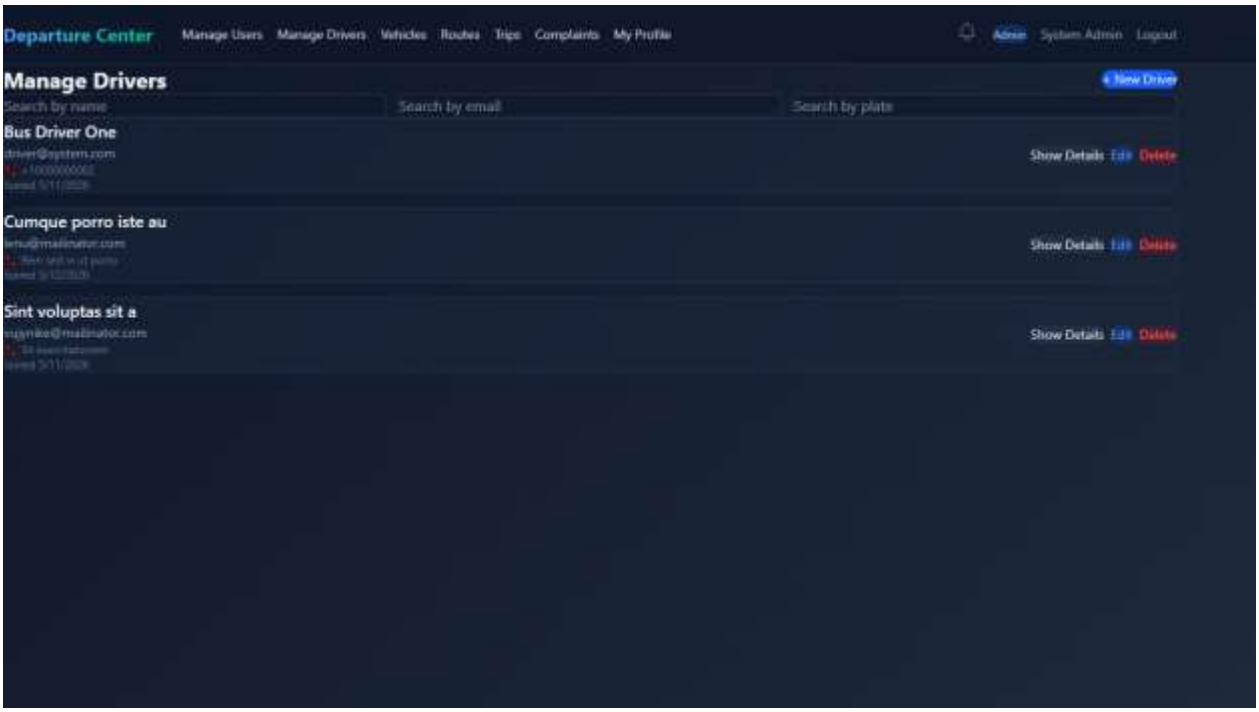


Figure 51 interface mange drivers

8.Admin Add Driver

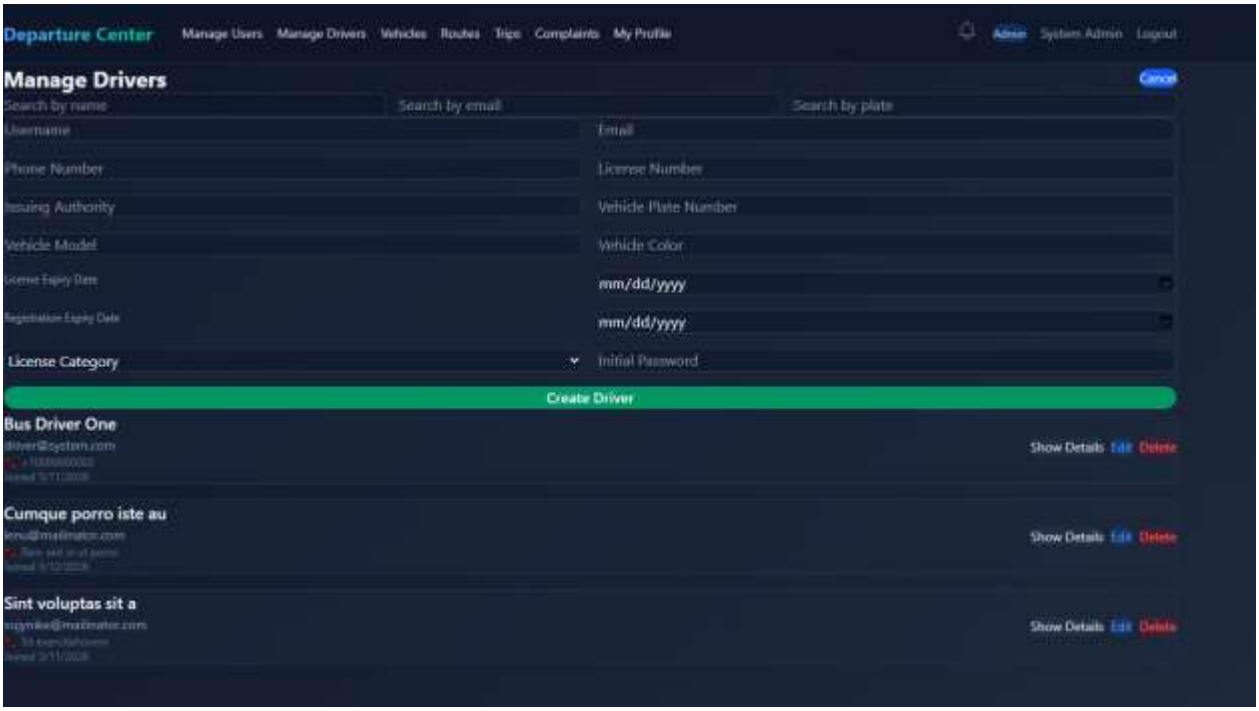


Figure 52 interface add driver

9.Admin update Driver

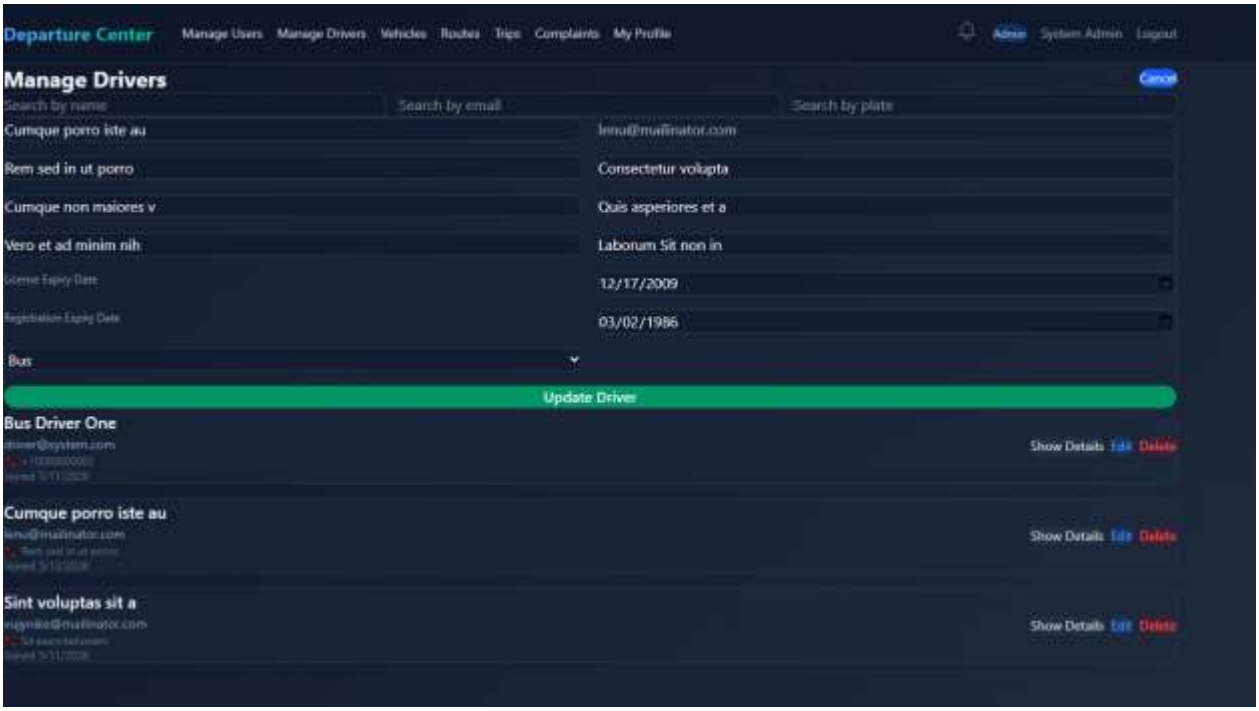


Figure 53 interface update driver

10.Admin show Driver details

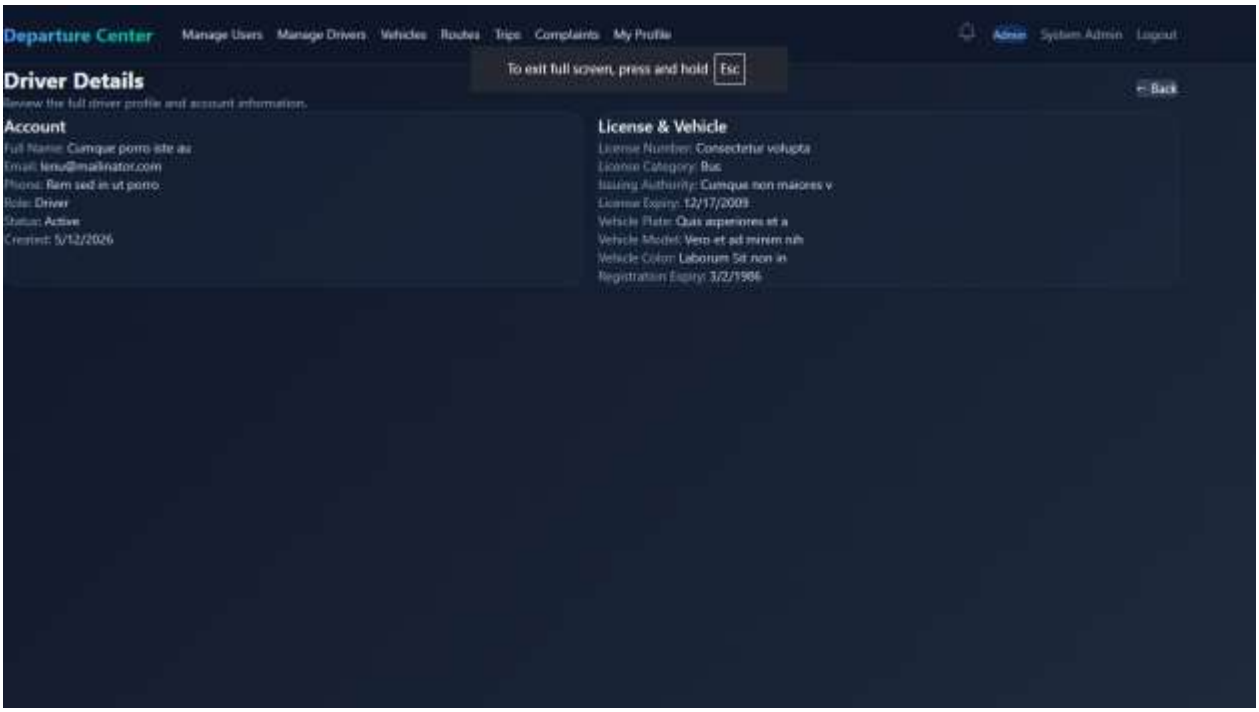


Figure 54 interface show driver derails

11.Admin Manage Vehicles

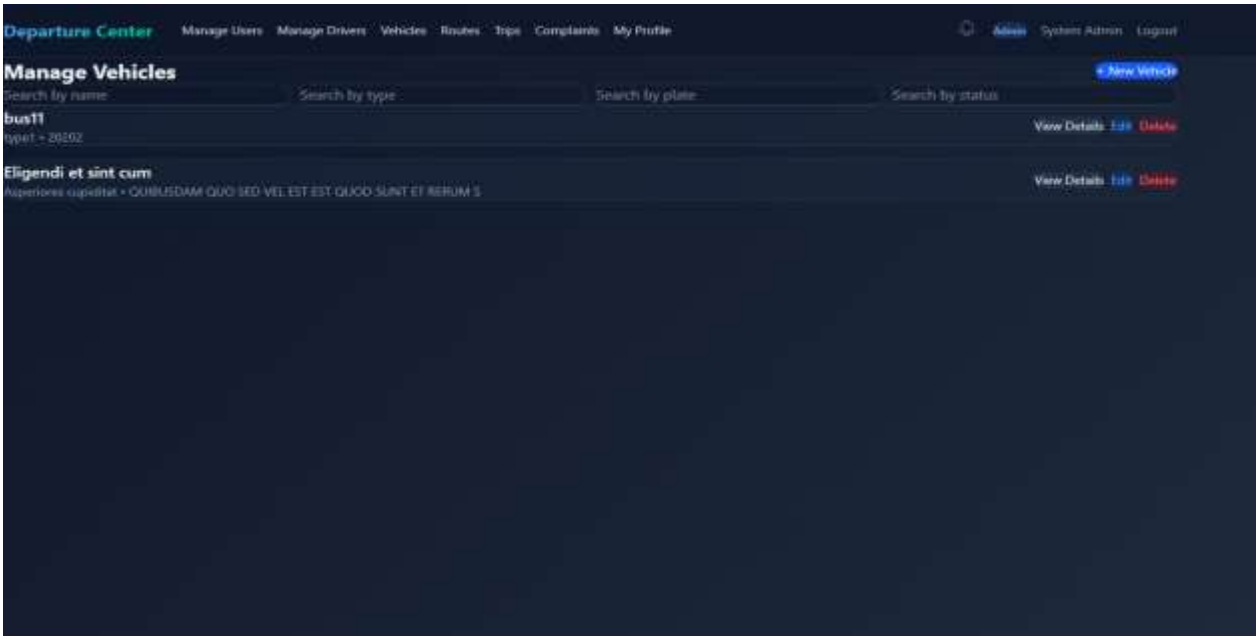


Figure 55 interface mange vehicles

12.Admin Add Vehicle

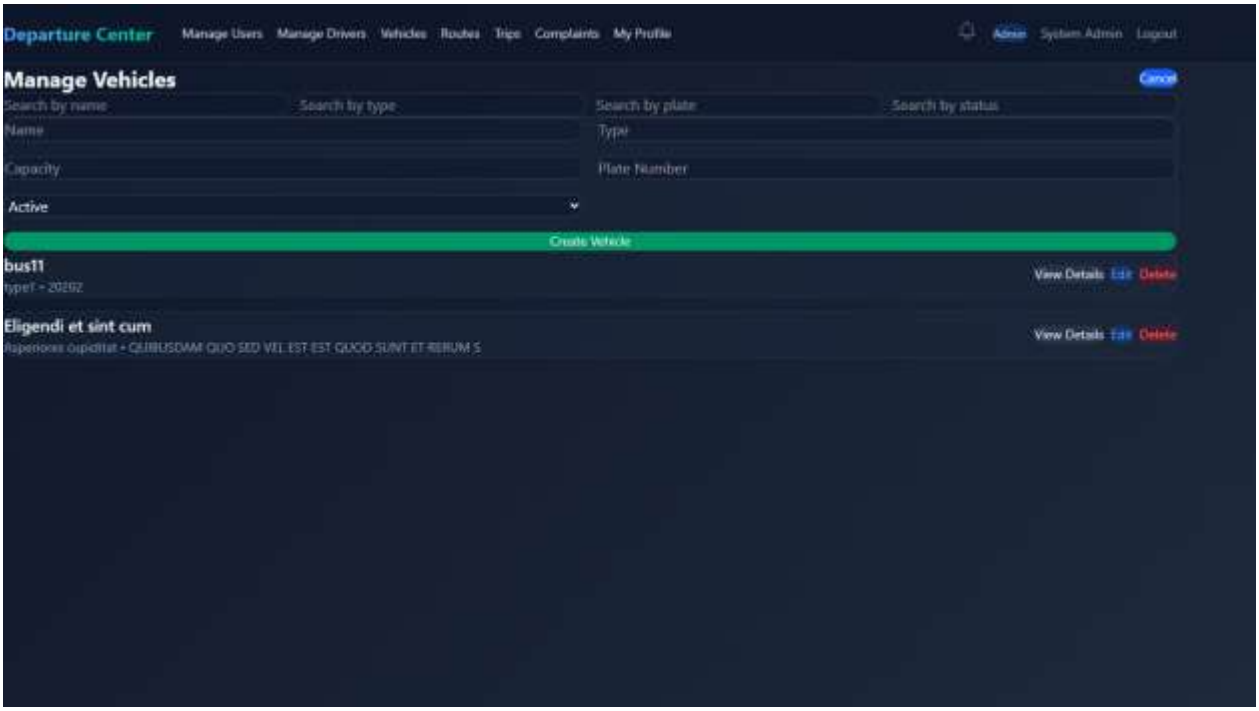


Figure 56 interface add vehicle

13.Admin update Vehicle

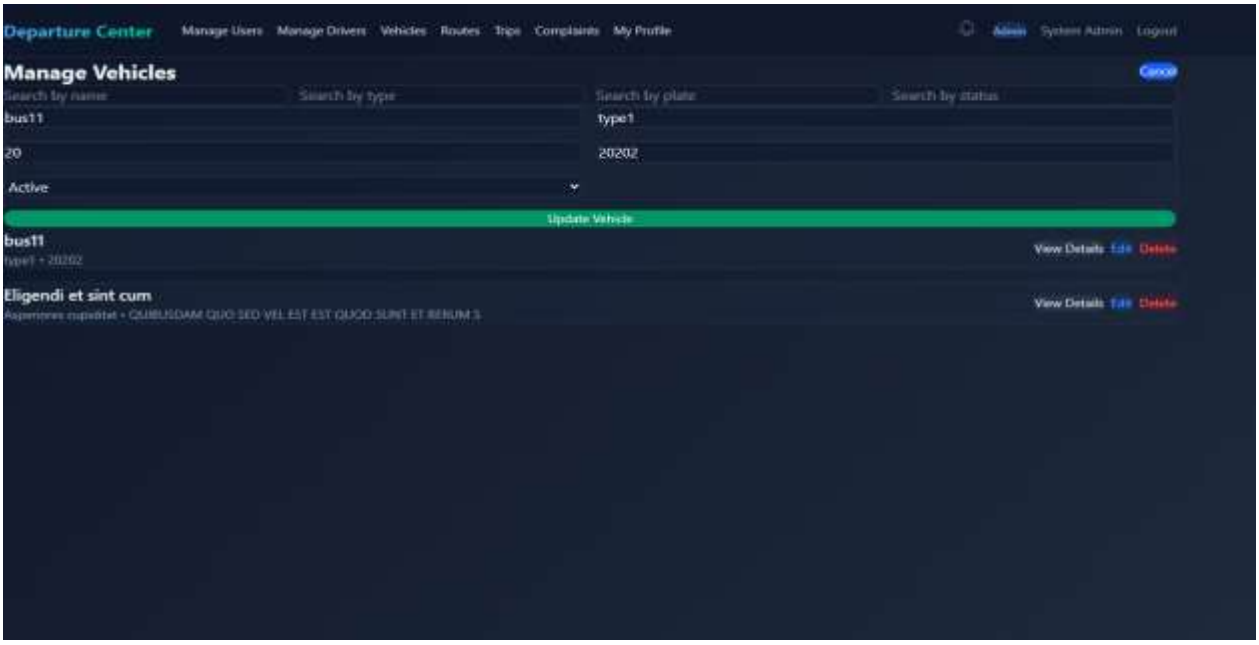


Figure 57 interface update vehicle

14.Admin show Vehicle Details

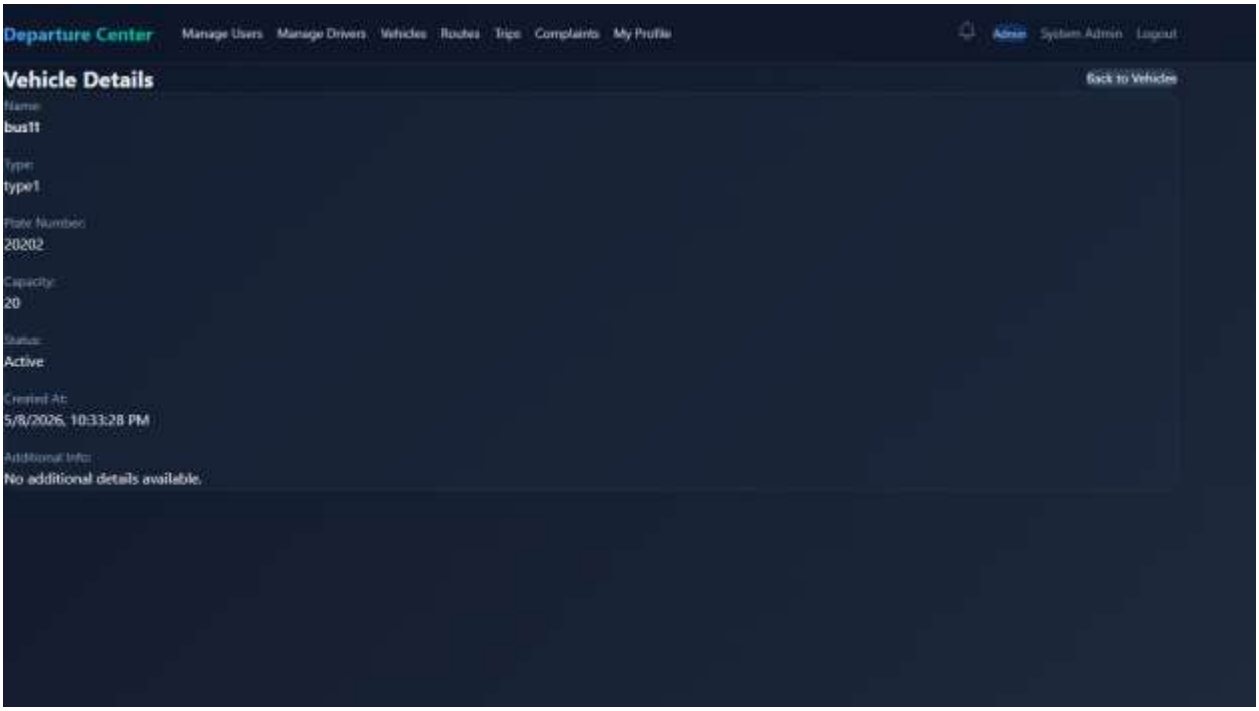


Figure 58 intreface show vehicle detalils

15.Admin Logout (same all users)

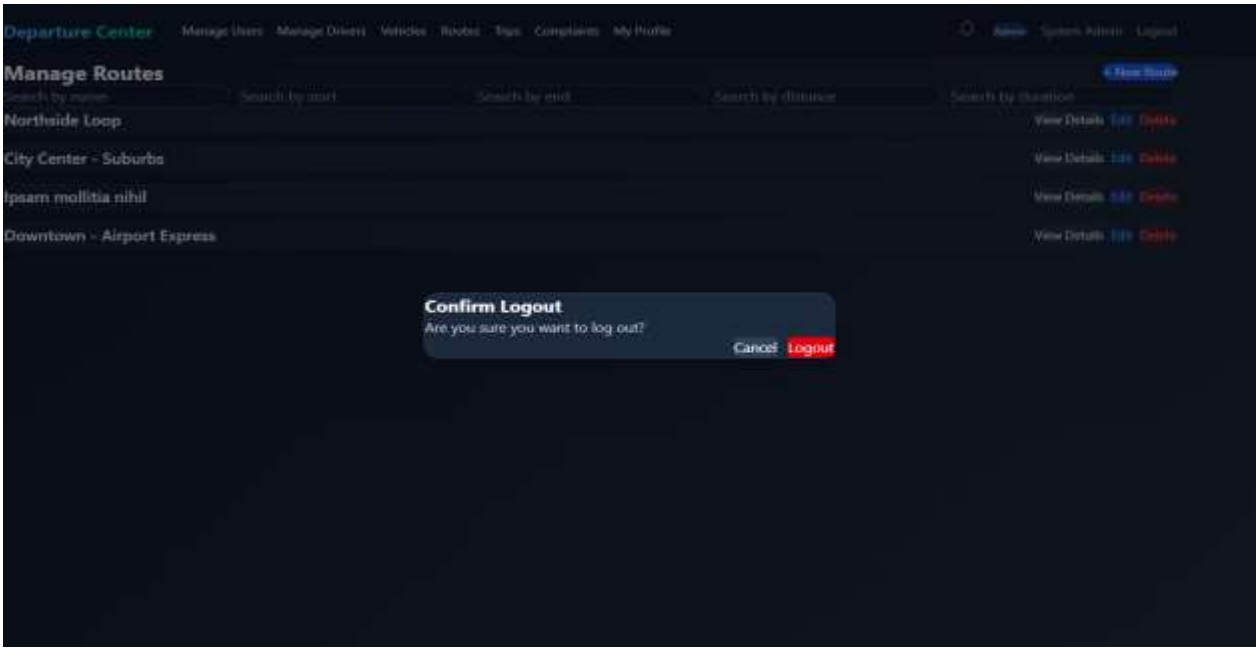


Figure 59 interface logout

16.Admin Manage Routes

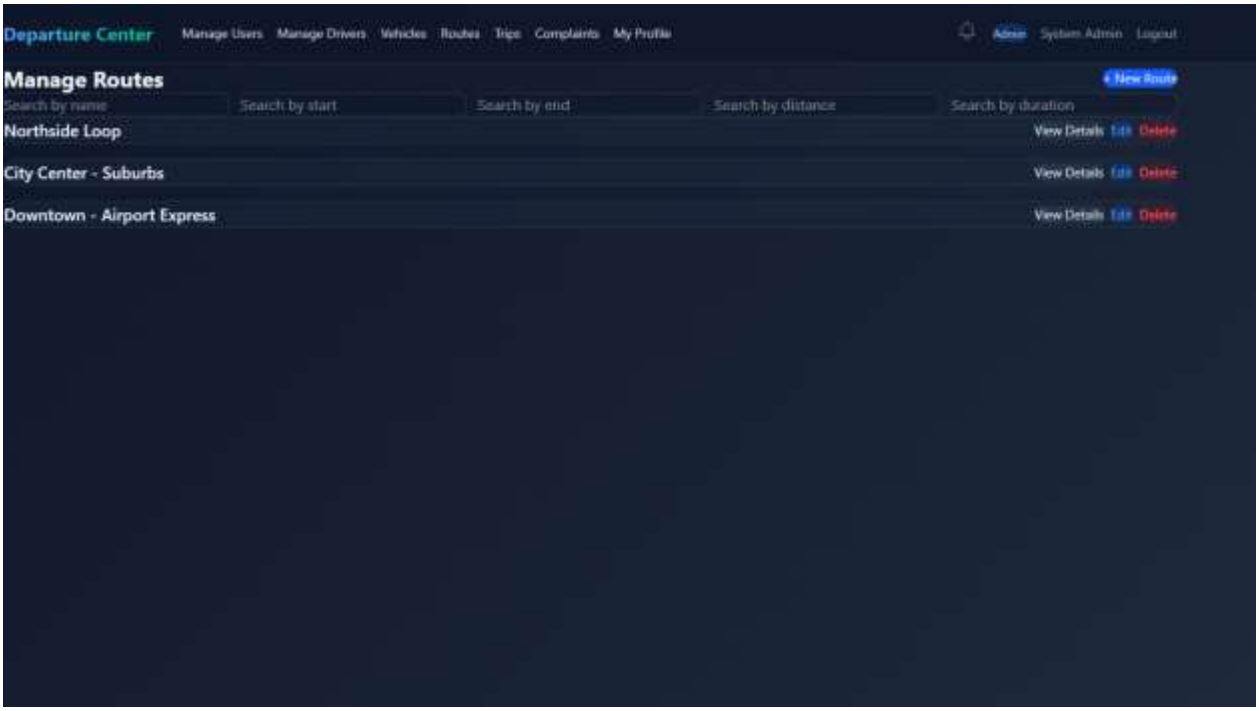


Figure 60 interface manage routes

17.Admin Add Route

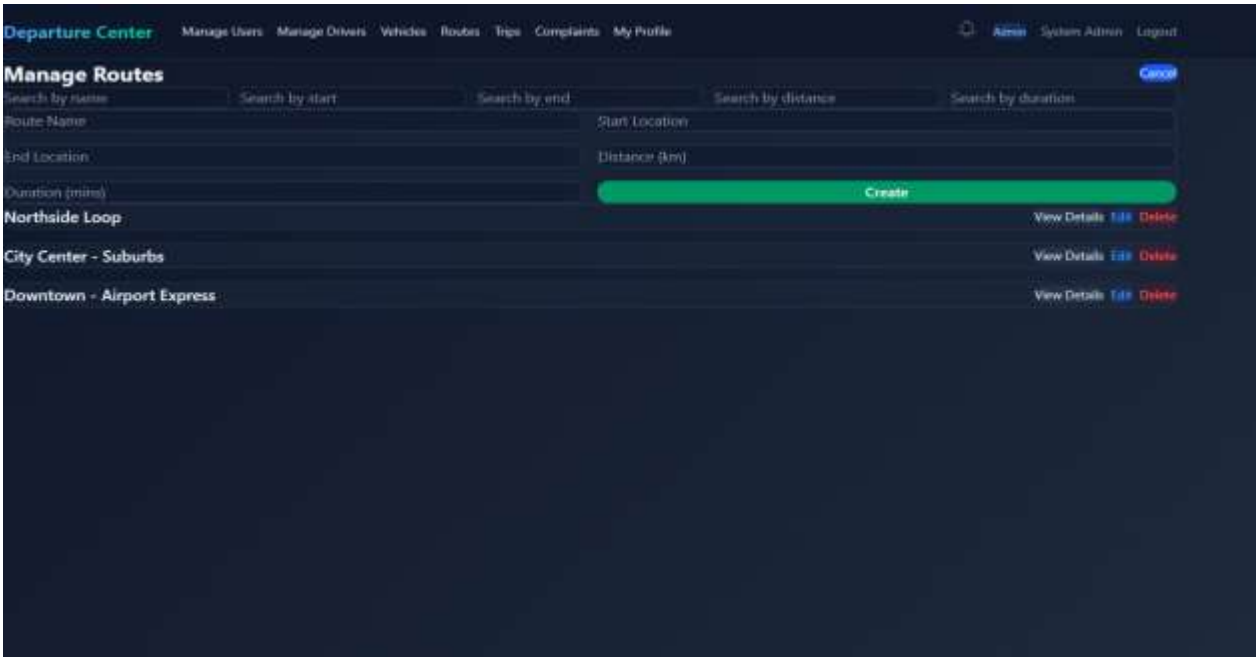


Figure 61 interface admin add rouit

18.Admin update Routes

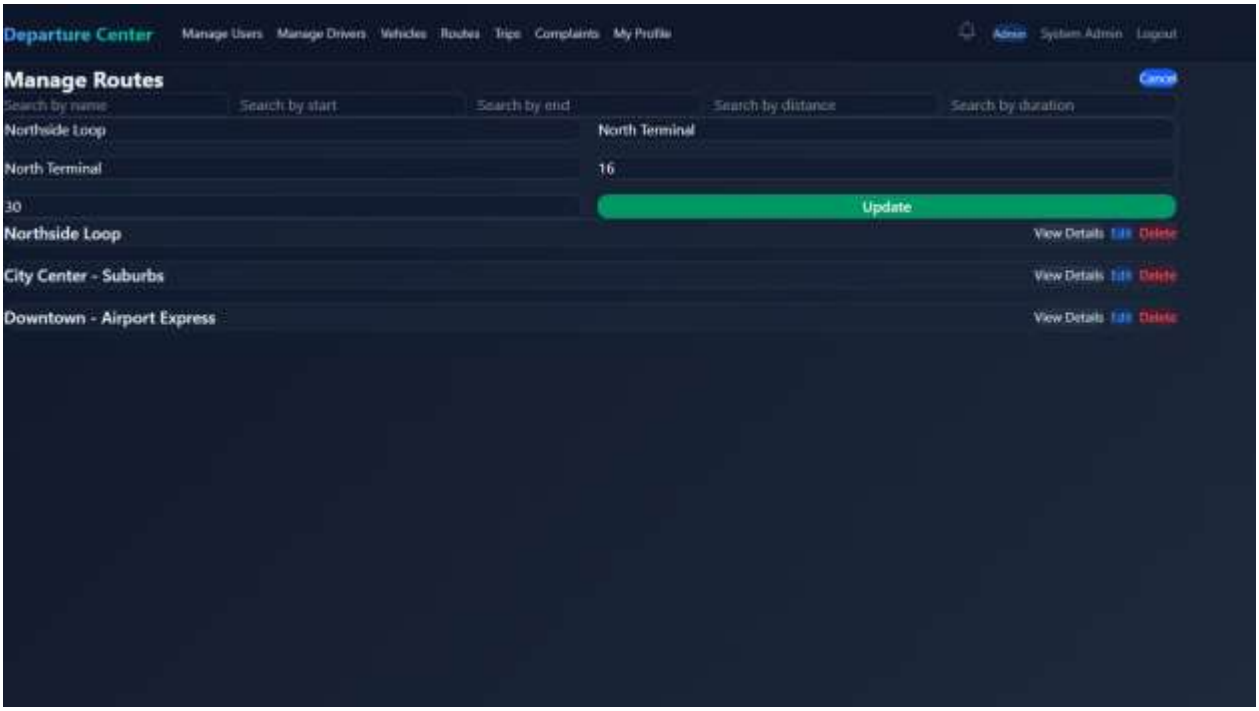


Figure 62 interface update routes

19.Admin show Route Details

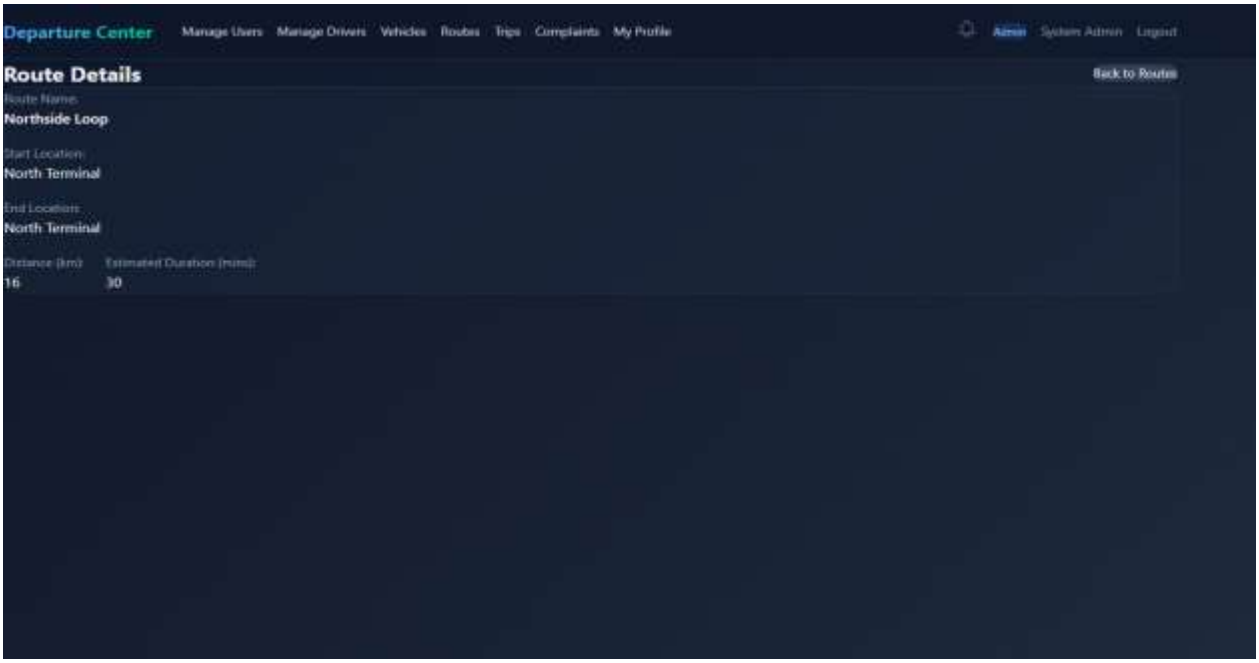


Figure 63 interface show route details

20.Admin Manage Trips

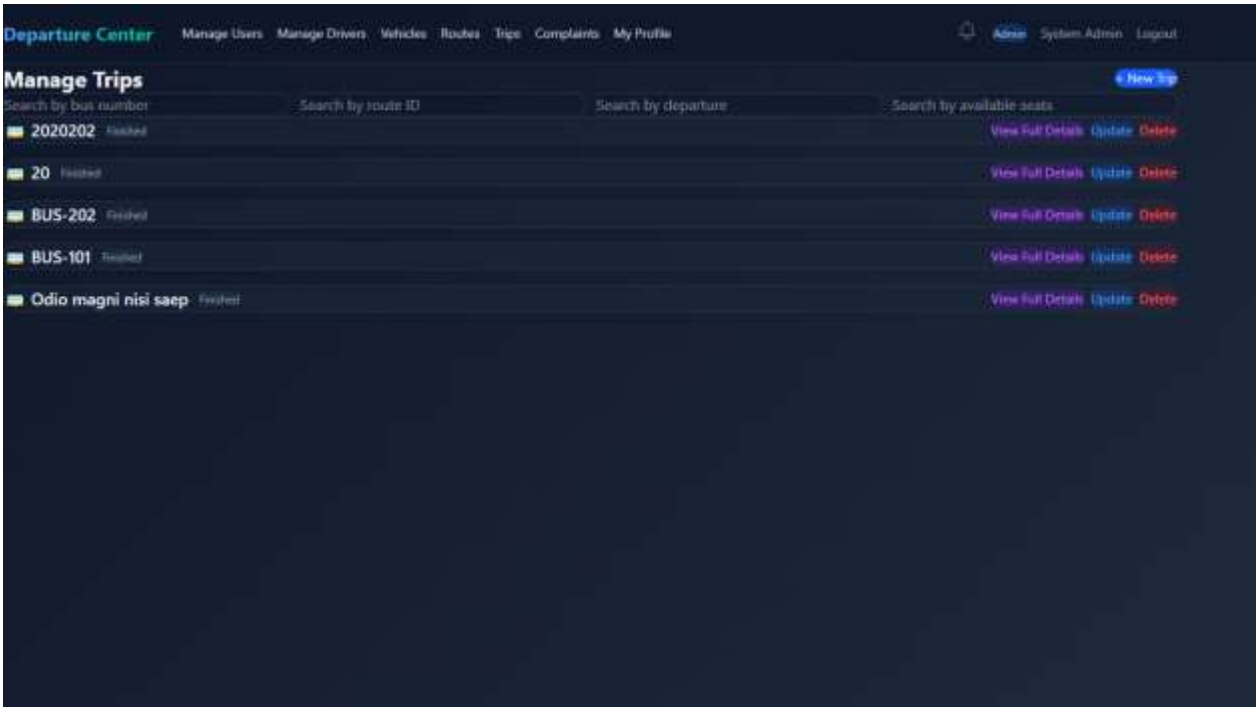


Figure 64 interface manage trips

21.Admin Add Trip

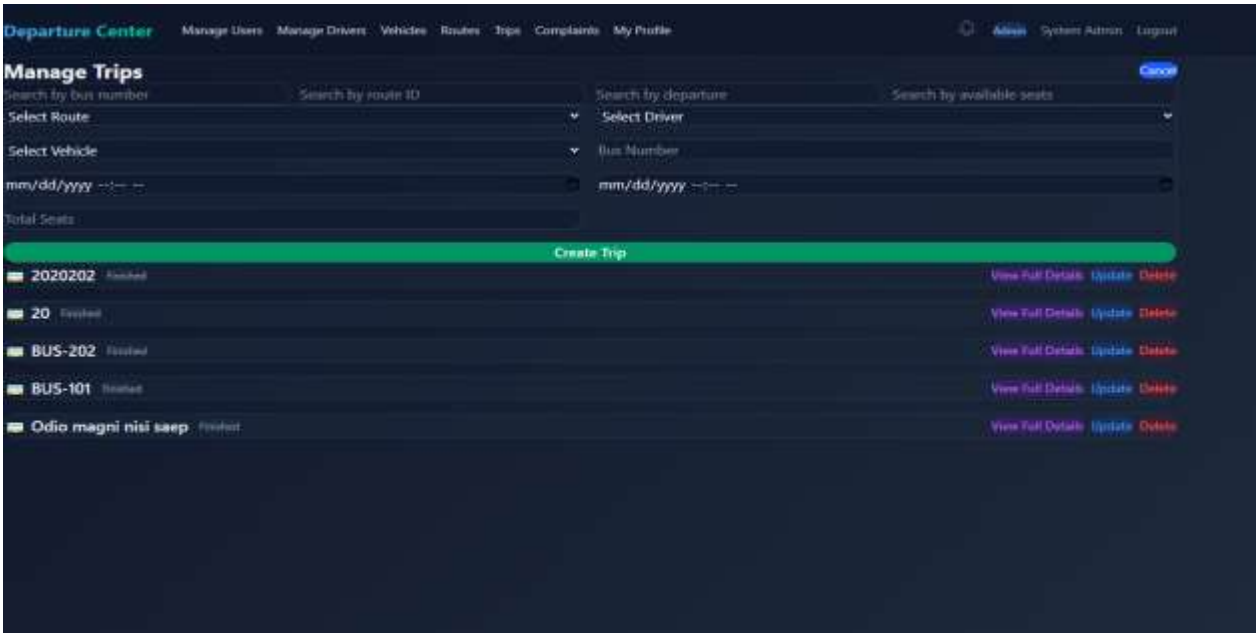


Figure 65 interface add trip

22.Admin Update Trip

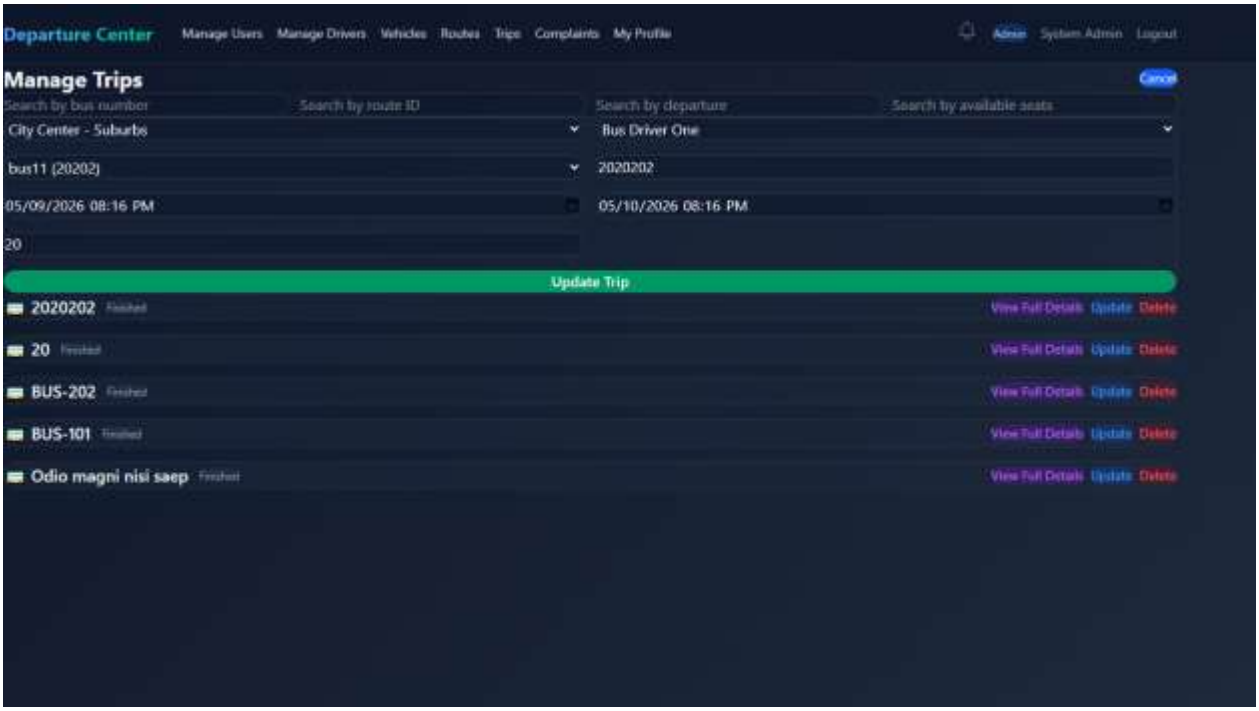


Figure 66 interface add trip

23.Admin show Trip Details

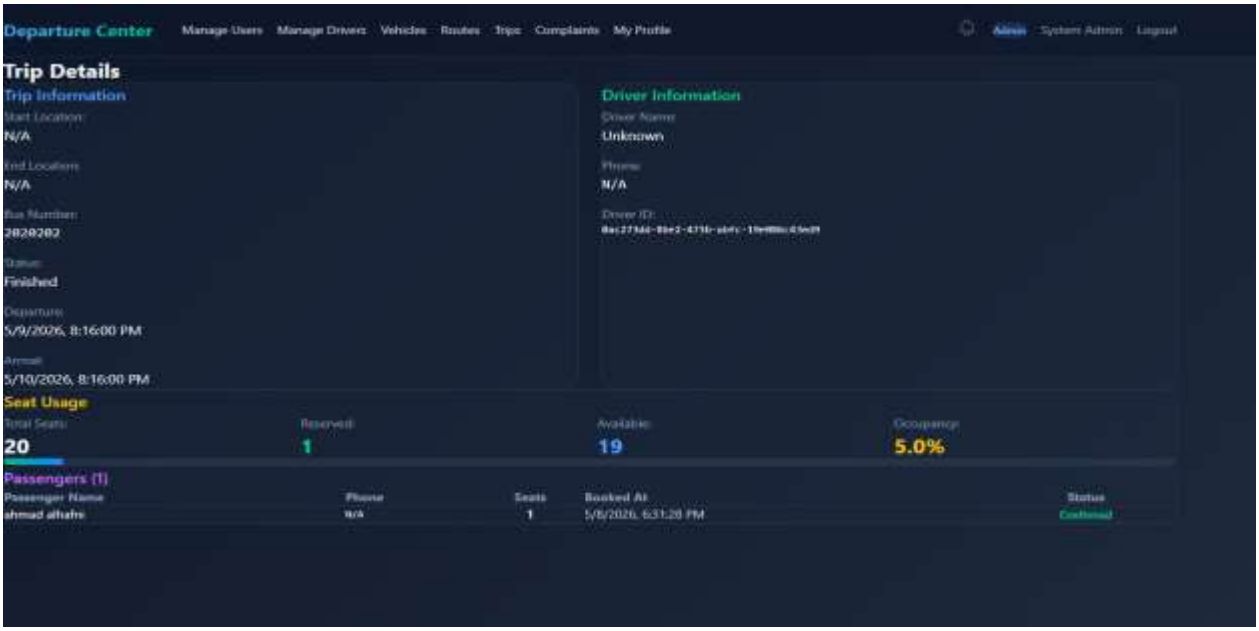


Figure 67 interface show trip details

24.Admin Manage Complaints

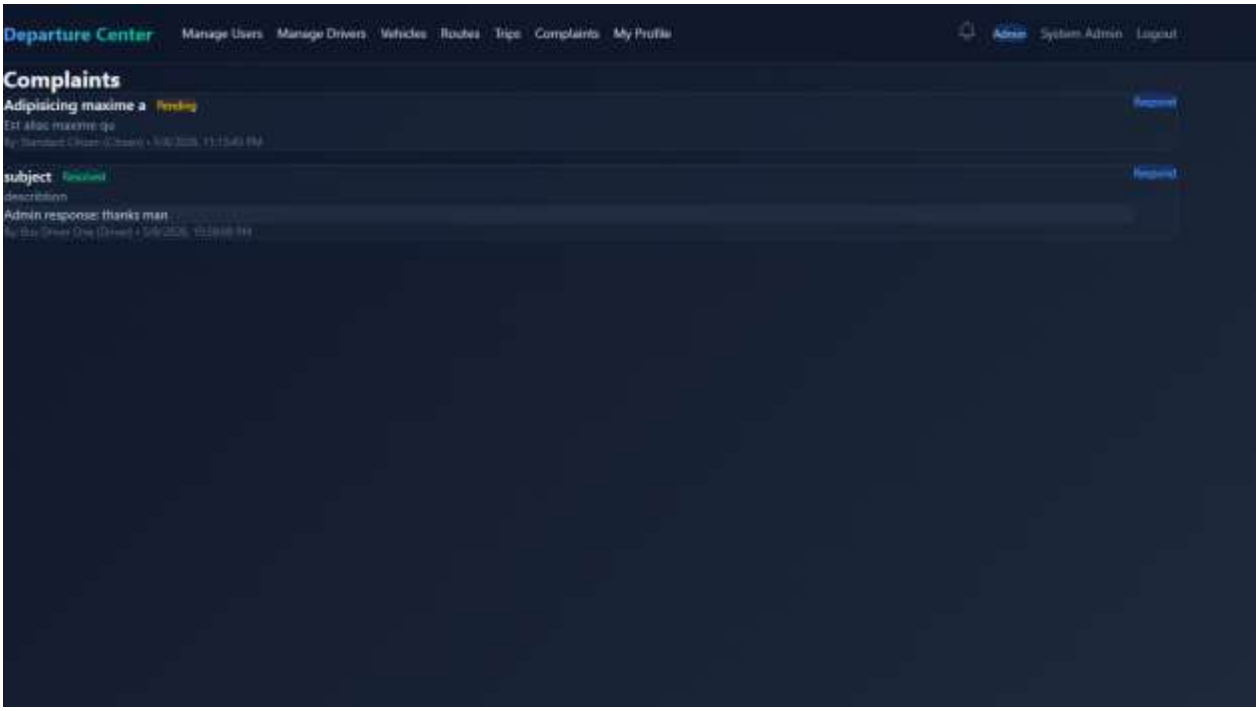


Figure 68 interface mange complaints

25.Admin Response to a Complaint

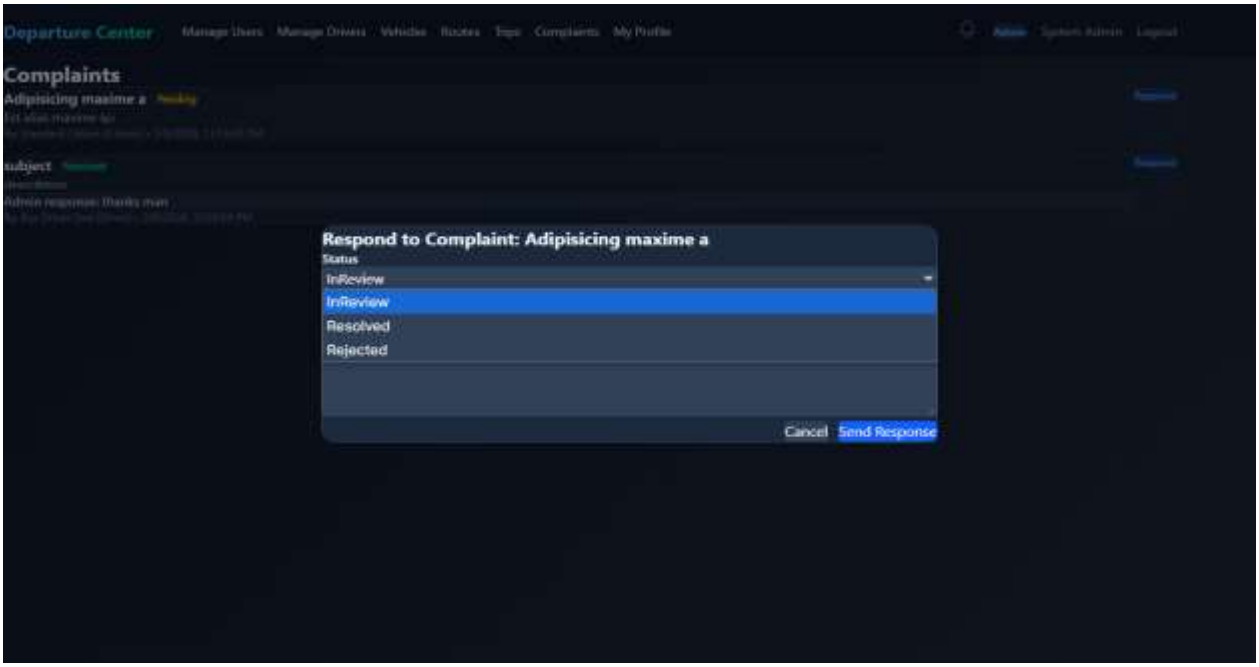


Figure 69 interface respose to a complaint

26.Admin Manage Profile

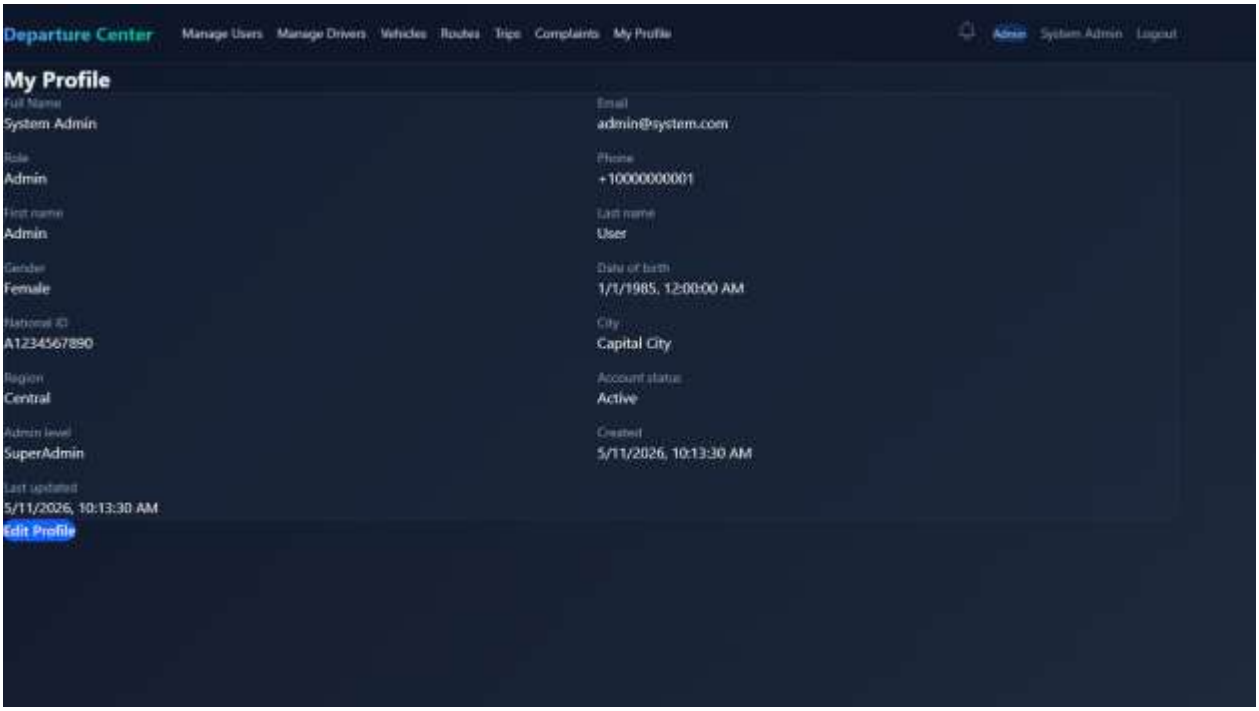


Figure 70 interface manage profile

27.Admin edit Profile info

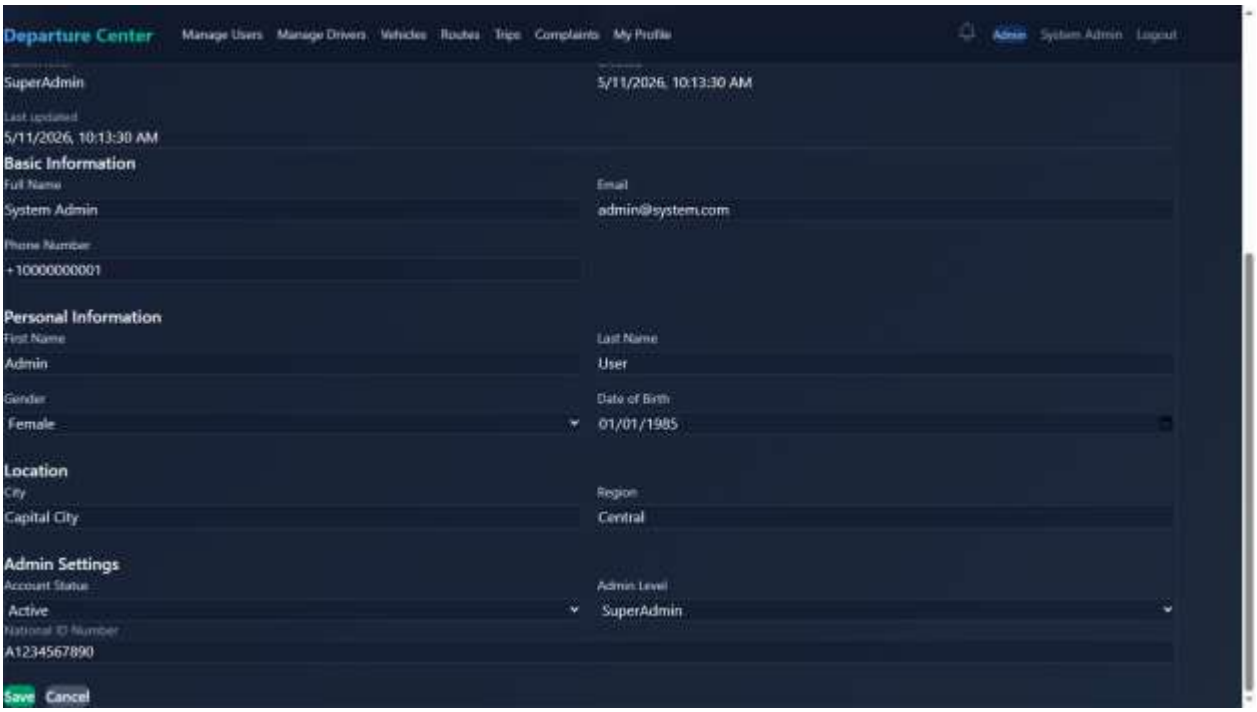


Figure 71 interface edit profile info

28.Driver Manage Trips

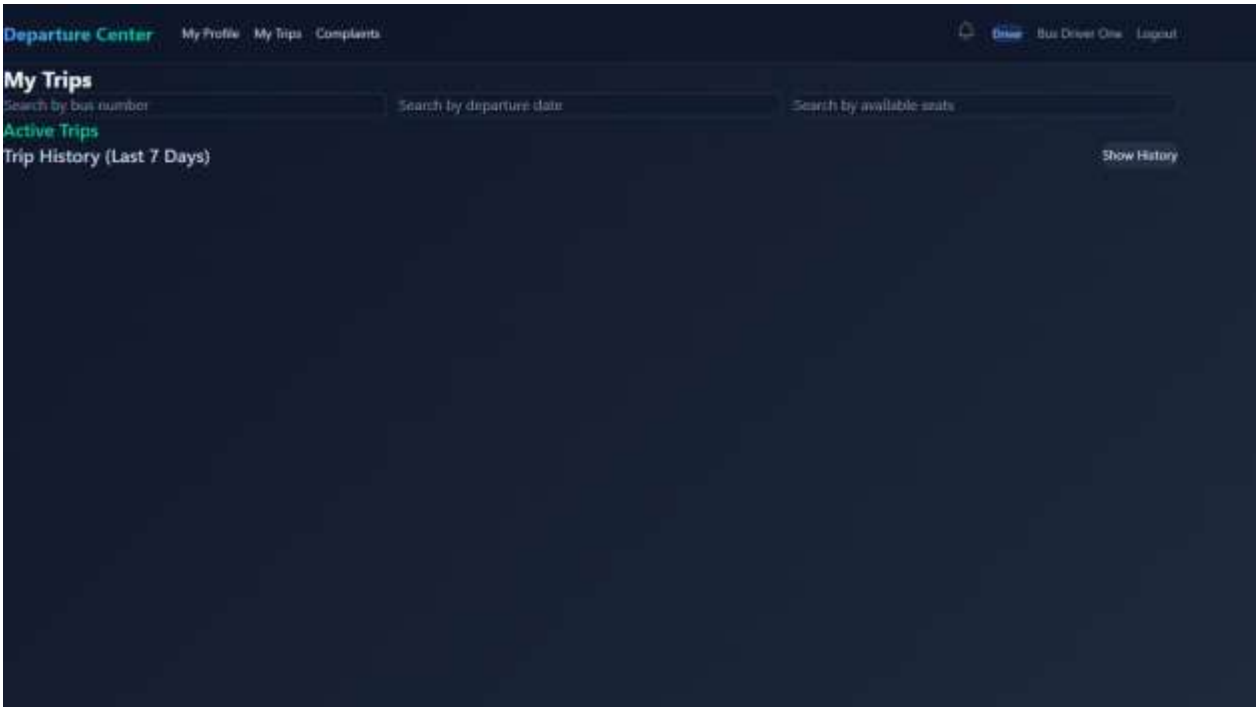


Figure 72 interface driver manage trips

29.Driver Manage complaints

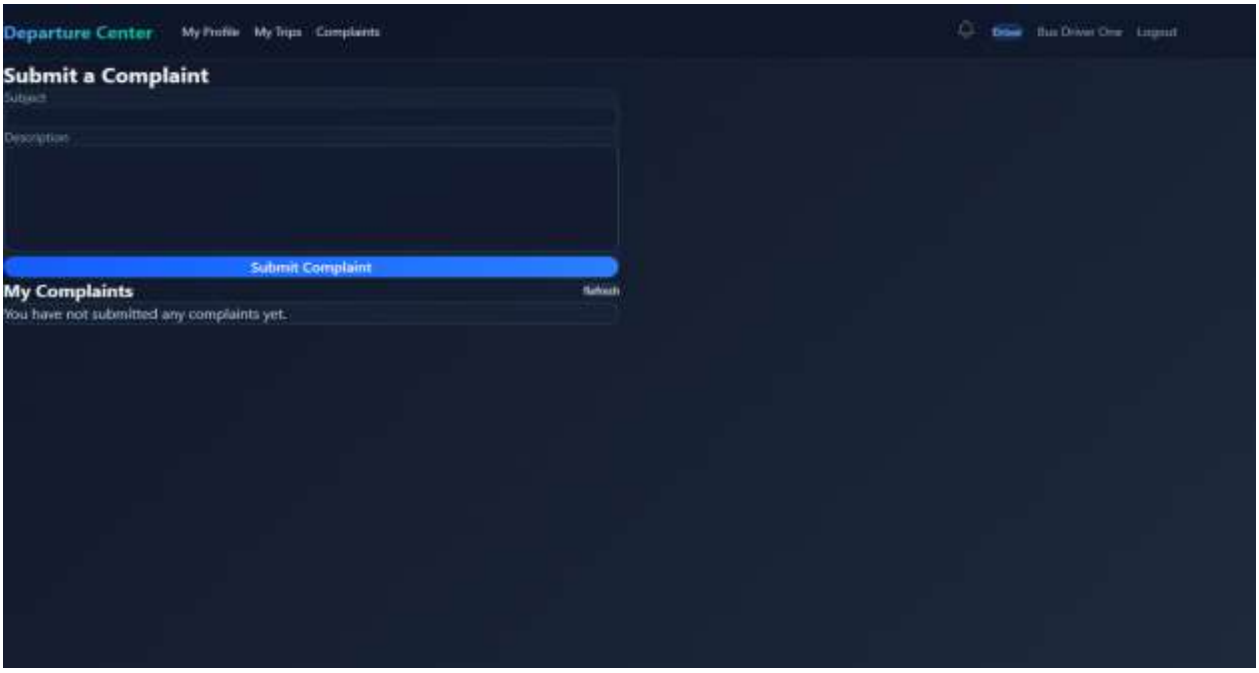


Figure 73 interface driver manage complaints

30.Citizen Manage Profile

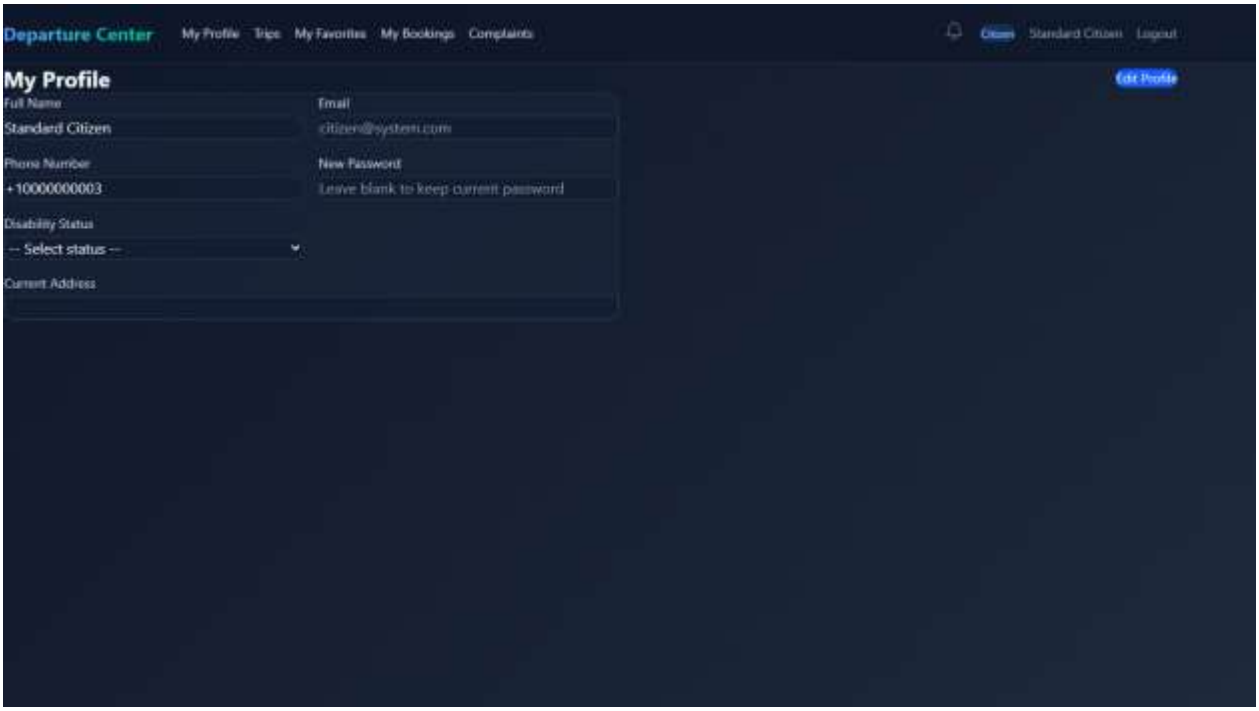


Figure 74 interface citizen manage profile

31.Citizen Edit Profile info

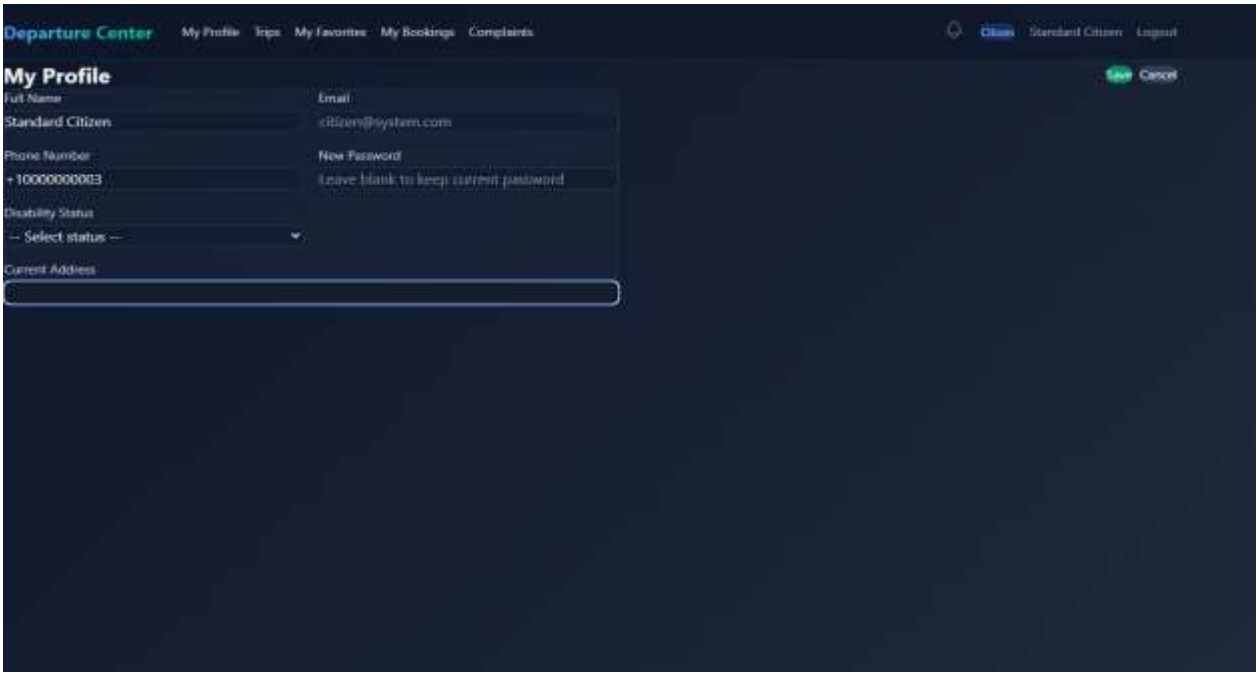


Figure 75 interface citizen edit progile info

32.Citizen Available Rotes Page

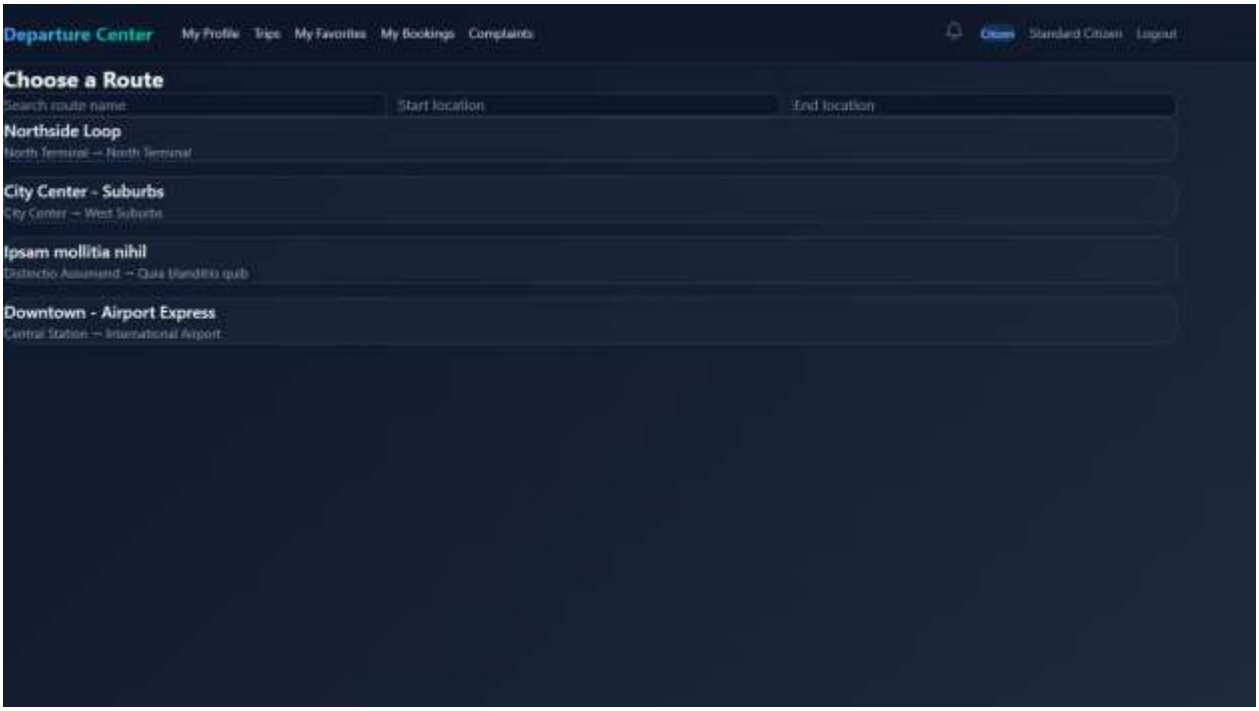


Figure 76 interface citizen avalabel rotes pasge

32.Citizen Trips inside a Route page and booking page

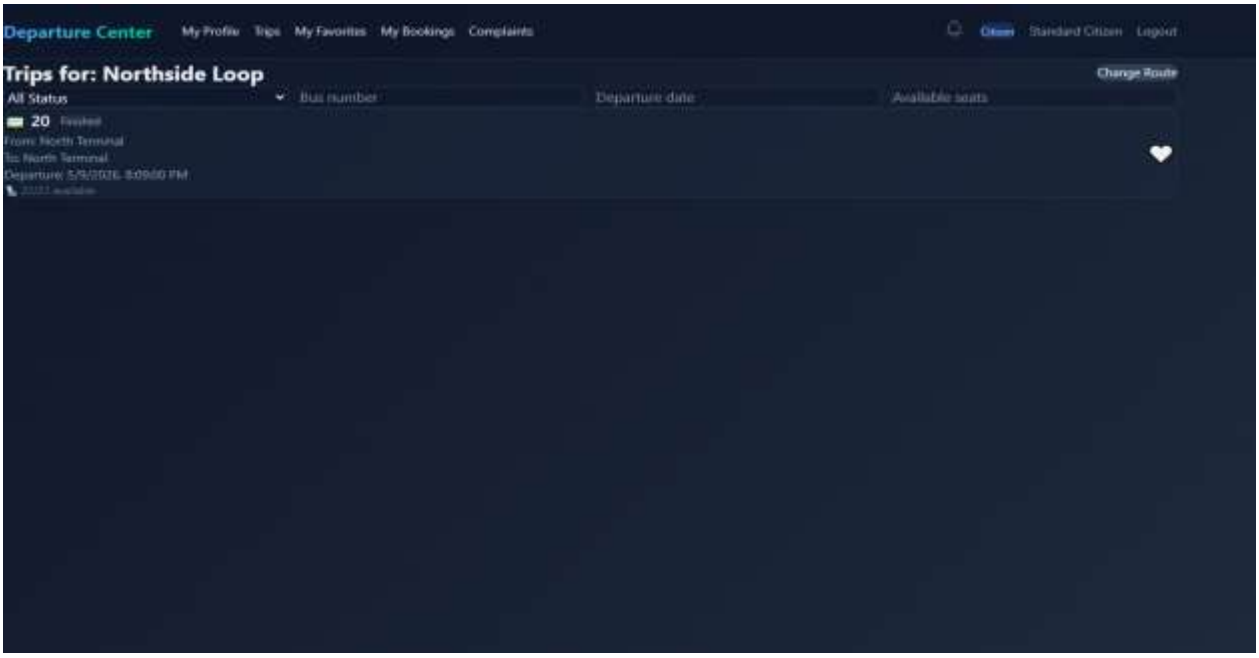


Figure 77 interface Citizen Trips inside a Route page and booking page

33.Citizen My Favorite Page

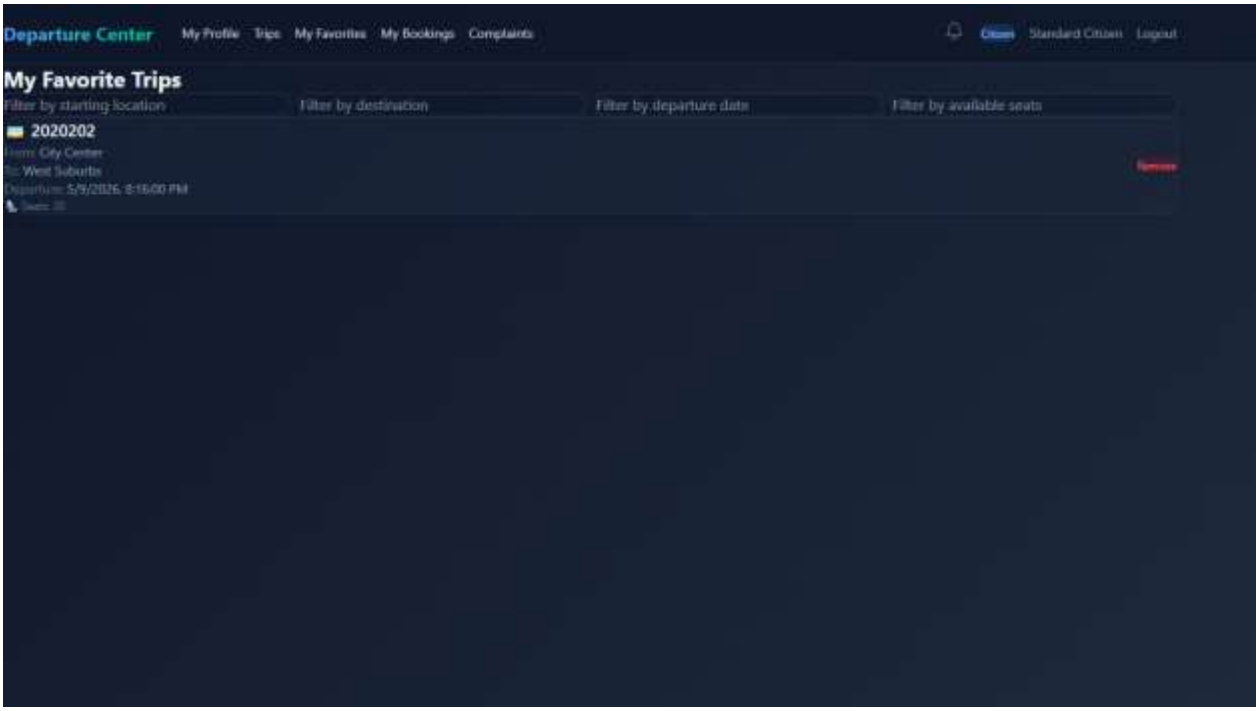


Figure 78 interface my favorite trips

34.Citizen My Bookings Page

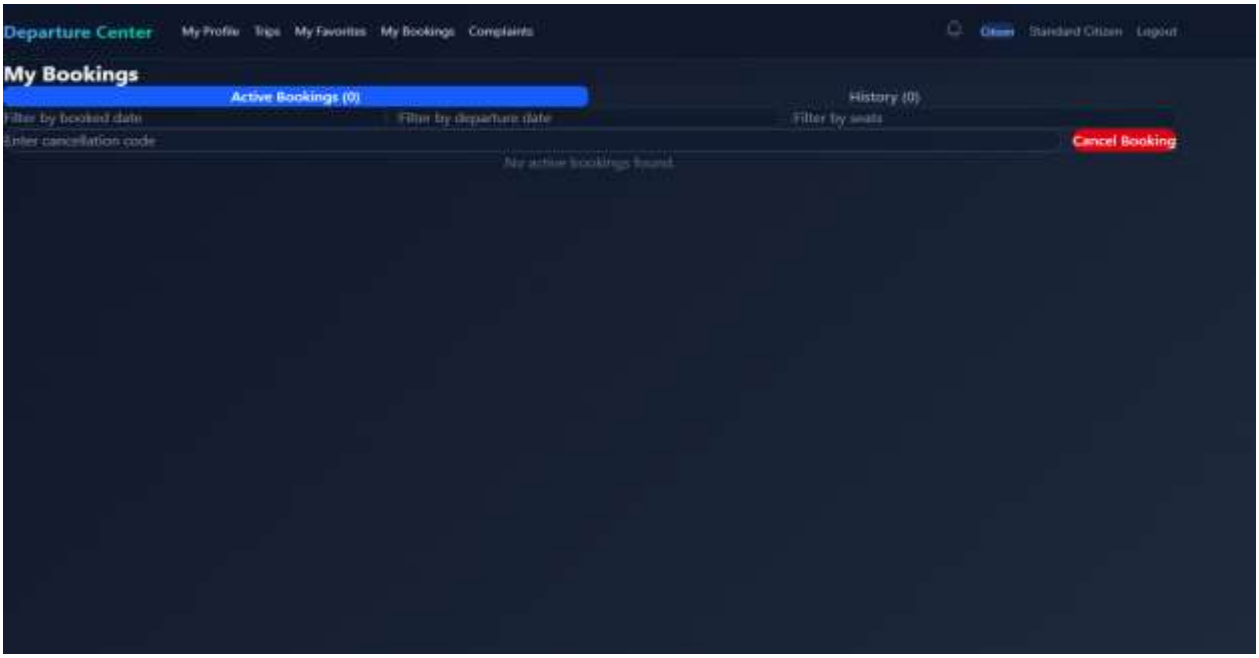


Figure 79 interface my booking page

35.Citizen My History Bookings Page

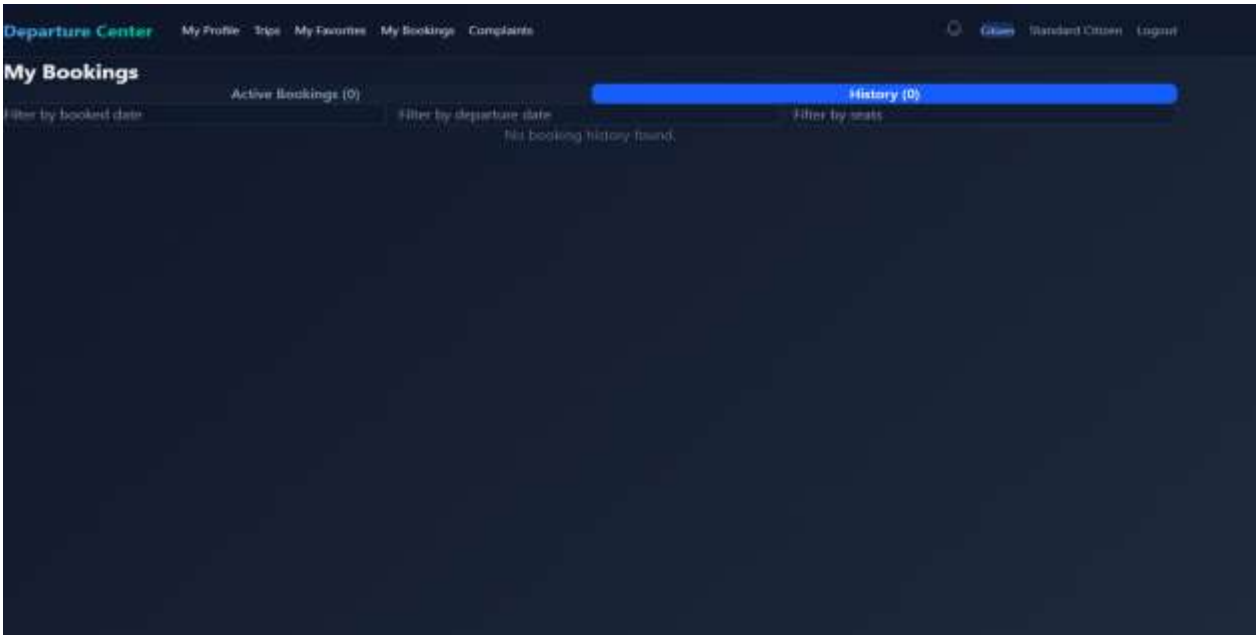


Figure 80 interface my history

36.Citizen Manage Complaints Page

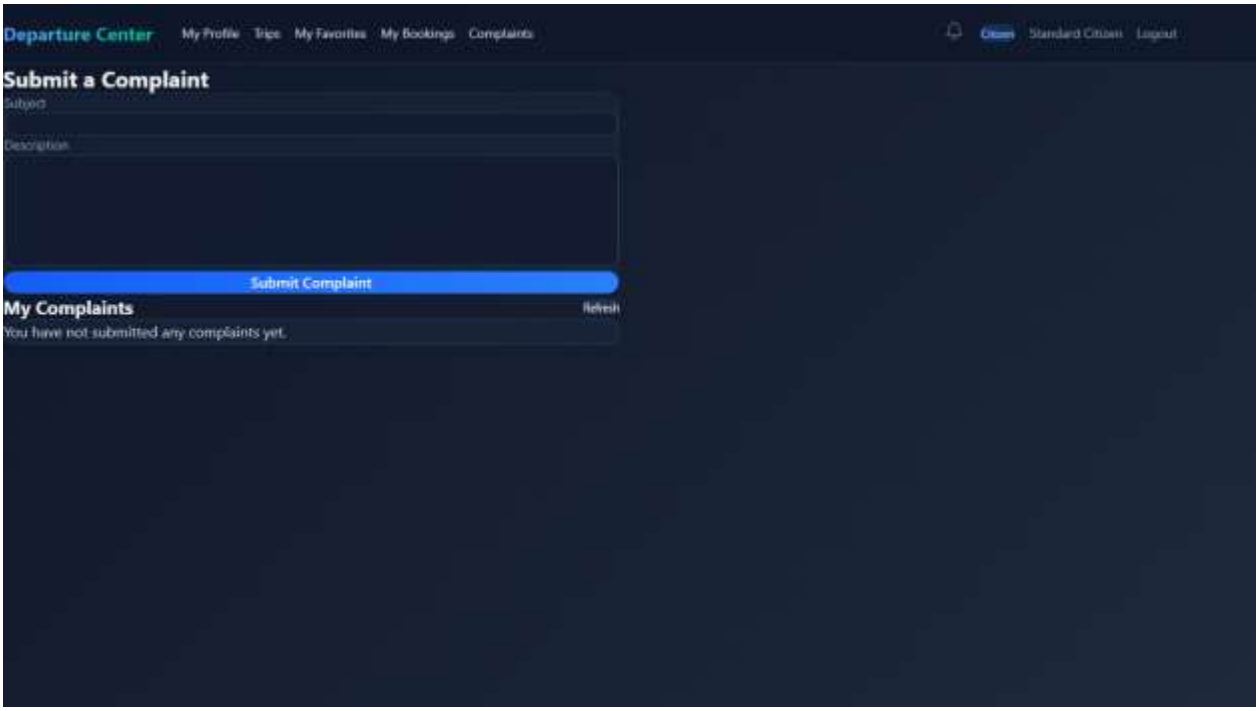


Figure 81 interface citizen complaints

7.7 Performance Testing Implementation

The system performance was tested using k6 load testing scripts.

Testing Objectives

- Measure API response time
- Evaluate concurrent user handling
- Detect bottlenecks
- Test system stability under load

k6 Test Example

```
import http from 'k6/http';
import { sleep } from 'k6';
export default function () {
    http.get('http://localhost:5000/api/trips');
    sleep(1);
}
```

Testing Scenarios

- Concurrent booking requests
- Authentication stress testing
- Trip retrieval performance
- Gateway routing performance

7.8 Summary

The DepartureCenterSystem implementation follows a scalable microservices architecture combined with Clean Architecture principles. The system was implemented using ASP.NET Core, React, SQL Server, Docker, and Entity Framework Core. Each service was independently structured and deployed, enabling better maintainability, scalability, and extensibility. The use of JWT authentication, API Gateway routing, repository patterns, dependency injection, and load testing tools contributed to building a robust and production-oriented software system.

Chapter 8

Testing and Evaluation

8.1 Testing Plan

The testing phase of the DepartureCenterSystem project was designed to ensure that the system functions correctly, performs efficiently under different workloads, and maintains acceptable security and reliability standards. The testing process focused primarily on API validation, performance testing, load testing, and security assessment.

The testing activities were performed throughout the development lifecycle to continuously validate system stability and functionality.

Testing Objectives

The main objectives of the testing process were:

- Verify that all APIs function correctly
- Ensure stable communication between microservices
- Validate authentication and authorization mechanisms
- Measure system performance under different workloads
- Detect bottlenecks and performance limitations
- Evaluate system behavior during stress conditions
- Assess the security posture of exposed APIs

Testing Tools Used

Tool	Purpose
Postman	Manual API testing and endpoint validation
k6	Load and performance testing
Docker Compose	Test environment orchestration

8.2 Types of Tests

The project included multiple categories of testing to evaluate different aspects of the system.

8.2.1 Functional Testing

Functional testing was performed manually to validate API behavior and business workflows.

Functional Testing Scope

- User registration
- User authentication
- JWT token validation
- Trip retrieval
- Complaint submission
- Authorization and role validation

8.2.1.1 Add Driver Test Case Table

Test Case ID	Use Case ID	Test Case Name	Preconditions	Test Steps	Test Data	Expected Result	Status
TC-01	UC-01	Add Driver	User is logged in, dashboard is loaded, and user is on Manage Drivers page	1. Click "Manage Drivers" 2. Click "New Driver" 3. Enter valid driver information 4. Click "Create Driver"	Valid Username, Email, Phone Number, License Number, Vehicle Details, Expiry Dates, License Category, Password	System validates data, saves the new driver, and displays message: "Driver created successfully"	Pass
TC-02	UC-01	Add Driver with Duplicate Email	User is logged in, dashboard is loaded, and user is on Manage Drivers page	1. Click "Manage Drivers" 2. Click "New Driver" 3. Enter driver information using an existing email 4. Click "Create Driver"	Email: existing@test.com	System displays error message: "Email already in use" and the driver is not created	Pass

8.2.1.2 Update Driver Test Case Table

Test Case ID	Use Case ID	Test Case Name	Preconditions	Test Steps	Test Data	Expected Result	Status
TC-01	UC-02	Update Driver	User is logged in, on Manage Drivers page, and driver already exists	1. Click "Edit" on a driver 2. System displays edit form 3. Modify valid driver information 4. Click "Update Driver" 5. Confirm update	Updated Username, Phone Number, License Details, Vehicle Details, Expiry Dates, License Category	System saves changes and displays message: "Driver updated successfully"	Pass
TC-02	UC-02	Update Driver with Invalid Data	User is logged in, on Manage Drivers page, and driver already exists	1. Click Edit on a driver 2.Modifydriver information with invalid data 3.ClickUpdate Driver	Invalid Phone Number or invalid expiry date	System displays validation error message and does not save changes	Pass

8.2.1.3 Add Vehicle Test Case Table

Test Case ID	Use Case ID	Test Case Name	Preconditions	Test Steps	Test Data	Expected Result	Status
TC-01	UC-05	Add Vehicle	User is logged in, dashboard is loaded, and user is on Manage Vehicles page	1.Navigate to Manage Vehicles 2. Click "New Vehicle" 3. Enter valid vehicle information 4.Click "Create Vehicle"	Name: Truck A, Type: Heavy, Capacity: 5000kg, Plate Number: ABC123, Status: Active	System validates data, saves the new vehicle, and displays message: "Vehicle created successfully"	Pass

TC-02	UC-05	Add Vehicle with Duplicate Plate Number	User is logged in, dashboard is loaded, and user is on Manage Vehicles page	1.Navigate to "Manage Vehicles 2. Click "New Vehicle" 3. Enter vehicle information using existing plate number 4. Click "Create Vehicle"	Plate Number: Existing123	System displays error message for duplicate plate number and vehicle is not created	Pass

8.2.1.4 Delete Vehcile Test Case Table

Test Case ID	Use Case ID	Test Case Name	Preconditions	Test Steps	Test Data	Expected Result	Status
TC-01	UC-07	Delete Vehicle	User is logged in, on Manage Vehicles page, and vehicle exists	1. Click "Delete" on a vehicle 2. System shows confirmation message 3. Click "Delete" to confirm	Existing Vehicle Record	System removes the vehicle from database and displays message: "Vehicle deleted successfully"	Pass

TC-02	UC-07	Cancel Vehicle Deletion	User is logged in, on Manage Vehicles page, and vehicle exists	1. Click "Delete" on a vehicle 2. System shows confirmation message 3. Click "Cancel"	Existing Vehicle Record	System keeps the vehicle unchanged and no deletion occurs	Pass

8.2.1.5 Submit Complaint Test Case Table

Test Case ID	Use Case ID	Test Case Name	Preconditions	Test Steps	Test Data	Expected Result	Status
TC-01	UC-13	Submit Complaint	User is logged in and dashboard is loaded	1. Click "Complaints" from dashboard 2. System displays complaint form 3. Enter valid subject and description 4. Click "Submit Complaint"	Subject: Late Bus, Description: The bus arrived 30 minutes late	System validates input, saves complaint with status "Pending", displays success message, and shows complaint under "My Complaints"	Pass

TC-02	UC-13	Submit Complaint with Missing Required Fields	User is logged in and dashboard is loaded	1. Click "Complaints" from dashboard 2. Leave subject or description empty 3. Click "Submit Complaint"	Missing Subject or Description	System displays validation error message and complaint is not submitted	Pass

8.2.1.6 Reply to Complaint Test Case Table

Test Case ID	Use Case ID	Test Case Name	Preconditions	Test Steps	Test Data	Expected Result	Status
TC-01	UC-16	Reply to Complaint	Garage Manager is logged in, complaints list is displayed, and complaint exists	1. Select a complaint 2. Click "Response" 3. System displays response form 4. Enter valid status and admin response 5. Click "Send Response"	Status: Resolved, Admin Response: Issue has been fixed, Description: Bus schedule adjusted	System validates input, saves response, updates complaint status, displays confirmation message, and response	Pass

						becomes visible in user's "My Complaints"	
TC-02	UC-16	Reply to Complaint with Missing Input	Garage Manager is logged in, complaints list is displayed, and complaint exists	1. Select a complaint 2. Click "Response" 3. Leave required fields empty 4. Click "Send Response"	Missing Admin Response or Status	System displays validation error message and no response is saved	Pass

8.2.1.7 Update Profile Test Case Table

Test Case ID	**Use Case ID**	**Test Case Name**	**Preconditions**	**Test Steps**	**Test Data**	**Expected Result**	**Status**
TC-01	UC-20	Update Profile	User is logged in and Profile page is displayed	1. Click "Edit Profile" 2.Modify allowed fields based on role 3. Click "Save"	Ex:Full Name: Ahmad Alhafni, Phone Number: 0999999999, Address: Damascus	System validates input, updates profile data, and displays message: "Profile updated successfully"	Pass

TC-02	UC-20	Update Profile with Invalid Input	User is logged in and Profile page is displayed	1. Click "Edit Profile" 2. Enter invalid data in editable fields 3. Click "Save"	Invalid Phone Number or empty required field	System displays validation error message and profile is not updated	Pass

8.2.1.8 View Trip History Test Case Table

Test Case ID	Use Case ID	Test Case Name	Preconditions	Test Steps	Test Data	Expected Result	Status
TC-01	UC-17	View Trip History	Driver is logged in and dashboard is loaded	1. Click "My Trips" 2. System displays active trips 3. Click "Show History" 4. System displays trip history	Existing Trip History data	System display trip history correctly,	Pass
TC-02	UC-17	No History Available	Driver is logged in and dashboard is loaded	1. Click "My Trips" 2. System shows no history trips	No trip records in system	System displays appropriate messages: "No trip history available"	Pass

8.2.1.9 Delete Citizen Test Case Table

Test Case ID	Use Case ID	Test Case Name	Preconditions	Test Steps	Test Data	Expected Result	Status
TC-01	UC-11	Delete Citizen	User is logged in, on Manage Users page, and citizen exists	1. Click "Delete" on a citizen 2. System displays confirmation message 3. Click "Delete" to confirm	Existing Citizen Record	System removes citizen from database and displays message: "Citizen deleted successfully"	Pass
TC-02	UC-11	Cancel Citizen Deletion	User is logged in, on Manage Users page, and citizen exists	1. Click "Delete" on a citizen 2. System displays confirmation message 3. Click "Cancel"	Existing Citizen Record	System keeps citizen unchanged and no deletion occurs	Pass

Example API Test Cases

Test Case	Expected Result
Login with valid credentials	JWT token returned
Login with invalid password	401 Unauthorized
Create booking with valid data	Booking successfully created
Access protected endpoint without token	401 Unauthorized
Submit complaint	Complaint stored successfully

Example Postman Request

Login API

This endpoint is used to authenticate a user by logging them into the system. It accepts user credentials and, upon successful authentication, returns a token that can be used for subsequent requests.

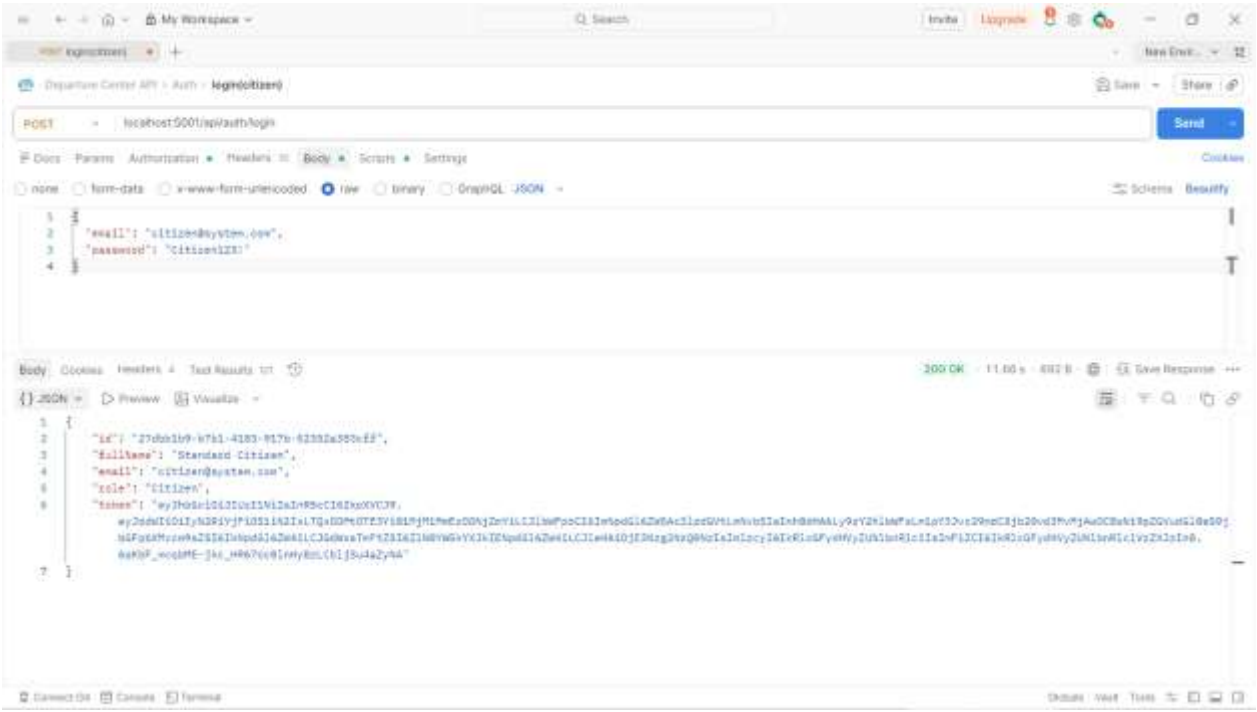
Request

- **Method**: POST
- **URL**: {{authBaseUrl}}/api/Auth/login

Request Body

The request body must be in JSON format and should include the following parameters:

- **email** (string): The email address of the user attempting to log in.
- **password** (string): The password associated with the user's account.



8.2.2 Performance and Load Testing

Performance testing was implemented using k6 to evaluate system responsiveness, scalability, and stability under concurrent traffic conditions.

k6 Testing Structure

```
k6-tests/
├── utils/
├──────────helper.js
├──────────config.js
└── scenarios/
├──────────laod.js
├──────────smoke.js
```

```
├──────────spike.js
├──────────stress.js
```

Performance Test Scenarios

The project supports multiple load-testing scenarios:

Scenario	Description
Smoke Test	Basic workflow validation
Load Test	Normal expected traffic
Stress Test	Heavy load and saturation testing
Spike Test	Sudden traffic increase and recovery testing

Test Execution

Local Execution

```
cd k6-tests & k6 run filename.js
```

Environment Variables

Variable	Description
K6_BASE_URL	API Gateway base URL
K6_USER_EMAIL	Test account email
K6_USER_PASSWORD	Test account password
K6_REGISTER	Register new user before tests
K6_SCENARIO	Selected testing scenario

Example Command

```
K6_BASE_URL=http://localhost:3000 \
K6_SCENARIO=load \
K6_USER_EMAIL=citizen@system.com \
K6_USER_PASSWORD=Citizen123! \
k6 run main.js
```

Performance Metrics

The following metrics were monitored during testing:

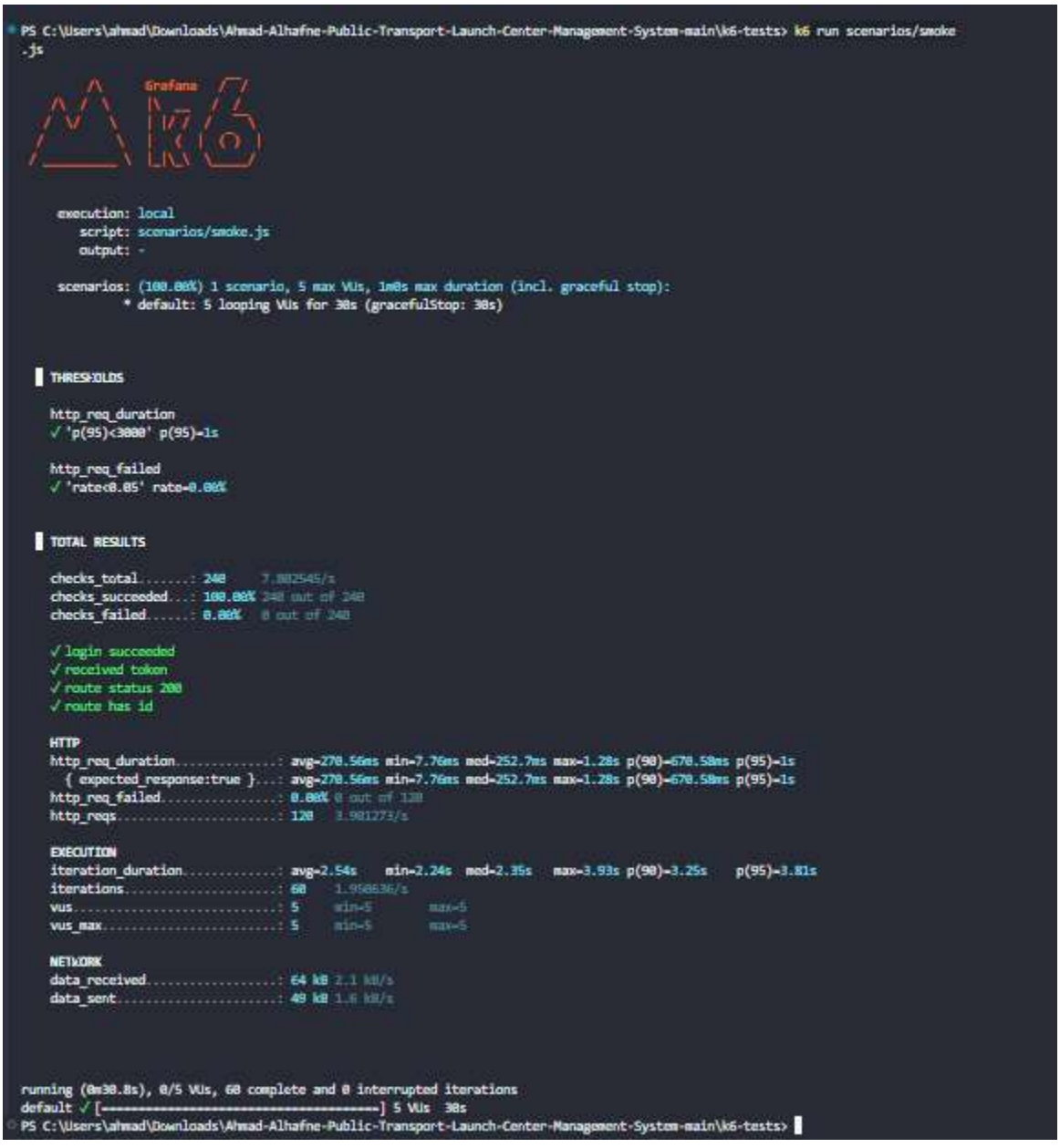
Metric	Description
http_req_duration p(95)	95th percentile response time
http_req_duration avg	Average response time
http_req_failed	Request failure percentage
checks	Business validation success rate
vus	Number of virtual users

Target Thresholds

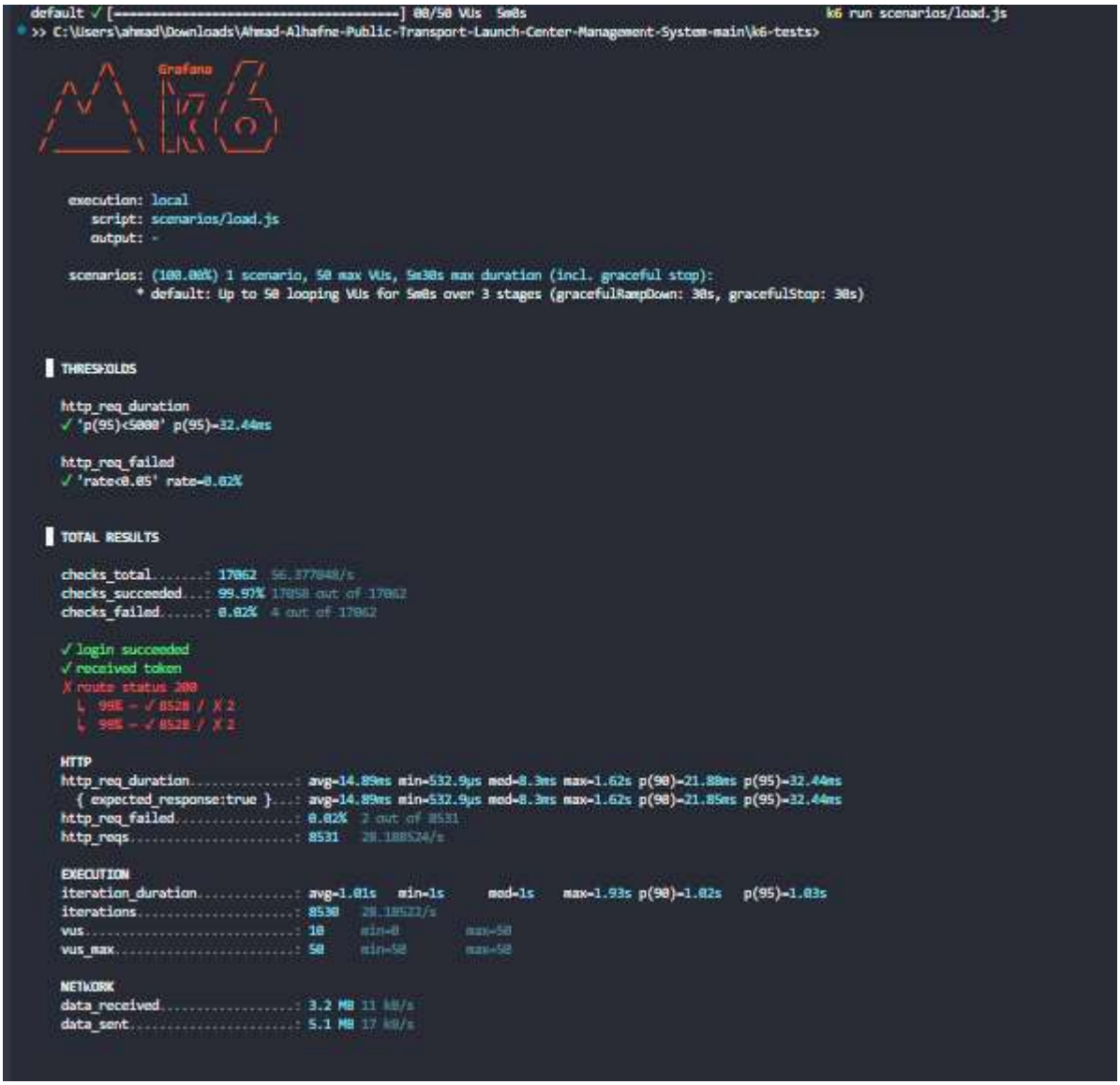
Metric	Target
p(95) Response Time	< 1s
Average Response Time	< 3s
Failed Requests	< 1%
Successful Checks	> 90%

Performance Results Placeholder

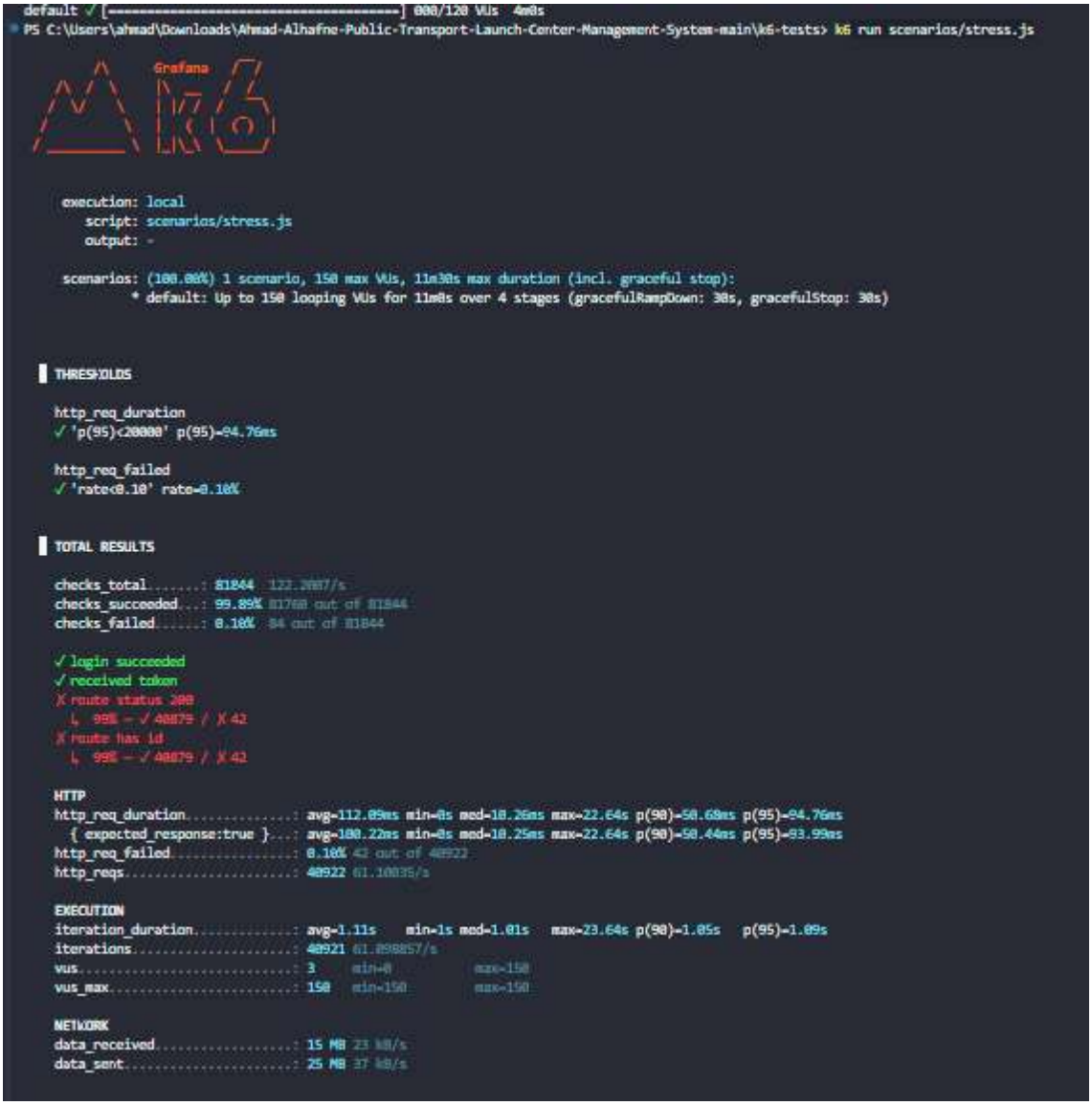
Smoke Test Results



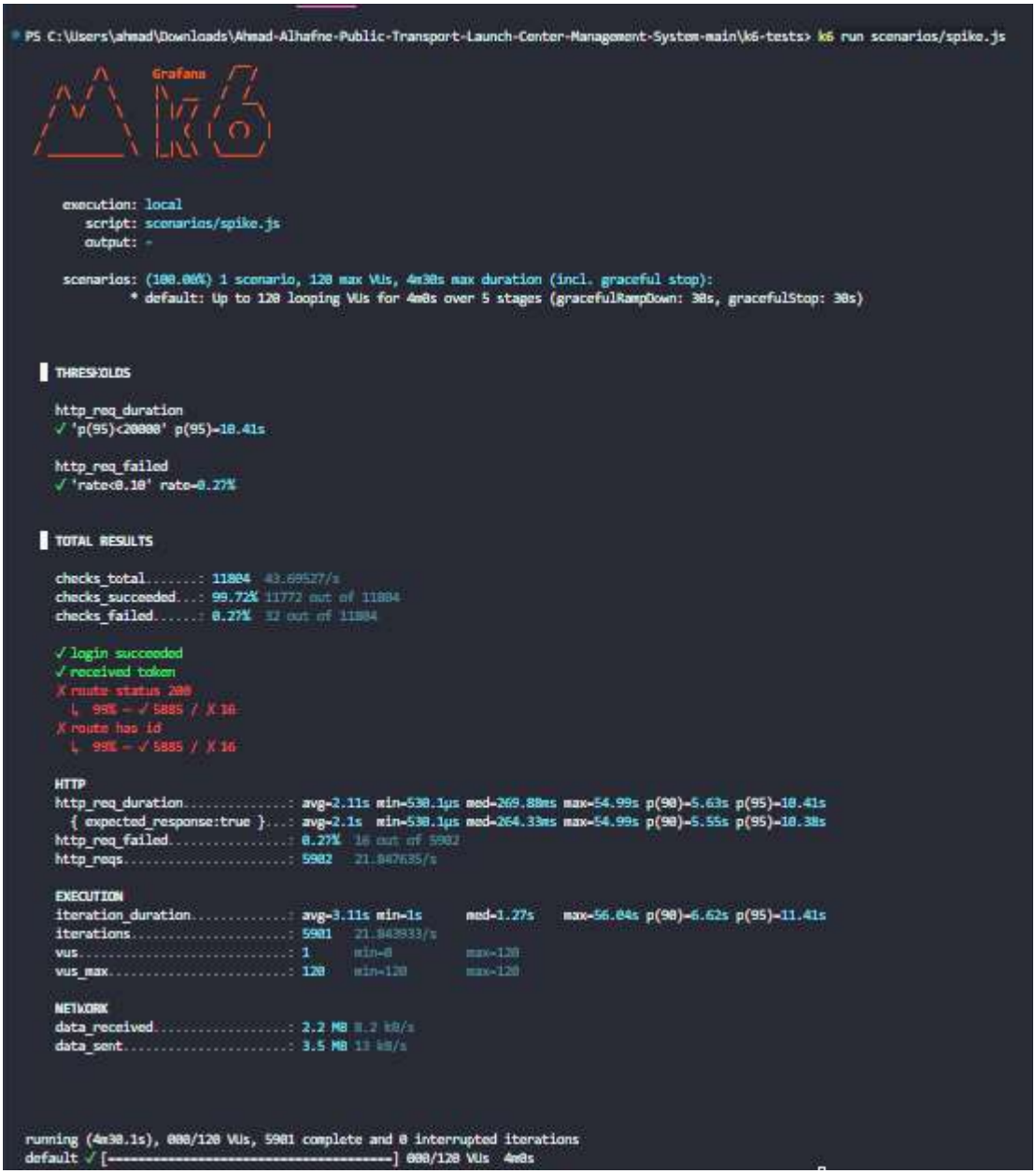
Load Test Results



Stress Test Results



Spike Test Results



8.3 Testing Environment

The testing environment was containerized using Docker Compose to ensure consistency and reproducibility across different executions.

Test Environment Components

Component	Technology
Backend Services	ASP.NET Core Microservices
Frontend	React + Vite
Database	SQL Server
API Gateway	Ocelot
Performance Testing	k6
Containerization	Docker & Docker Compose

8.4 Testing Architecture

The testing process interacted with the system through the API Gateway to simulate real-world client behavior.

Testing Tools

(Postman / k6)

↓

API Gateway (Ocelot)

↓

Backend Microservices

↓

SQL Server Databases

This approach ensured that all requests passed through the same routing, authentication, and communication layers used in production environments.

8.5 Test Results and Analysis

The testing process demonstrated that the system was capable of handling normal and high-load scenarios while maintaining acceptable response times and low failure rates.

The API Gateway successfully routed requests between services, and JWT authentication mechanisms operated correctly during concurrent access scenarios.

Performance testing also helped identify potential bottlenecks and resource-intensive operations that could be optimized in future improvements.

8.6 Quality Metrics

The project evaluation focused on several quality indicators.

Metric	Description
Response Time	Average API processing duration
Throughput	Number of handled requests per second
Failure Rate	Percentage of failed requests

8.7 Challenges During Testing

Several challenges were encountered during the testing phase:

- Managing concurrent requests across multiple microservices
- Maintaining stable communication between containers
- Handling authentication tokens during automated tests
- Monitoring performance bottlenecks during stress scenarios
- Configuring realistic load distributions for system simulation

These challenges were addressed through iterative optimization, improved Docker configurations, and reusable testing utilities inside the k6 test suite.

8.8 Summary

The testing and evaluation phase confirmed that the DepartureCenterSystem provides stable API behavior, scalable request handling, and secure communication mechanisms. The use of Postman, k6 enabled comprehensive evaluation of functionality, performance aspects of the system.

The performance testing infrastructure supports multiple traffic scenarios and the overall results indicate that the system architecture is suitable for scalable and maintainable deployment environments.

Part Five:

Result and Recomendation

Chapter 9

Result and Analysis

9.1 System Application Results

The DepartureCenterSystem successfully achieved the implementation of an integrated electronic transportation management platform based on a microservices architecture. The system provides centralized management capabilities for transportation operations while supporting passengers, administrators, and transportation staff through a unified web platform.

The implemented system enables users to browse available trips, create bookings, submit complaints, and manage transportation–related operations through secure RESTful APIs and a modern frontend interface.

Main Achieved Functionalities

The final implemented system includes the following functionalities:

Module	Achieved Functionality
Authentication System	User registration, login, JWT authentication
User Management	User profile management and authorization
Trip Management	Trip creation, scheduling, and retrieval
Booking System	Booking creation and booking history
Complaint System	Complaint submission and tracking
Vehicle Management	Vehicle registration and management
API Gateway	Centralized routing and service communication

User Interface Results

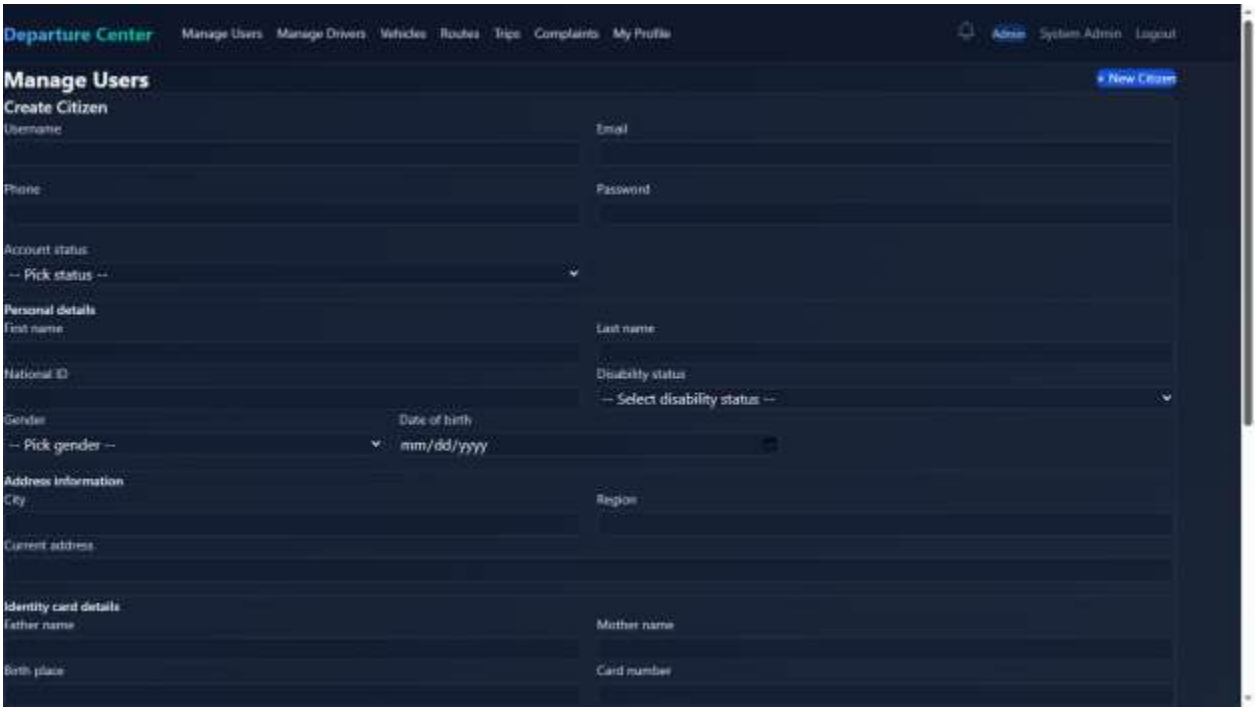
The frontend application successfully provided an interactive and user–friendly interface for managing transportation operations.

Implemented Interfaces

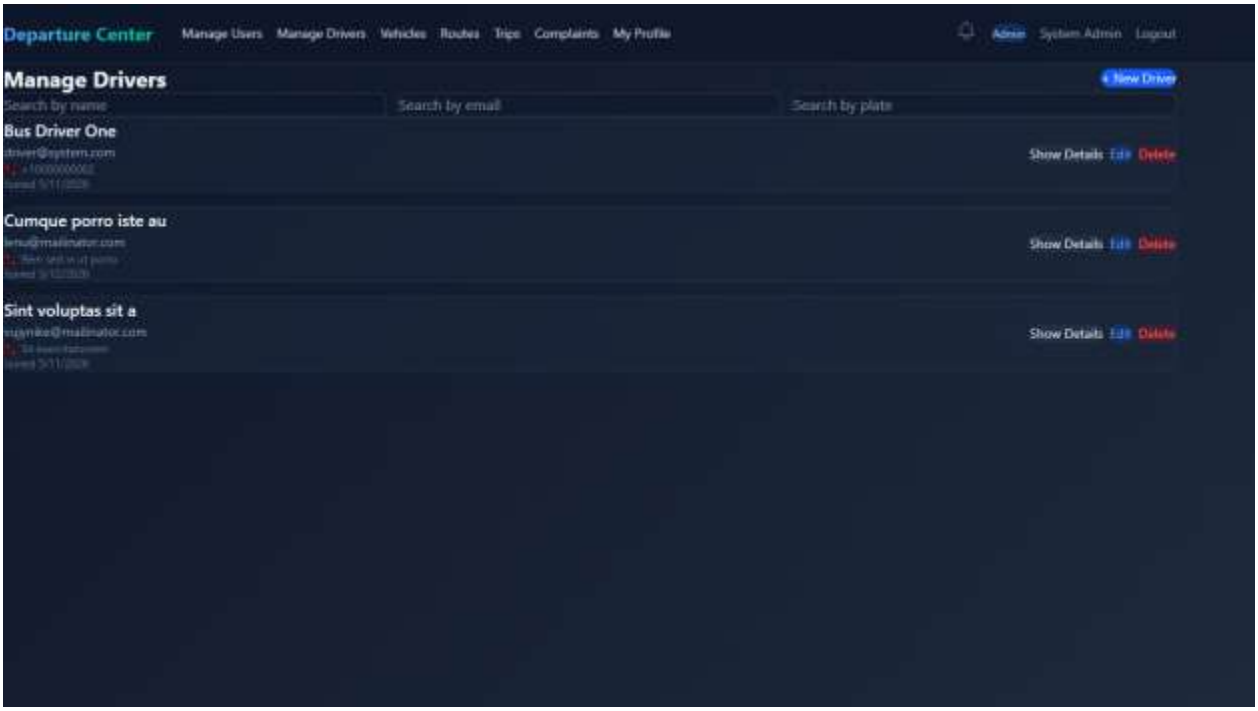
- Login and registration pages
- Trip browsing interface
- Complaint submission pages
- Administrative management dashboard

UI Results

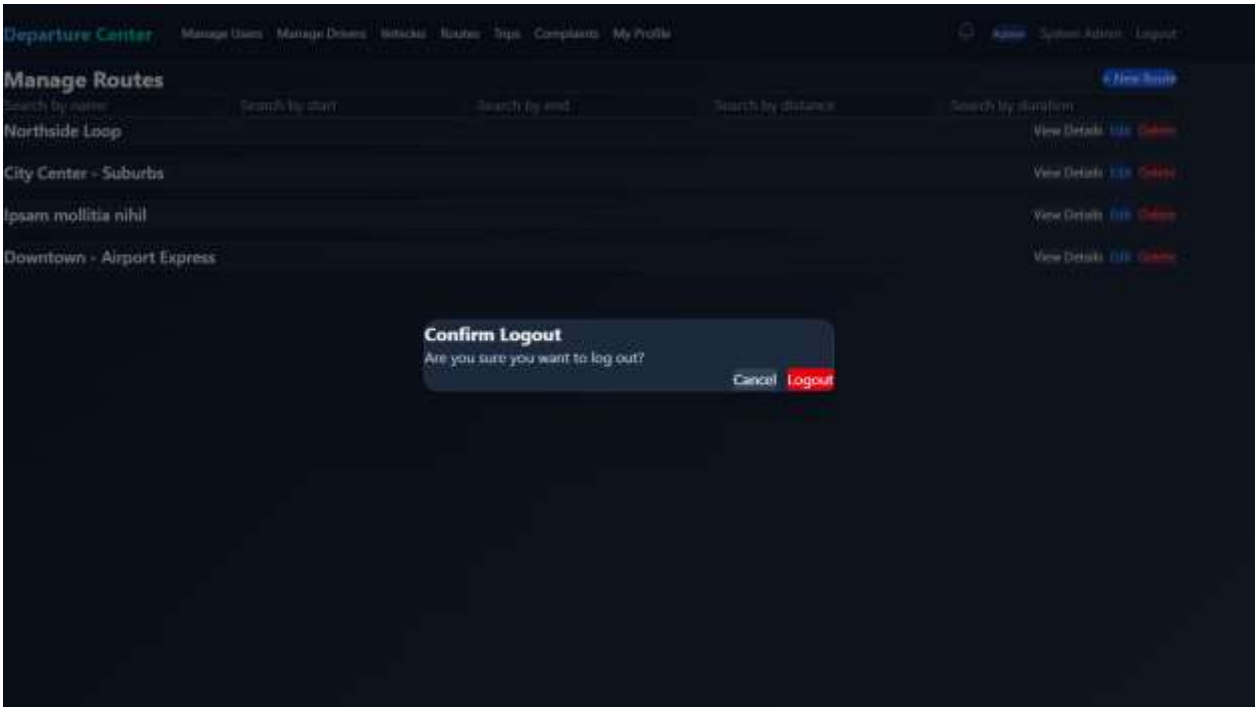
4.Admin add user



7.Admin Manage Drivers



15. Logout



API and System Results

The backend services successfully communicated through the API Gateway using REST APIs and JWT authentication mechanisms.

Achieved Backend Features

- Independent microservice deployment
- Service isolation using separate databases
- Secure JWT–based authentication
- Role–based authorization
- Docker containerization support
- Scalable service communication

9.2 Comparative Analysis with Existing Systems

We currently don't have any Analysis report to our steady application.

9.3 Achievement of Project Objectives

The project objectives defined in Chapter 1 were reviewed and evaluated against the final implementation results.

Main Objective Evaluation

Objective 1– Create an integrated information system for transportation center management

Achieved successfully through the implementation of multiple integrated backend services and centralized management capabilities.

Objective 2 –Develop an platform that facilitates travel between cities

Achieved through trip management, booking workflows, and route management functionalities.

Objective 3– Improve public transportation services

Partially achieved through automation of booking operations, complaint management, and centralized transportation management.

Future improvements such as live tracking and real–time notifications can further enhance this objective.

Technical Objectives Achievement

Technical Objective	Status
Microservices Architecture	Achieved
API Gateway Integration	Achieved
JWT Authentication	Achieved
Docker Containerization	Achieved
SQL Server Integration	Achieved
Performance Testing	Achieved
Frontend Integration	Achieved

9.4 Strengths and Weaknesses Analysis

System Strengths

The developed system includes several important strengths:

1. **Scalable Architecture**

The microservices architecture enables independent deployment and scaling of services without affecting the entire system.

2. **Maintainability**

The use of Clean Architecture improves code organization and separation of concerns, making maintenance easier.

3. **Security**

JWT authentication and role-based authorization provide secure access control.

4. **Flexibility**

The modular service design allows future extensions and additional services to be integrated easily.

5. **Containerization Support**

Docker support simplifies deployment and testing across different environments.

6. **Performance Testing Infrastructure**

The integration of k6 performance testing enables continuous monitoring of system scalability and stability.

System Weaknesses

Despite the achieved results, several limitations and weaknesses were identified.

1. Limited Real-Time Features

The current implementation does not fully support live vehicle tracking or real-time trip updates.

2. Absence of Mobile Application

The system currently focuses on a web-based platform without dedicated mobile applications.

3. Limited AI Integration

Although the architecture supports future intelligent features, the current implementation does not yet include advanced AI-based prediction or recommendation models.

4. Infrastructure Complexity

Microservices architectures increase deployment and monitoring complexity compared to monolithic systems.

5. Limited Production-scale Testing

Testing was primarily performed in development and simulated environments rather than large-scale real-world deployments.

9.5 Key Performance Indicators (KPIs)

Several KPIs were used to evaluate the system performance

Performance KPIs

KPI	Source
94.76 ms average response	**Stress test sustained high load**
10.41 s p95	**Worst-case spike latency**
150 VUs	**Maximum tested concurrency**
0.27% failed requests	**Highest observed failure rate**
100% auth success	**Smoke test login checks passed**

System Quality KPIs

KPI	Description
Service Availability	Stability of backend services
API Reliability	Successful request processing
Scalability	Ability to handle increasing load
User Experience	Ease of navigation and interaction

KPI Analysis

The collected metrics indicate that the system maintained stable API responses and low failure rates under normal and moderate load scenarios. The testing infrastructure also demonstrated the ability of the platform to recover from traffic spikes and concurrent request loads.

The modular architecture contributed to better service isolation and operational stability during testing scenarios.

9.6 General Analysis

The final implementation demonstrates that the DepartureCenterSystem successfully provides an integrated transportation management platform based on modern software engineering practices.

The project combines microservices architecture, Clean Architecture principles, RESTful APIs, JWT authentication, Docker containerization, and performance testing methodologies into a unified system capable of supporting scalable transportation management operations.

The conducted tests and comparisons indicate that the system provides a strong technical foundation for future expansion toward more advanced smart transportation solutions.

Chapter 10
Conclusion
&
Recommendations

10-1 Conclusion

At the conclusion of this research, and thanks to God, an integrated system for managing public transport departure centers has been developed. The primary goal was to automate administrative and field operations within departure centers. The project addressed the system's lifecycle, starting from studying the current situation and analyzing functional and non-functional requirements, to software design and the implementation of user interfaces serving the three stakeholders (the manager, the driver, and the citizen).

The system helps solve issues of chaotic trip organization and provides managers with effective supervisory tools to monitor the fleet and financial reports, in addition to improving passenger experience through real-time tracking and trip search features. This work represents a step towards digital transformation in the public transport sector, enhancing the efficiency of services provided to citizens.

10-2 Difficulties and challenges

Dynamics of requirements and data structuring: Difficulty emerged in converting traditional and complex paper-based mechanisms for departure garages into flexible software algorithms, especially in organizing the overlap between trip schedules, driver availability, vehicle conditions, and ensuring no data conflicts occur (data consistency).

Resource management and time constraints: Reconciling the massive scale of the required system (user management, complaints, and trips) with the limited project timeline was a real test of the team's ability to manage priorities and allocate technical tasks effectively under deadline pressure.

10-3 Future Work

This system does not represent the end, but rather a cornerstone that can be developed in the future through the following prospects:

Electronic Payment Integration: Enabling ticket booking and payment via electronic payment gateways to facilitate the process and reduce cash handling.

Use of Artificial Intelligence: Integrating prediction algorithms to forecast bus arrival times more accurately based on historical traffic patterns, and smartly distribute trips to reduce congestion.

Dedicated Mobile Application for Drivers: Expanding the driver app's functions to include integrated navigation (GPS) and smart safety alerts.

Periodic Maintenance Management System: Adding a software module that notifies the manager of vehicle maintenance schedules based on kilometers recorded in the system.

10-4Recommendations for institutions:

adopt smart systems to reduce waste and prevent price manipulation, and rely on the analytical reports provided by the system to make data-driven decisions. For students: focus on studying similar systems globally before starting the design, and adhere to agile development methodologies (Iterative Incremental) to ensure continuous delivery of functional components.

Section Six: Appendices and References

Appendix A: Requirement Collection Questionnaires and Interview Outcomes

The requirements gathering process relied on a mixed methodology to ensure system comprehensiveness, and its results can be summarized as follows:

Field Interviews: A series of meetings were held with stakeholders, primarily the director of the Daraa Garage, to understand the traditional mechanism used in managing shifts, distributing trips, and documented driver and vehicle information.

Key Questions Raised: These focused on how congestion is addressed, the mechanism for citizens to submit complaints, and the methods for verifying vehicles' legal papers.

Questionnaire Outcomes: The study concluded the necessity of having a central system that connects the director, driver, and citizen in a single digital loop to ensure transparency and reduce time wastage.

Deriving Requirements: The questions posed during the analysis phase focused on:

How is the "role" organized and how are trips currently distributed?

What are the gaps in the paper-based complaint system?

What are the essential data that must be available in the record of each driver and bus?

Appendix B: Detailed Design

B-1: Main Use Case Diagrams:

The system includes comprehensive diagrams covering:

Driver Management: (Add a new driver, edit data, verify driving license).

Bus Management: (Add a bus, delete a bus, and update its status between 'Available' or 'On Trip').

Complaint System: (Submit a complaint by the citizen, and handle it by the manager).

B-2: Activity Diagrams:

Workflows were drawn for each function within the system to ensure smooth transitions between interfaces, starting from login to executing complex tasks such as trip management.

Appendix C: Test Case Descriptions

Test cases were documented to ensure system quality, and here are examples of actual records included in the project:

Driver Management:
TC-01: Successfully add a new driver (Expected: Saved successfully).
TC-02: Attempt to add a duplicate license number (Expected: Unique Constraint Error).

Bus Management:

TC-06: Add a new bus (Expected: Bus added with status Available).

TC-07: Delete an available bus (Expected: Bus removed from list).

Complaint System:

TC-08: Submit a complaint (Expected: Saved as Pending with tracking number).

TC-09: Respond to the complaint (Expected: Status becomes Replied).

Appendix D: User Manual

A guide for using the digital system:

Personal Account Management: Instructions on how to change the password (TC-10) and update profile data.

Operations Management (for the manager): How to access control interfaces for drivers and buses and review reports.

Public Interaction: Steps for a citizen to submit a complaint and track the response status using the reference number.

First: Reference Studies and Similar Applications (Reference Study)

A set of leading global systems and platforms in the field of transportation management and logistics services were studied and analyzed to derive functional requirements and compare them:

Careem & Lyft Platforms: Studying driver management interfaces and intelligent ride organization mechanisms.

Moovit & City mapper Applications: Analyzing public transportation routing systems and the user experience in tracking schedules.

Transit App: A reference on how to display routes and schedule departure trips for public vehicles.

Second: Tools and Technologies Used (Tools & Technologies)

These references rely on the software tools and management platforms employed to build and document the project:

GitHub Projects: The main reference for managing source code and tracking the team's programming tasks.